

IF-ITB-IF3250-PPL-TD-2026

DOKUMEN PEMBANGUNAN PERANGKAT LUNAK

Technical Document

Aplikasi Penggajian Terintegrasi

Versi: Draft-29/05/2026

untuk:

PT LAPI Divusi

Dipersiapkan oleh:

Kelompok 04

Ahmad Wafi Idzharulhaqq	13523131
Anas Ghazi Al Gifari	13523159
Benedictus Nelson	13523150
Muhammad Farrel Wibowo	13523153
Naufarrel Zhafif Abhista	13523149

Program Studi Teknik Informatika - Institut Teknologi Bandung

Jl. Ganesha 10, Bandung 40132

	Nomor Dokumen		Halaman
	<i>IF-ITB-IF3250-PPL-TD-2026</i>		9
	Revisi	03	Tgl: 29/05/2026

Daftar Isi

1 Pendahuluan	5
1.1 Tujuan Penulisan Dokumen	6
1.2 Pemilik Perangkat Lunak	6
1.3 Manfaat Perangkat Lunak	6
1.4 Aturan Penomoran	7
Tabel I. Aturan Penomoran Dokumen Teknis	7
1.5 Referensi	7
1.6 Deskripsi Umum Dokumen (Ikhtisar)	7
2 Proses Kerja / Proses Bisnis	8
2.1 Unit Organisasi	9
2.2 Gambaran Proses Bisnis	9
Gambar 2.1 Diagram Pengelolaan Data Master dan Kepegawaian	10
Gambar 2.2 Diagram Penggajian Bulanan dan Pelaporan	12
2.3 Deskripsi Kebutuhan Perangkat Lunak	13
3 Kebutuhan Perangkat Lunak	15
3.1 Deskripsi Umum Sistem	15
Gambar 3.1 Diagram Blok Umum Aplikasi Penggajian Terintegrasi	16
3.2 Stakeholder	17
Tabel 3.1 Daftar Stakeholder Aplikasi Penggajian	17
3.3 Identifikasi Kebutuhan Perangkat Lunak	17
3.3.1 Product Backlog	17
Tabel 3.2 Product Backlog Aplikasi Penggajian	17
3.3.1.1 Tabel Use Case	19
Tabel 3.3 Daftar Use Case Aplikasi Penggajian	20
3.3.1.2 Diagram Use Case	21
Gambar 3.2 Use Case Diagram Sistem HR & Payroll	21
4 Deskripsi Perancangan Perangkat Lunak	21
4.1 Deskripsi Arsitektur	21
4.2 Dekomposisi Fungsional	22
Tabel 4.1 Dekomposisi Fungsional Subsistem dan Komponen	23
4.3 Architectural View	24
4.3.1 Structural View	24
4.3.1.1 Diagram Komponen	25
Gambar 4.2 Diagram Komponen Aplikasi Penggajian Terintegrasi	25
4.3.1.2 Diagram Kelas	27
Gambar 4.3 Diagram Kelas Aplikasi Penggajian Terintegrasi	27
4.3.1.3 Traceability Table	28
Tabel 4.2 Traceability Kelas terhadap Komponen Perancangan	28
4.3.2 Behavioural View	29
4.3.2.1 Skenario Use Case 01	29

Gambar 4.4 Sequence Diagram UC-01 Login	31
4.3.2.2 Skenario Use Case 02	31
Gambar 4.5 Sequence Diagram UC-02 Logout	33
4.3.2.3 Skenario Use Case 03	33
Gambar 4.6 Sequence Diagram UC-03 Pindah Perusahaan Aktif	34
4.3.2.4 Skenario Use Case 04	35
Gambar 4.7 Sequence Diagram UC-04 Kelola User	35
4.3.2.5 Skenario Use Case 05	35
Gambar 4.8 Sequence Diagram UC-05 Kelola Hak Akses User	37
4.3.2.6 Skenario Use Case 06	37
Gambar 4.9 Sequence Diagram UC-06 Kelola Perusahaan	39
4.3.2.7 Skenario Use Case 07	39
Gambar 4.10 Sequence Diagram UC-07 Kelola Struktur Organisasi	41
4.3.2.8 Skenario Use Case 08	41
Gambar 4.11 Sequence Diagram UC-08 Kelola Data Karyawan	43
4.3.2.9 Skenario Use Case 09	43
Gambar 4.12 Sequence Diagram UC-09 Kelola Komponen Gaji	45
4.3.2.10 Skenario Use Case 10	45
Gambar 4.13 Sequence Diagram UC-10 Proses Penggajian	47
4.3.2.11 Skenario Use Case 11	47
Gambar 4.14 Sequence Diagram UC-11 Konfirmasi / Finalisasi Penggajian	49
4.3.2.12 Skenario Use Case 12	49
Gambar 4.15 Sequence Diagram UC-12 Assign Pegawai ke Perusahaan	50
4.3.2.13 Skenario Use Case 13	50
Gambar 4.15 Sequence Diagram UC-13 Lihat Slip Gaji	51
4.3.2.14 Skenario Use Case 14	51
Gambar 4.16 Sequence Diagram UC-14 Lihat Slip Gaji	53
4.3.2.15 Skenario Use Case 15	53
Gambar 4.17 Sequence Diagram UC-15 Evaluasi Formula dengan Sandbox	55
4.3.2.16 Skenario Use Case 16	55
Gambar 4.18 Sequence Diagram UC-16 Kelola Kehadiran	57
4.3.2.17 Skenario Use Case 17	57
Gambar 4.19 Sequence Diagram UC-17 Kelola Lookup Variabel Komponen Gaji	58
4.3.2.18 Skenario Use Case 18	58
Gambar 4.20 Sequence Diagram UC-18 Analisis Varians Penggajian	59
4.3.2.19 Skenario Use Case 19	59
Gambar 4.21 Sequence Diagram UC-18 Verifikasi Slip Gaji melalui QR Code	61
4.3.3 Deployment View	61
4.3.3.1 Ringkasan	61
4.3.3.2 Kebutuhan Perangkat Keras dan Perangkat Lunak	62
4.3.3.3 Diagram Deployment (Arsitektur Fisik)	63
Gambar 4.3.3.3. Deployment Diagram	63
4.3.3.4 Alur Kerja Operasional dan Komunikasi Sistem	63

5 Implementasi Perangkat Lunak	64
5.1 Struktur Direktori Implementasi	64
5.2 Traceability Table	68
Tabel 5.1 Traceability Table Berkas Implementasi terhadap Komponen Perancangan	68
6 Pengujian Perangkat Lunak	74
6.1 Tujuan Pengujian	74
6.2 Rencana Pengujian	74
6.2.1 Jenis Pengujian	74
Tabel 6.1 Jenis Pengujian	75
6.2.2 Perangkat dan Lingkungan Pengujian	75
Tabel 6.2 Perangkat dan Lingkungan Pengujian	75
6.2.3 Perintah Pengujian	75
Tabel 6.3 Perintah Pengujian	76
6.2.4 Strategi Pengujian	76
6.2.5 Integrasi Berkelanjutan (CI/CD)	76
6.3 Kasus Uji	76
Tabel 6.4 Kasus Uji Fungsional	76
6.4 Pengujian Setiap Sprint	78
Tabel 6.6 Pengujian Setiap Sprint	78
6.5 Hasil Pengujian	79
6.5.1 Hasil Pengujian Unit	79
Gambar 6.1 Hasil Eksekusi Pengujian Unit Menggunakan Jest	79
6.5.2 Hasil Pengujian Integrasi	79
Gambar 6.2 Hasil Eksekusi Pengujian Integrasi Menggunakan Jest	80
6.5.3 Cakupan Kode	80
Gambar 6.3 Ringkasan Cakupan Kode Unit Test	80
Tabel 6.7 Ringkasan Cakupan Kode Unit Test	80
6.5.4 Ringkasan Hasil Pengujian	80
Tabel 6.9 Ringkasan Hasil Pengujian	81
7 Petunjuk Instalasi Perangkat Lunak	81
7.1 Kebutuhan Lingkungan	81
7.2 Persiapan Kode Sumber	82
7.3 Konfigurasi Environment	82
7.4 Instalasi Backend	82
7.5 Instalasi Frontend	82
7.6 Konfigurasi Basis Data	82
7.7 Menjalankan Aplikasi	82
7.8 Menjalankan Pengujian	83
7.9 Penanganan Masalah Umum	83
Tabel 7.1 Penanganan Masalah Umum Instalasi	83
8 Panduan Penggunaan Perangkat Lunak	83
8.1 Login dan Pemilihan Perusahaan	83
8.2 Pengelolaan Perusahaan dan Pengguna	84

8.2.1 Mendaftarkan Perusahaan Baru	84
8.2.2 Membuat Akun Pengguna	84
8.2.3 Menugaskan pengguna ke perusahaan	84
8.2.4 Mengelola Integration API Key	84
8.3 Pengelolaan Data Master	84
8.3.1 Mengelola Departemen dan Jabatan	84
8.3.2 Mengelola Employee Grade dan Tunjangan Masa Kerja	84
8.3.3 Mengelola Master Komponen Gaji	84
8.3.4 Mengelola Tabel Referensi	85
8.3.5 Mengonfigurasi Cutoff Penggajian	85
8.4 Pengelolaan Data Karyawan	85
8.4.1 Menambah karyawan baru	85
8.4.2 Mengedit data karyawan	85
8.4.3 Mengunggah dokumen karyawan	85
8.4.4 Menyalin komponen gaji ke karyawan	85
8.4.5 Menghapus karyawan	86
8.5 Pengelolaan Data Kehadiran	86
8.6 Eksekusi Penggajian Bulanan	86
8.6.1 Menjalankan payroll baru	86
8.6.2 Melihat rincian payroll periode sebelumnya	86
8.7 Rekap dan Validasi Penggajian	86
8.7.1 Memvalidasi integritas rekap	86
8.7.2 Mengekspor rekap	87
8.8 Analisis Variansi Penggajian	87
8.9 Slip Gaji Karyawan (Self-Service)	87
8.10 Verifikasi Keaslian Slip Gaji	87
8.11 Pengelolaan Log Aktivitas	87

1 Pendahuluan

1.1 Tujuan Penulisan Dokumen

Dokumen ini disusun untuk memberikan gambaran komprehensif mengenai teknis pengembangan Perangkat Lunak Penggajian. Perangkat lunak ini merupakan sistem berbasis web yang dirancang untuk mengelola proses penggajian karyawan secara otomatis dan terintegrasi dengan formula yang variatif dan fleksibel untuk setiap karyawan dan integrasi dengan aplikasi HRD yang sudah ada. Dokumen ini mencakup gambaran proses bisnis, kebutuhan perangkat lunak, perancangan sistem, implementasi, pengujian, hingga panduan penggunaan. Selain itu, dokumen ini dimaksudkan sebagai pedoman bagi seluruh tim pengembang dan pemangku kepentingan dalam memahami ruang lingkup proyek.

1.2 Pemilik Perangkat Lunak

Pemilik perangkat lunak ini adalah Bapak Tony Fadjar dari perusahaan PT LAPI Divusi.

1.3 Manfaat Perangkat Lunak

Perangkat Lunak Penggajian dirancang untuk membantu perusahaan dalam mengelola proses penggajian karyawan secara lebih efisien, akurat, dan transparan. Secara umum, manfaat yang ditawarkan antara lain:

1. **Otomatisasi perhitungan gaji**, termasuk komponen tunjangan, potongan, pajak penghasilan (PPh 21), dan iuran BPJS, sehingga meminimalkan kesalahan perhitungan manual.
2. **Pengelolaan data kepegawaian yang terpusat**, memudahkan pencatatan dan pembaruan data karyawan yang berkaitan dengan penggajian.
3. **Transparansi penggajian**, dengan menyediakan slip gaji digital yang dapat diakses langsung oleh karyawan.
4. **Efisiensi proses persetujuan**, melalui alur persetujuan penggajian yang terstruktur antar departemen terkait.
5. **Pelaporan keuangan yang akurat**, mendukung pembuatan laporan penggajian periodik untuk keperluan audit dan pengambilan keputusan manajemen.

1.4 Aturan Penomoran

Aturan penomoran dan penamaan artefak dokumen teknis ditetapkan seperti pada Tabel 1.1.

Tabel 1. Aturan Penomoran Dokumen Teknis

Penomoran/Penamaan	Gaya Penomoran	Keterangan
Product Backlog	PB-XX	PB-XX adalah ID Product Backlog dengan nomor urut XX pada tabel Product Backlog
Sprint Backlog	SY-XX	SY-XX adalah ID Sprint Backlog pada Sprint ke-Y dengan nomor urut XX
Use Case	UC-XX	UC-XX adalah ID Use Case dengan nomor urut XX pada tabel Use Case
Functional Requirement	FR-XX	FR-XX adalah ID kebutuhan fungsional dengan nomor urut XX
Non-functional Requirement	NFR-XX	NFR-XX adalah ID kebutuhan nonfungsional dengan nomor urut XX

1.5 Referensi

Referensi yang digunakan selama pembuatan dokumen dan pengembangan perangkat lunak ini mencakup:

1. Modul perkuliahan mata kuliah IF3250 Proyek Perangkat Lunak, Program Studi Teknik Informatika ITB.
2. Modul perkuliahan mata kuliah WI2022 Manajemen Proyek, Program Studi Teknik Informatika ITB.
3. Modul perkuliahan mata kuliah IF2150 Rekayasa Perangkat Lunak, Program Studi Teknik Informatika ITB.
4. Peraturan Direktur Jenderal Pajak Nomor PER-16/PJ/2016 tentang Pedoman Teknis Tata Cara Pemotongan, Penyetoran, dan Pelaporan Pajak Penghasilan Pasal 21.
5. Peraturan Pemerintah Nomor 36 Tahun 2021 tentang Pengupahan.

1.6 Deskripsi Umum Dokumen (Ikhtisar)

Dokumen ini disusun secara sistematis dalam beberapa bab sebagai berikut:

Bab 1 adalah pendahuluan yang memuat tujuan penulisan dokumen, pemilik perangkat lunak, manfaat perangkat lunak, aturan penomoran, referensi, serta deskripsi umum dokumen.

Bab 2 membahas proses kerja dan proses bisnis organisasi terkait, mencakup unit organisasi yang terlibat, gambaran alur proses bisnis penggajian, serta deskripsi kebutuhan perangkat lunak secara umum.

Bab 3 menguraikan kebutuhan perangkat lunak, meliputi deskripsi umum sistem, daftar stakeholder yang terlibat, serta hasil identifikasi kebutuhan dalam bentuk Product Backlog dan Sprint Backlog.

Bab 4 menjelaskan deskripsi perancangan perangkat lunak, yang mencakup deskripsi arsitektur sistem, dekomposisi fungsional, serta berbagai sudut pandang arsitektur meliputi Structural View, Behavioural View, dan Deployment View.

Bab 5 memuat implementasi perangkat lunak, termasuk struktur folder dan file implementasi beserta matriks ketertelusuran terhadap kebutuhan.

Bab 6 berisi hasil pengujian perangkat lunak, mencakup kasus uji, pra-kondisi, aksi pengujian, keluaran yang diharapkan, dan hasil pengujian aktual.

Bab 7 merupakan panduan instalasi perangkat lunak bagi administrator sistem untuk melakukan deployment aplikasi.

Bab 8 menyajikan panduan penggunaan perangkat lunak bagi seluruh kategori pengguna, mencakup berbagai skenario penggunaan yang mungkin terjadi.

2 Proses Kerja / Proses Bisnis

2.1 Unit Organisasi

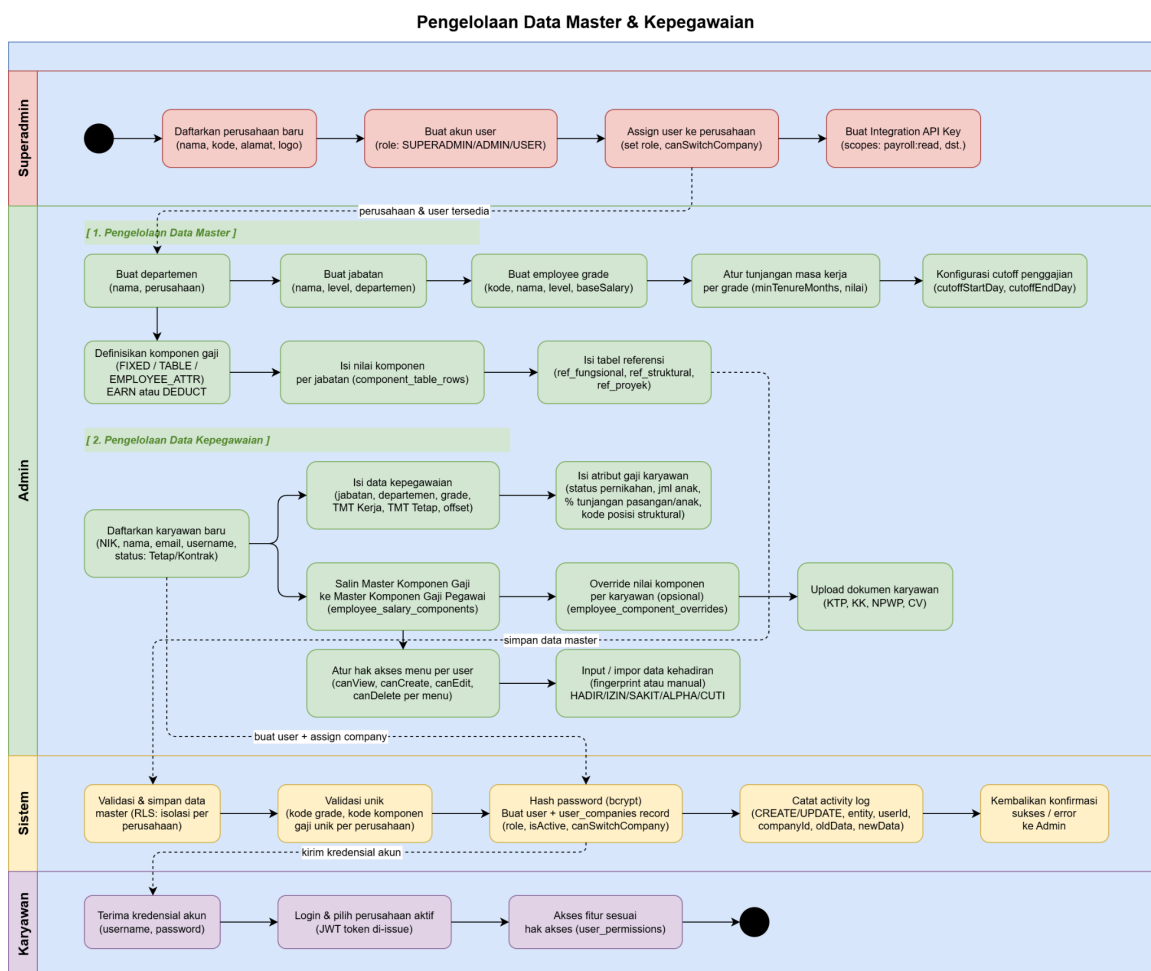
Sistem penggajian ini digunakan oleh beberapa unit dalam perusahaan untuk mendukung pengelolaan data kepegawaian serta proses penggajian. Sistem ini akan digunakan oleh unit-unit perusahaan berikut:

1. Administrator Sistem
2. User/Karyawan Biasa

Unit HRD bertanggung jawab dalam pengelolaan data karyawan. Unit Finance bertanggung jawab dalam pengelolaan komponen gaji serta proses penggajian bulanan. Administrator sistem memiliki tanggung jawab dalam pengelolaan akun pengguna serta hak akses sistem. Sementara itu, karyawan menggunakan sistem untuk melihat slip gaji mereka secara mandiri

2.2 Gambaran Proses Bisnis

Sistem penggajian memiliki empat proses bisnis utama yang saling berkaitan, yaitu pengelolaan data master, pengelolaan data kepegawaian, proses penggajian bulanan, serta pemantauan dan pelaporan. Proses pengelolaan data master dan kepegawaian disajikan pada Gambar 2.1, sedangkan proses penggajian bulanan dan pelaporan disajikan pada Gambar 2.2.



Gambar 2.1 Diagram Pengelolaan Data Master dan Kepegawaian

Gambar 2.1 menampilkan alur kerja sama antara Superadmin, Admin, Sistem, dan Karyawan dalam menyiapkan data master dan data kepegawaian sebelum penggajian dapat dijalankan. Penjelasan tiap proses diuraikan berikut ini.

1. Pengelolaan Data Master

Proses ini melibatkan dua peran utama, yaitu Superadmin dan Admin, dan harus diselesaikan sebelum sistem dapat digunakan untuk penggajian. Superadmin memulai proses dengan mendaftarkan perusahaan baru ke dalam sistem, mencakup nama, kode, alamat, dan logo. Selanjutnya, Superadmin membuat akun pengguna dengan penetapan peran, kemudian mengaitkan akun tersebut ke perusahaan yang sesuai beserta konfigurasi `canSwitchCompany`. Superadmin juga bertanggung jawab membuat Integration API Key untuk kebutuhan integrasi dengan sistem HRD eksternal dengan cakupan akses (scopes) yang dapat dikonfigurasi.

Setelah perusahaan dan akun tersedia, Admin melanjutkan dengan membangun struktur organisasi melalui pembuatan departemen dan jabatan. Admin kemudian mendefinisikan grade karyawan beserta tabel tunjangan masa kerja per grade yang mengatur nilai tunjangan berdasarkan ambang batas masa kerja minimum. Selanjutnya, Admin mendefinisikan komponen-komponen gaji dengan tipe yang tersedia beserta formula perhitungannya. Selain itu, Admin juga mengisi tabel referensi pendukung dan mengatur konfigurasi cutoff penggajian.

Sistem merespons setiap operasi dengan memvalidasi data, menerapkan isolasi data per perusahaan menggunakan Row Level Security (RLS), menegakkan unique constraint (misalnya kode grade dan kode komponen gaji harus unik per perusahaan), serta mencatat setiap perubahan ke dalam activity log.

Masukan proses ini adalah data perusahaan, struktur organisasi, definisi grade, formula komponen gaji, dan tabel referensi. Keluarannya adalah master data yang siap digunakan sebagai acuan dalam proses penggajian.

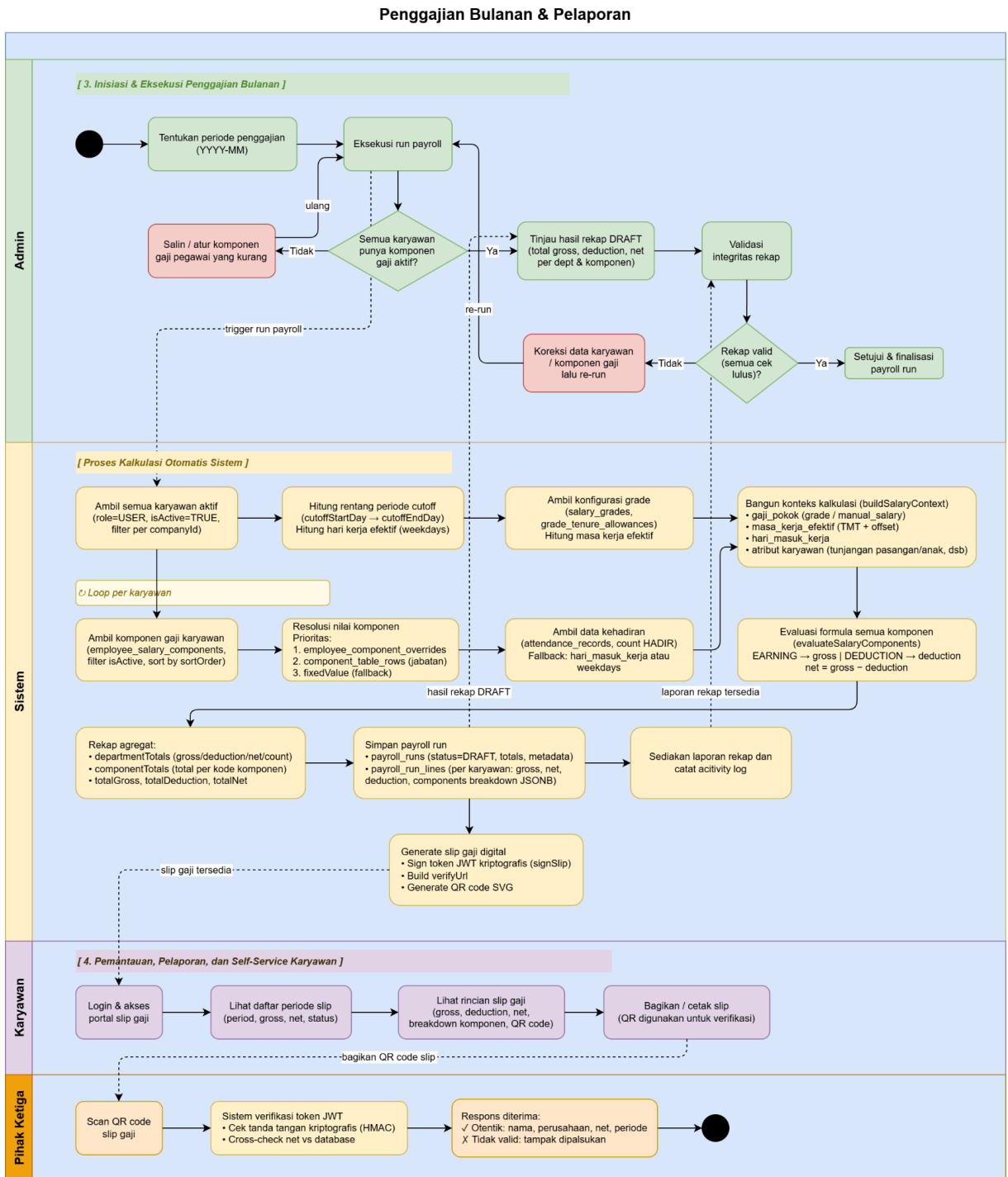
2. **Pengelolaan Data Kepegawaian**

Proses ini dilakukan oleh Admin dan berlanjut setelah data master tersedia. Admin mendaftarkan karyawan baru dengan mengisi data identitas (NIK, nama, email, username, status karyawan Tetap/Kontrak). Selanjutnya, Admin melengkapi data kepegawaian yang memengaruhi perhitungan masa kerja, yaitu jabatan, departemen, grade, TMT Kerja, TMT Tetap, dan nilai offset masa kerja. Admin kemudian mengisi atribut gaji karyawan yang digunakan dalam formula, meliputi jenis kelamin, status pernikahan, jumlah anak, persentase tunjangan pasangan dan anak, serta kode posisi struktural.

Setelah data karyawan lengkap, Admin menyalin Master Komponen Gaji ke Master Komponen Gaji Pegawai agar komponen gaji berlaku untuk karyawan tersebut. Apabila karyawan memiliki kesepakatan nilai yang berbeda dari nilai jabatan atau nilai tetap, Admin dapat melakukan override nilai komponen secara individual. Admin juga dapat mengunggah dokumen pendukung karyawan (KTP, KK, NPWP, CV) serta mengatur hak akses menu per pengguna. Selain itu, Admin menginput atau mengimpor data kehadiran harian karyawan yang dapat bersumber dari mesin fingerprint maupun pencatatan manual, dengan status HADIR, IZIN, SAKIT, ALPHA, atau CUTI.

Sistem memproses pembuatan karyawan dengan melakukan hash password menggunakan bcrypt, membuat record users sekaligus user_companies, dan mencatat seluruh aktivitas ke dalam activity log. Karyawan yang telah terdaftar kemudian dapat menerima kredensial akun, login ke sistem, memilih perusahaan aktif untuk mendapatkan JWT token, dan mengakses fitur sesuai hak yang ditetapkan.

Masukan proses ini adalah data identitas dan kepegawaian karyawan, dokumen pendukung, serta data kehadiran. Keluarannya adalah data karyawan yang lengkap dan siap diproses penggajiannya.



Gambar 2.2 Diagram Penggajian Bulanan dan Pelaporan

Gambar 2.2 menampilkan inti sistem berupa proses penggajian bulanan dan pelaporan. Penjelasannya diuraikan berikut ini.

3. Proses Penggajian Bulanan

Proses ini merupakan inti dari sistem, dilakukan setiap bulan oleh Admin, dan melibatkan serangkaian kalkulasi otomatis oleh sistem. Alur prosesnya mengikuti Gambar 2.2.2. Admin memulai dengan menentukan periode penggajian, kemudian mengeksekusi run payroll. Sebelum kalkulasi dimulai, sistem memeriksa apakah seluruh karyawan aktif sudah memiliki komponen gaji aktif. Apabila belum,

Admin perlu menyalin atau mengatur komponen gaji yang kurang terlebih dahulu sebelum mengulangi eksekusi.

Jika seluruh karyawan memiliki komponen gaji aktif, sistem menjalankan kalkulasi secara otomatis dengan alur sebagai berikut. Pertama, sistem mengambil seluruh karyawan aktif perusahaan, menghitung rentang tanggal cutoff berdasarkan konfigurasi, dan menghitung jumlah hari kerja efektif pada periode tersebut. Kedua, sistem mengambil konfigurasi grade dan menghitung masa kerja efektif setiap karyawan (TMT Kerja ditambah nilai offset). Ketiga, untuk setiap karyawan, sistem mengambil komponen gajinya dan menjalankan resolusi nilai komponen berdasarkan hierarki prioritas. Keempat, sistem mengambil data kehadiran dari `attendance_records`. Apabila tidak tersedia, sistem menggunakan nilai `hari_masuk_kerja` yang dikonfigurasi pada data karyawan atau menghitung jumlah hari kerja pada periode cutoff secara otomatis. Kelima, sistem membangun konteks kalkulasi (`buildSalaryContext`) yang mencakup gaji pokok, masa kerja efektif, data kehadiran, dan seluruh atribut karyawan yang relevan. Keenam, sistem mengevaluasi formula seluruh komponen gaji (`evaluateSalaryComponents`), memisahkan komponen EARNING sebagai gross dan DEDUCTION sebagai potongan, lalu menghitung $\text{net} = \text{gross} - \text{deduction}$.

Setelah kalkulasi seluruh karyawan selesai, sistem merekap agregat per departemen dan per komponen, kemudian menyimpan hasilnya ke `payroll_runs` dengan status DRAFT dan `payroll_run_lines` yang memuat detail per karyawan dalam format JSONB. Sistem juga mencatat aktivitas RUN PAYROLL ke dalam activity log. Admin kemudian meninjau hasil rekap DRAFT dan dapat menjalankan validasi integritas. Apabila validasi menemukan ketidaksesuaian, Admin melakukan koreksi data dan menjalankan ulang payroll. Setelah rekap dinyatakan valid, Admin menyetujui dan memfinalisasi payroll run dengan mengubah status menjadi CONFIRMED. Sistem kemudian menghasilkan slip gaji digital untuk setiap karyawan, dilengkapi token kriptografis yang ditandatangani (`signSlip`), URL verifikasi, dan QR code dalam format SVG.

Masukan proses ini adalah periode penggajian, data kehadiran, dan data komponen gaji karyawan. Keluarannya adalah rekap penggajian bulanan (payroll run) beserta slip gaji digital per karyawan.

4. Pemantauan, Pelaporan, dan Self-Service Karyawan

Proses ini berlangsung secara berkelanjutan setelah payroll run selesai dieksekusi, melibatkan Admin, Karyawan, dan pihak ketiga. Admin dapat mengakses laporan rekap penggajian dalam berbagai dimensi: ringkasan per periode, rincian per departemen, rincian per karyawan, dan rekap per komponen dalam rentang periode tertentu. Admin juga dapat melakukan analisis variansi antar periode untuk memantau perubahan biaya gaji, serta mengekspor data rekap dalam format CSV.

Dari sisi karyawan, sistem menyediakan portal self-service di mana karyawan hanya dapat mengakses slip gaji milik mereka sendiri. Karyawan dapat melihat daftar periode beserta ringkasan gross, net, dan status, kemudian membuka rincian komponen untuk periode yang dipilih beserta QR code verifikasi.

Keaslian slip gaji dapat diverifikasi oleh siapa pun, termasuk pihak ketiga, melalui pemindaian QR code. Sistem memvalidasi tanda tangan kriptografis (HMAC), kemudian mencocokkan nilai net pada token dengan data aktual di database. Sistem mengembalikan konfirmasi beserta informasi nama perusahaan, nama karyawan, periode, dan nilai net apabila slip dinyatakan otentik, atau peringatan apabila slip terindikasi dipalsukan.

Masukan proses ini adalah parameter periode, filter pelaporan, dan token QR slip. Keluarannya adalah laporan rekap penggajian, data variansi, slip gaji digital, dan konfirmasi verifikasi keaslian slip.

2.3 Deskripsi Kebutuhan Perangkat Lunak

Bagian proses yang memerlukan dukungan perangkat lunak diuraikan berikut ini, dengan acuan pada Gambar 2.1 dan Gambar 2.2.

1. **Otomatisasi Kalkulasi Gaji dengan Formula Fleksibel.** Pada Gambar 2.2.2, proses kalkulasi gaji melibatkan banyak variabel karyawan dan formula yang berbeda-beda antar karyawan maupun jabatan sehingga tidak dapat dilakukan secara manual secara efisien. Perangkat lunak diperlukan untuk mengotomatisasi evaluasi formula komponen gaji menggunakan konteks data karyawan secara real-time, termasuk resolusi nilai komponen berdasarkan hierarki prioritas `override` → `table` → `fixed`.

2. **Pengelolaan Master Data Perusahaan oleh Superadmin.** Pada Gambar 2.2.1, pendaftaran perusahaan, pembuatan akun pengguna, penugasan pengguna ke perusahaan, dan pengelolaan Integration API Key merupakan tanggung jawab eksklusif Superadmin yang menjadi fondasi bagi seluruh operasi sistem. Perangkat lunak diperlukan untuk menyediakan antarmuka khusus Superadmin yang memungkinkan pengelolaan entitas lintas perusahaan, terpisah dari operasi Admin.
3. **Pengelolaan Master Data Komponen Gaji dan Referensi oleh Admin.** Pada Gambar 2.2.1, Admin bertanggung jawab mendefinisikan struktur organisasi, grade, tunjangan masa kerja, komponen gaji, dan tabel referensi yang harus terisolasi per perusahaan. Perangkat lunak diperlukan untuk menyediakan antarmuka pengelolaan data master yang menerapkan isolasi data antar perusahaan secara otomatis melalui Row Level Security.
4. **Pengelolaan Data Karyawan Berorientasi Penggajian.** Pada Gambar 2.2.1, data karyawan yang perlu dikelola mencakup atribut yang langsung memengaruhi nilai gaji seperti masa kerja, grade, status pernikahan, kode posisi struktural, dan komponen gaji individual. Perangkat lunak diperlukan untuk mengelola data karyawan secara lengkap dan terintegrasi dengan sistem kalkulasi, termasuk mekanisme override nilai komponen per individu dan penyimpanan dokumen kepegawaian digital.
5. **Pengelolaan dan Impor Data Kehadiran.** Pada Gambar 2.2.1 dan Gambar 2.2.2, data kehadiran merupakan salah satu variabel dalam formula penggajian yang dapat bersumber dari mesin fingerprint maupun pencatatan manual. Perangkat lunak diperlukan untuk menampung dan mengelola data kehadiran harian per karyawan sehingga dapat digunakan secara otomatis dalam kalkulasi gaji dengan mekanisme fallback yang terstruktur.
6. **Manajemen Siklus Payroll dengan Loop Validasi dan Koreksi.** Pada Gambar 2.2.2, siklus penggajian memiliki dua titik keputusan yang memungkinkan Admin kembali ke tahap sebelumnya apabila komponen gaji belum lengkap atau validasi rekap tidak lulus. Perangkat lunak diperlukan untuk mendukung eksekusi ulang payroll yang bersifat upsert serta menyediakan mekanisme validasi integritas rekap sebelum finalisasi.
7. **Penerbitan Slip Gaji Digital dengan Verifikasi Keaslian Kriptografis.** Pada Gambar 2.2.2, slip gaji harus dapat diverifikasi keasliannya oleh pihak ketiga tanpa bergantung pada konfirmasi manual. Perangkat lunak diperlukan untuk menghasilkan slip gaji digital yang ditandatangani secara kriptografis, dilengkapi QR code verifikasi, serta menyediakan endpoint publik yang mencocokkan token dengan data aktual di database untuk mendeteksi pemalsuan.
8. **Pelaporan, Rekap, dan Analisis Variansi Penggajian.** Pada Gambar 2.2.2, Admin membutuhkan data penggajian yang dapat diakses dalam berbagai dimensi analisis. Perangkat lunak diperlukan untuk menyediakan rekap per periode, per departemen, per karyawan, dan per komponen, dilengkapi analisis variansi antar periode serta kemampuan ekspor CSV untuk keperluan audit dan pelaporan keuangan.
9. **Self-Service Karyawan untuk Akses Slip Gaji.** Pada Gambar 2.2.2, karyawan membutuhkan akses mandiri ke slip gaji mereka tanpa melalui Admin. Perangkat lunak diperlukan untuk menyediakan portal self-service yang membatasi akses karyawan hanya pada slip gaji milik mereka sendiri, menampilkan rincian komponen per periode beserta QR code verifikasi.
10. **Pengelolaan Hak Akses Berbasis Peran dan Menu.** Pada Gambar 2.2.1 dan Gambar 2.2.2, seluruh proses melibatkan data sensitif yang harus diakses sesuai dengan peran masing-masing pengguna. Perangkat lunak diperlukan untuk mengelola autentikasi berbasis JWT, otorisasi berbasis peran (SUPERADMIN, ADMIN, USER) dengan granularitas per menu, serta isolasi data level basis data menggunakan Row Level Security.
11. **Pencatatan Log Aktivitas yang Bersifat Immutable.** Pada Gambar 2.2.1 dan Gambar 2.2.2, seluruh proses yang mengubah data sensitif memerlukan jejak audit yang tidak dapat dimodifikasi. Perangkat lunak diperlukan untuk mencatat setiap aktivitas pengguna secara otomatis ke dalam activity log yang dijaga kekekalannya melalui trigger basis data, mencakup aksi, entitas yang diubah, nilai sebelum dan sesudah perubahan, serta metadata IP address dan user agent.
12. **Integrasi dengan Sistem HRD Eksternal melalui API Key.** pada Gambar 2.2.1, sistem dirancang untuk beroperasi berdampingan dengan aplikasi HRD eksternal yang sudah ada. Perangkat lunak diperlukan untuk menyediakan mekanisme integrasi berbasis API Key yang dikelola Superadmin,

dengan middleware validasi yang memeriksa hash kunci, status aktif, dan cakupan akses setiap kali kunci digunakan.

3 Kebutuhan Perangkat Lunak

3.1 Deskripsi Umum Sistem

Aplikasi Penggajian Terintegrasi adalah perangkat lunak berbasis web yang dirancang untuk mengotomatisasi seluruh siklus penggajian karyawan di PT LAPI Divusi, mulai dari pengelolaan data master pegawai, pengaturan komponen gaji yang fleksibel untuk setiap karyawan, proses perhitungan payroll periodik, hingga distribusi slip gaji digital kepada karyawan. Sistem ini bersifat *multi-tenant*, artinya satu instalasi sistem dapat melayani beberapa perusahaan atau entitas bisnis sekaligus, dengan isolasi data per perusahaan dan kemampuan bagi pengguna tertentu untuk berpindah konteks perusahaan tanpa harus keluar dari aplikasi.

Secara arsitektural, perangkat lunak ini terdiri atas dua aplikasi utama, yaitu:

- Antarmuka pengguna (*frontend*) yang dibangun dengan Next.js
- Layanan API (*backend*) yang dibangun dengan Express/TypeScript di atas basis data PostgreSQL

Kedua aplikasi tersebut berinteraksi melalui protokol HTTP/HTTPS dengan autentikasi berbasis JWT untuk akses pengguna interaktif. Selain itu, sistem juga menyediakan kanal integrasi *machine-to-machine* yang memungkinkan aplikasi HRD eksternal milik perusahaan menarik data slip gaji secara terprogram menggunakan API key per perusahaan dengan scope `payroll:read`.

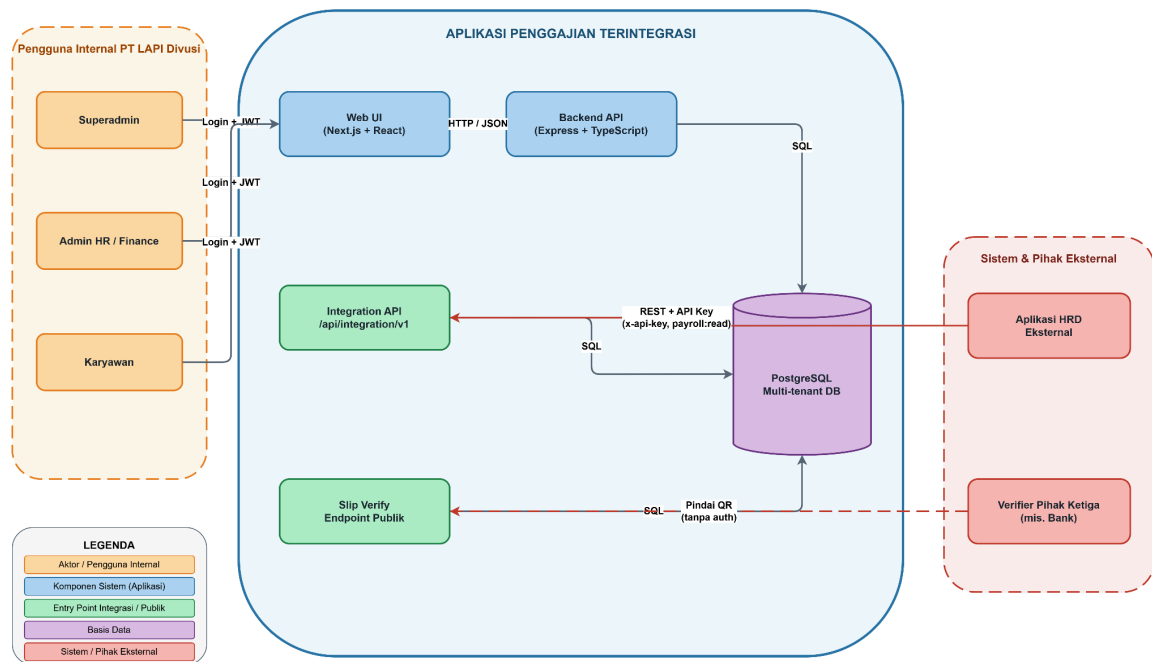
Sistem ini berfokus pada tiga area utama:

- Fungsi penggajian
- Pengelolaan master data kepegawaian yang relevan terhadap penggajian, meliputi
 - Biodata
 - Jabatan
 - Departemen
 - Status kerja
 - Riwayat jabatan
 - Dokumen pendukung
- Administrasi sistem, meliputi:
 - Manajemen pengguna
 - Hak akses per menu/aksi
 - Manajemen perusahaan
 - *Table builder* untuk pengelolaan skema basis data melalui antarmuka pengguna
 - *Activity log*

Sistem ini tidak menangani fungsi HRD secara penuh, seperti rekrutmen, *performance appraisal*, atau manajemen cuti. Fungsi-fungsi tersebut tetap berada di aplikasi HRD eksternal yang dimiliki perusahaan.

Sebagai jembatan integrasi, sistem ini menyediakan endpoint terstandarisasi, yaitu `/api/integration/v1/payroll/...`, agar aplikasi HRD eksternal dapat menampilkan slip gaji pegawai berdasarkan NIK.

Gambar 3.1.1 Diagram Blok Umum Aplikasi Penggajian Terintegrasi



Gambar 3.1 Diagram Blok Umum Aplikasi Penggajian Terintegrasi

[Gambar 3.1 Diagram Blok Umum Aplikasi Penggajian Terintegrasi](#)

Gambar 3.1.1 memperlihatkan gambaran umum hubungan antara pengguna, sistem yang dibangun, basis data, serta pihak eksternal yang berinteraksi dengan Aplikasi Penggajian Terintegrasi. Diagram tersebut membagi elemen sistem ke dalam tiga area utama, yaitu pengguna internal PT LAPI Divusi, komponen Aplikasi Penggajian Terintegrasi, dan sistem atau pihak eksternal.

Pada sisi pengguna internal, terdapat tiga aktor utama, yaitu Superadmin, Admin HR/Finance, dan Karyawan. Ketiga aktor tersebut mengakses sistem melalui *Web UI*. Setelah proses autentikasi berhasil, pengguna dapat menjalankan fitur sesuai dengan peran dan hak aksesnya masing-masing.

Pada *boundary* utama sistem, terdapat *Web UI*, *Backend API*, *Integration API*, *Slip Verify Endpoint Publik*, dan basis data PostgreSQL. *Web UI* menjadi antarmuka utama bagi pengguna internal. *Backend API* menangani permintaan dari *Web UI*, menjalankan logika bisnis, dan mengakses data pada PostgreSQL. *Integration API* menjadi pintu masuk bagi aplikasi HRD eksternal untuk mengambil data slip gaji. *Slip Verify Endpoint Publik* digunakan oleh pihak ketiga untuk memverifikasi keaslian slip gaji berdasarkan kode QR.

Pada sisi eksternal, terdapat aplikasi HRD eksternal dan verifier pihak ketiga. Aplikasi HRD eksternal berinteraksi dengan *Integration API* untuk memperoleh data payroll atau slip gaji sesuai kebutuhan integrasi. Verifier pihak ketiga, misalnya bank, dapat mengakses halaman verifikasi slip melalui kode QR yang tercantum pada slip gaji. Dengan pembagian ini, batas antara sistem yang dibangun, pengguna internal, dan pihak eksternal dapat terlihat dengan jelas.

Berdasarkan diagram tersebut, Aplikasi Penggajian Terintegrasi memiliki tiga pintu masuk utama, yaitu *Web UI* untuk pengguna internal, *Integration API* untuk aplikasi HRD eksternal, dan *Slip Verify Endpoint Publik* untuk verifikasi slip oleh pihak ketiga. Seluruh kanal tersebut tetap mengacu pada basis data yang sama sehingga data penggajian tetap konsisten, terpusat, dan dapat ditelusuri melalui log aktivitas.

3.2 Stakeholder

Stakeholder yang terlibat dalam pengembangan dan penggunaan sistem ditampilkan pada Tabel 3.1.

Tabel 3.1 Daftar Stakeholder Aplikasi Penggajian

No	Stakeholder	Deskripsi
1	Product Owner	Menentukan tujuan pengembangan sistem penggajian dan memiliki kebutuhan yang menjadi landasan sistem dibuat
3	Tim Developer	Mengembangkan dan mengimplementasikan sistem penggajian sesuai dengan kebutuhan yang telah ditentukan
4	Administrator Sistem	Mengelola akun pengguna, hak akses, dan konfigurasi dari sistem
7	Karyawan	Menggunakan sistem untuk melihat informasi gaji dan slip gaji

3.3 Identifikasi Kebutuhan Perangkat Lunak

3.3.1 Product Backlog

Hasil identifikasi kebutuhan dalam bentuk Product Backlog ditampilkan pada Tabel 3.2. Setiap baris memuat ID, user story, dan acceptance criteria.

Tabel 3.2 Product Backlog Aplikasi Penggajian

ID	User Story	Acceptance Criteria
PB-01	Sebagai Admin, saya ingin skema database awal sudah tersedia dan bisa dijalankan otomatis, sehingga struktur data langsung siap tanpa konfigurasi manual.	Migration script tersedia, seed data superadmin default, dokumentasi skema ada, rollback migration tersedia
PB-02	Sebagai Admin, saya ingin setiap endpoint API terlindungi oleh autentikasi dan pengecekan hak akses, sehingga data tidak bisa diakses sembarangan.	JWT issued saat login, middleware auth di semua protected route, respons 401 jika token invalid, 403 jika tidak punya akses
PB-03	Sebagai User, saya ingin dapat login menggunakan username dan password, sehingga saya bisa mengakses sistem sesuai perusahaan yang tepat.	Form login 2 field, session menyimpan konteks perusahaan, pesan error jelas
PB-04	Sebagai Admin, saya ingin dapat mendaftarkan dan mengelola data perusahaan, sehingga satu database dapat melayani banyak perusahaan sekaligus.	CRUD data perusahaan, kode perusahaan unik, perusahaan dapat dinonaktifkan, log perubahan tercatat
PB-05	Sebagai Admin, saya ingin dapat menetapkan user ke satu atau lebih perusahaan dan menentukan apakah user tersebut boleh berpindah perusahaan.	Assign user ke perusahaan, flag "dapat berpindah" per user, jika hanya

		1 perusahaan pilihan tidak muncul saat login
PB-06	Sebagai User, saya ingin dapat berpindah perusahaan aktif setelah login tanpa harus logout, sehingga pengelolaan anak perusahaan lebih efisien.	Switcher muncul jika akses > 1 perusahaan, perpindahan memperbarui konteks data, log perpindahan tercatat
PB-07	Sebagai Admin, saya ingin dapat membuat akun user baru dan membedakan user biasa dengan superadmin.	Form create user lengkap, flag is_superadmin, superadmin otomatis akses semua menu, minimal 1 superadmin aktif di sistem
PB-08	Sebagai Admin, saya ingin dapat menetapkan hak akses langsung per user ke menu atau modul tertentu tanpa menggunakan role.	Daftar menu bisa dipilih per user, akses berupa: baca/insert/update/delete/process, perubahan langsung berlaku, arsitektur siap RBAC ke depan
PB-09	Sebagai User, saya ingin menu dan tombol aksi yang tampil hanya sesuai hak akses saya, sehingga tidak ada aksi yang bisa dilakukan di luar izin saya.	Menu tidak diakses disembunyikan, tombol aksi muncul sesuai hak, validasi akses juga di backend, akses ditolak return 403
PB-10	Sebagai Admin, saya ingin semua aktivitas user (login, logout, buka menu, aksi data) dicatat otomatis di log, sehingga ada jejak audit yang lengkap.	Log memuat: user, waktu, perusahaan, menu/form, aksi (LOGIN/LOGOUT/READ/INSERT/UPDATE/DELETE/PROCESS), detail tambahan; tidak bisa diedit/dihapus user biasa
PB-11	Sebagai Admin, saya ingin dapat melihat dan memfilter activity log melalui UI, sehingga investigasi dan audit mudah dilakukan.	Filter by user/tanggal/menu/aksi, pagination, export ke Excel/CSV, hanya superadmin yang bisa akses
PB-12	Sebagai Admin, saya ingin dapat membuat tabel baru di database melalui UI tanpa perlu akses terminal SQL.	Form input nama tabel & kolom, validasi sebelum eksekusi, feedback sukses/gagal, log DDL tercatat
PB-13	Sebagai Admin, saya ingin dapat mengubah struktur tabel (ALTER TABLE) melalui UI, sehingga tidak perlu menulis SQL manual.	Tambah/hapus/ubah kolom, konfirmasi sebelum perubahan destruktif, log tercatat
PB-14	Sebagai Admin, saya ingin dapat menghapus tabel dari database melalui UI dengan konfirmasi, sehingga penghapusan tidak terjadi secara tidak sengaja.	Daftar tabel ditampilkan, double-check konfirmasi sebelum DROP, log tercatat

PB-15	Sebagai Admin, saya ingin dapat melakukan INSERT, UPDATE, dan DELETE data pada tabel melalui UI, sehingga pengelolaan data tidak perlu lewat terminal.	Form insert per tabel, edit inline per baris, konfirmasi sebelum DELETE, validasi tipe data, log tercatat
PB-16	Sebagai Admin, saya ingin dapat mengelola data karyawan (biodata, jabatan, departemen, status kerja), sehingga informasi kepegawaian terpusat dan terstruktur.	CRUD data karyawan, field: NIK, nama, jabatan, departemen, tanggal masuk, status aktif/resign; data terisolasi per perusahaan
PB-17	Sebagai Admin, saya ingin dapat mengelola struktur organisasi (departemen dan jabatan), sehingga pengelompokan karyawan lebih teratur.	CRUD departemen dan jabatan, jabatan dapat dikaitkan ke departemen, log perubahan tercatat
PB-18	Sebagai Admin, saya ingin dapat mencatat riwayat perubahan jabatan dan departemen karyawan, sehingga histori karir karyawan bisa dilacak.	Setiap perubahan jabatan/departemen tersimpan sebagai histori, dapat dilihat per karyawan, tidak menimpa data lama
PB-19	Sebagai Admin, saya ingin dapat mengelola komponen gaji (gaji pokok, tunjangan, potongan) per karyawan, sehingga struktur penggajian fleksibel sesuai kebijakan perusahaan.	CRUD komponen gaji per karyawan, tipe komponen: penambah/pengurang, komponen dapat diaktifkan/dinonaktifkan, log perubahan tercatat
PB-20	Sebagai Admin, saya ingin dapat menjalankan proses penggajian untuk periode tertentu (bulanan), sehingga perhitungan gaji seluruh karyawan bisa dilakukan sekaligus.	Pilih periode (bulan/tahun), sistem menghitung gaji berdasarkan komponen aktif tiap karyawan, hasil tersimpan sebagai draft sebelum dikonfirmasi, log proses tercatat
PB-21	Sebagai Admin, saya ingin dapat mengonfirmasi atau membatalkan hasil penggajian sebelum finalisasi, sehingga ada tahap review sebelum data gaji dikunci.	Status payroll: draft → confirmed → finalized, pembatalan hanya bisa dilakukan sebelum finalized, data finalized tidak bisa diubah
PB-22	Sebagai Admin, saya ingin dapat melihat rekapitulasi penggajian per periode dan per karyawan, sehingga pelaporan keuangan lebih mudah.	Tabel rekap gaji per periode, filter by karyawan/departemen/periode, export ke Excel/CSV
PB-23	Sebagai User, saya ingin dapat melihat slip gaji saya sendiri per periode, sehingga saya bisa memantau rincian penghasilan saya.	Halaman slip gaji per karyawan, menampilkan rincian komponen penambah dan pengurang, hanya bisa melihat data milik sendiri

3.3.1.1 Tabel Use Case

Daftar use case beserta deskripsi ditampilkan pada Tabel 3.3. Daftar telah dilengkapi sehingga seluruh use case yang skenarionya diuraikan pada Subbab 4.3.2 tercantum, yaitu UC-01 sampai UC-20.

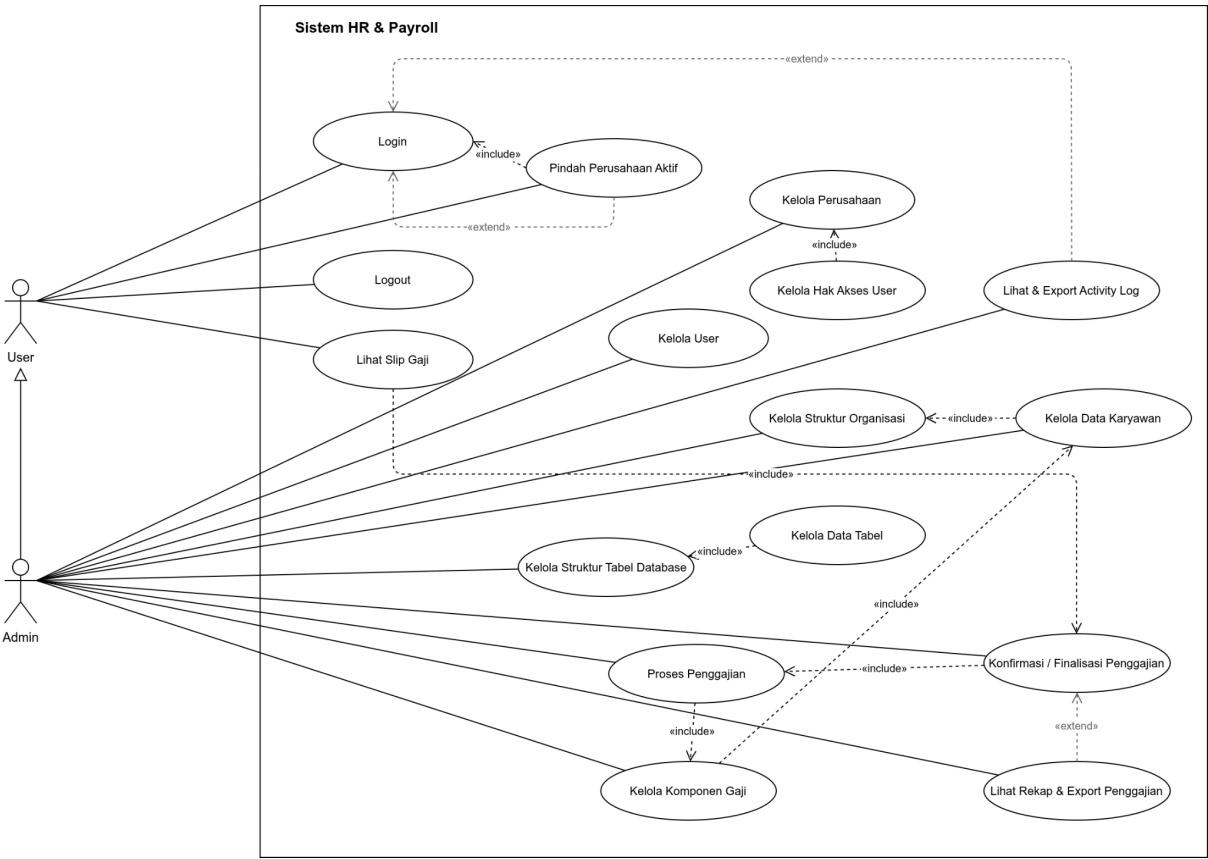
Tabel 3.3 Daftar Use Case Aplikasi Penggajian

ID	Use Case	Deskripsi
UC-01	Login	User dapat masuk ke sistem dengan memasukkan username/email dan password.
UC-02	Logout	User dapat mengakhiri sesi aktif dan keluar dari sistem
UC-03	Pindah Perusahaan Aktif	User dapat berpindah konteks perusahaan tanpa harus logout terlebih dahulu
UC-04	Kelola User	Admin dapat membuat, mengubah, dan menonaktifkan akun user serta mengatur flag superadmin dan assignment perusahaan
UC-05	Kelola Hak Akses User	Admin dapat menetapkan menu dan jenis aksi yang diizinkan per user
UC-06	Kelola Perusahaan	Admin dapat membuat, mengubah, dan menonaktifkan data perusahaan
UC-07	Kelola Struktur Organisasi	Admin dapat membuat, mengubah, dan menghapus departemen dan jabatan
UC-08	Kelola Data Karyawan	Admin dapat membuat, mengubah, dan menonaktifkan data karyawan beserta histori perubahan jabatannya
UC-09	Kelola Struktur Tabel Database	Admin dapat membuat, mengubah, dan menghapus tabel database melalui UI tanpa SQL manual
UC-10	Kelola Data Tabel	Admin dapat melakukan insert, update, dan delete data pada tabel melalui UI
UC-11	Lihat & Export Activity Log	Admin dapat memfilter dan melihat riwayat aktivitas user serta mengekspornya ke Excel/CSV
UC-12	Kelola Komponen Gaji	Admin dapat menetapkan gaji pokok, tunjangan, dan potongan per karyawan
UC-13	Proses Penggajian	Admin dapat menjalankan perhitungan payroll untuk periode tertentu dan menyimpan hasilnya sebagai draft
UC-14	Konfirmasi / Finalisasi Penggajian	Admin dapat mengonfirmasi atau membatalkan draft penggajian sebelum data dikunci sebagai final
UC-15	Lihat Rekap & Export Penggajian	Admin dapat melihat rekapitulasi penggajian per periode dan departemen serta mengekspornya ke Excel/CSV

UC-16	Lihat Slip Gaji	User dapat melihat rincian slip gaji miliknya sendiri per periode
-------	-----------------	---

3.3.1.2 Diagram Use Case

Diagram use case ditampilkan pada Gambar 3.2. Diagram memuat aktor User, Admin, dan Superadmin yang dihubungkan ke use case sesuai Tabel 3.3, beserta relasi include dan extend antar use case.



Gambar 3.2 Use Case Diagram Sistem HR & Payroll

Untuk *use case* yang terhubung dengan lebih dari satu aktor, skenario pada Subbab 4.3.2 disusun agar konsisten melibatkan aktor terkait. Sebagai contoh, UC-08 Kelola Data Karyawan dan UC-17 *Assign* Pegawai ke Perusahaan dapat dijalankan oleh Admin maupun Superadmin, sehingga skenarionya menuliskan aktor sebagai Admin/Superadmin.

4 Deskripsi Perancangan Perangkat Lunak

4.1 Deskripsi Arsitektur

Perangkat lunak ini dirancang menggunakan arsitektur three-tier client-server berbasis web yang memisahkan lapisan presentasi, lapisan logika bisnis, dan lapisan data secara eksplisit. Ketiga lapisan tersebut dikembangkan sebagai dua aplikasi terpisah yang berkomunikasi melalui REST API, dengan basis data relasional sebagai penyimpanan terpusat.

1. **Lapisan Presentasi**, diimplementasikan menggunakan Next.js dengan React sebagai kerangka kerja frontend. Aplikasi frontend berjalan di sisi klien melalui peramban web dan menggunakan Next.js App Router untuk pengelolaan rute halaman. Proteksi rute dilakukan melalui Next.js Middleware yang

memeriksa keberadaan token autentikasi (hr_token) pada cookie sebelum mengizinkan akses ke halaman di bawah /dashboard. Halaman publik seperti /login, /select-company, dan /slip-verify dikecualikan dari proteksi ini. Antarmuka dibangun menggunakan komponen React dengan Tailwind CSS untuk penataan tampilan.

2. **Lapisan Logika Bisnis**, diimplementasikan menggunakan Node.js dengan kerangka kerja Express.js dan ditulis dalam TypeScript. Lapisan ini menerima seluruh permintaan dari frontend melalui endpoint REST API yang diawali dengan prefiks /api. Setiap permintaan diproses melalui serangkaian middleware secara berurutan: CORS untuk penanganan cross-origin request, request context untuk pelacakan perintah SQL dalam satu siklus permintaan, activity logger untuk pencatatan aktivitas pengguna secara otomatis, kemudian auth middleware dan permission middleware untuk validasi token JWT dan hak akses per menu. Khusus untuk integrasi sistem HRD eksternal, tersedia API key middleware sebagai jalur autentikasi alternatif berbasis kunci API. Logika bisnis dikelompokkan ke dalam delapan belas modul rute yang masing-masing menangani domain fungsional tertentu. Dokumentasi API tersedia secara otomatis melalui Swagger UI pada endpoint /api/docs. Komponen paling kritis pada lapisan ini adalah Formula Engine, yang terdiri dari dua modul: formulaInterpreter.ts dan formulaResolver.ts. Formula Engine bertanggung jawab membangun konteks kalkulasi dari atribut karyawan (buildSalaryContext), mengevaluasi ekspresi matematika secara dinamis menggunakan library expr-eval (evaluateSalaryComponents), mendeteksi referensi sikular antar komponen gaji (detectCircularDeps), dan menentukan urutan evaluasi yang aman melalui pengurutan topologis (topoSort).
3. **Lapisan Data**, diimplementasikan menggunakan PostgreSQL sebagai sistem manajemen basis data relasional. Akses basis data dilakukan menggunakan driver pg (node-postgres) secara langsung tanpa ORM, melalui modul lib/db.ts yang menyediakan fungsi-fungsi query generik. Isolasi data antar perusahaan diterapkan di level basis data menggunakan mekanisme Row Level Security (RLS) yang memfilter seluruh kueri secara otomatis berdasarkan variabel sesi app.current_company_id. Skema basis data dikelola melalui sistem migrasi bertahap menggunakan skrip SQL bernomor urut yang dapat dijalankan dan di-rollback melalui perintah npm run db:migrate dan npm run db:rollback.

Terkait dengan Architectural Drivers, terdapat beberapa pertimbangan arsitektural utama yang memotivasi keputusan desain sistem ini. Multi-tenancy menjadi driver utama karena sistem dirancang untuk melayani beberapa perusahaan sekaligus dalam satu instalasi. Hal ini dimotivasi oleh kebutuhan efisiensi operasional dan pemeliharaan. Solusi yang dipilih adalah kombinasi antara isolasi logis pada lapisan aplikasi (JWT menyimpan companyId, setiap kueri difilter berdasarkan companyId) dan isolasi fisik pada lapisan data melalui RLS PostgreSQL, sehingga kebocoran data antar perusahaan dicegah berlapis.

Fleksibilitas formula penggajian menjadi driver penting karena setiap perusahaan memiliki struktur komponen gaji yang berbeda dan dapat berubah sewaktu-waktu tanpa memerlukan perubahan kode. Solusi yang dipilih adalah formula engine berbasis ekspresi teks yang dievaluasi secara dinamis pada saat runtime, sehingga Admin dapat mendefinisikan dan memodifikasi formula melalui antarmuka tanpa keterlibatan pengembang.

Keamanan dan auditabilitas menjadi driver karena data penggajian bersifat sangat sensitif. Solusi yang dipilih mencakup autentikasi berbasis JWT dengan masa berlaku terbatas, otorisasi granular per menu, enkripsi password menggunakan bcrypt, pencatatan activity log yang bersifat immutable melalui trigger basis data, serta tanda tangan kriptografis pada slip gaji untuk mencegah pemalsuan.

Kemudahan integrasi menjadi driver karena sistem harus dapat beroperasi berdampingan dengan aplikasi HRD yang sudah ada. Solusi yang dipilih adalah penyediaan REST API terdokumentasi (Swagger) dan mekanisme API Key untuk akses machine-to-machine tanpa memerlukan kredensial pengguna.

4.2 Dekomposisi Fungsional

Sistem didekomposisi menjadi subsistem dan komponen berdasarkan domain fungsional, seperti pada Tabel 4.1. Kolom Role-Stereotype mengacu pada klasifikasi peran objek yang ditampilkan pada Gambar 4.1.

Tabel 4.1 Dekomposisi Fungsional Subsistem dan Komponen

ID	Nama subsistem/ komponen/kelas	Tanggung Jawab	Role- Stereotype
SS-01	Frontend Application	Menyajikan antarmuka pengguna berbasis web dan mengorkestrasi alur navigasi antar halaman menggunakan Next.js App Router.	Interfacer
SS-02	Backend API Server	Menerima, memvalidasi, dan mendistribusikan seluruh permintaan HTTP dari klien ke modul-modul yang sesuai melalui Express.js.	Controller
SS-03	Database (PostgreSQL)	Menyimpan, mengisolasi, dan menjaga integritas seluruh data sistem menggunakan RLS multi-tenancy dan constraint relasional.	Information Holder
C-01	Auth Module (authController, authMiddleware)	Mengelola proses autentikasi pengguna, penerbitan JWT, pemilihan perusahaan aktif, dan perpindahan perusahaan.	Controller
C-02	Permission Module (permissionController, permissionMiddleware)	Mengelola definisi dan penegakan hak akses granular per pengguna, per perusahaan, dan per menu.	Controller
C-03	Company Module (companyController, companyModel)	Mengelola data master perusahaan dan penugasan pengguna ke perusahaan beserta konfigurasi hak switching.	Information Holder
C-04	User Module (userController, userModel)	Mengelola data akun pengguna sistem pada level global (lintas perusahaan), termasuk pembuatan dan pengelolaan peran.	Information Holder
C-05	Employee Module (employeePayrollController, employeePayrollModel)	Mengelola data karyawan yang berorientasi penggajian, mencakup atribut kepegawaian, atribut gaji, dan relasi ke jabatan, departemen, serta grade.	Information Holder
C-06	Department & Jabatan Module (departmentController, jabatanController)	Mengelola struktur organisasi perusahaan berupa hierarki departemen dan jabatan yang menjadi acuan nilai komponen gaji bertipe TABLE.	Information Holder
C-07	Employee Grade Module (gradeAllowanceController)	Mengelola master grade karyawan dan tabel tunjangan masa kerja per grade sebagai acuan kalkulasi gaji pokok dan tunjangan berbasis senioritas.	Information Holder
C-08	Salary Component Module (salaryComponentController, salaryComponentModel)	Mengelola definisi master komponen gaji perusahaan (FIXED/TABLE/EMPLOYEE_ATTR, EARN/DEDUCT) beserta nilai per jabatan dan override per karyawan.	Information Holder
C-09	Formula Engine (formulaInterpreter.ts, formulaResolver.ts)	Membangun konteks kalkulasi dari atribut karyawan, mengevaluasi ekspresi formula secara dinamis, mendeteksi dependensi sirkular, dan menentukan urutan evaluasi yang aman melalui pengurutan topologis.	Service Provider

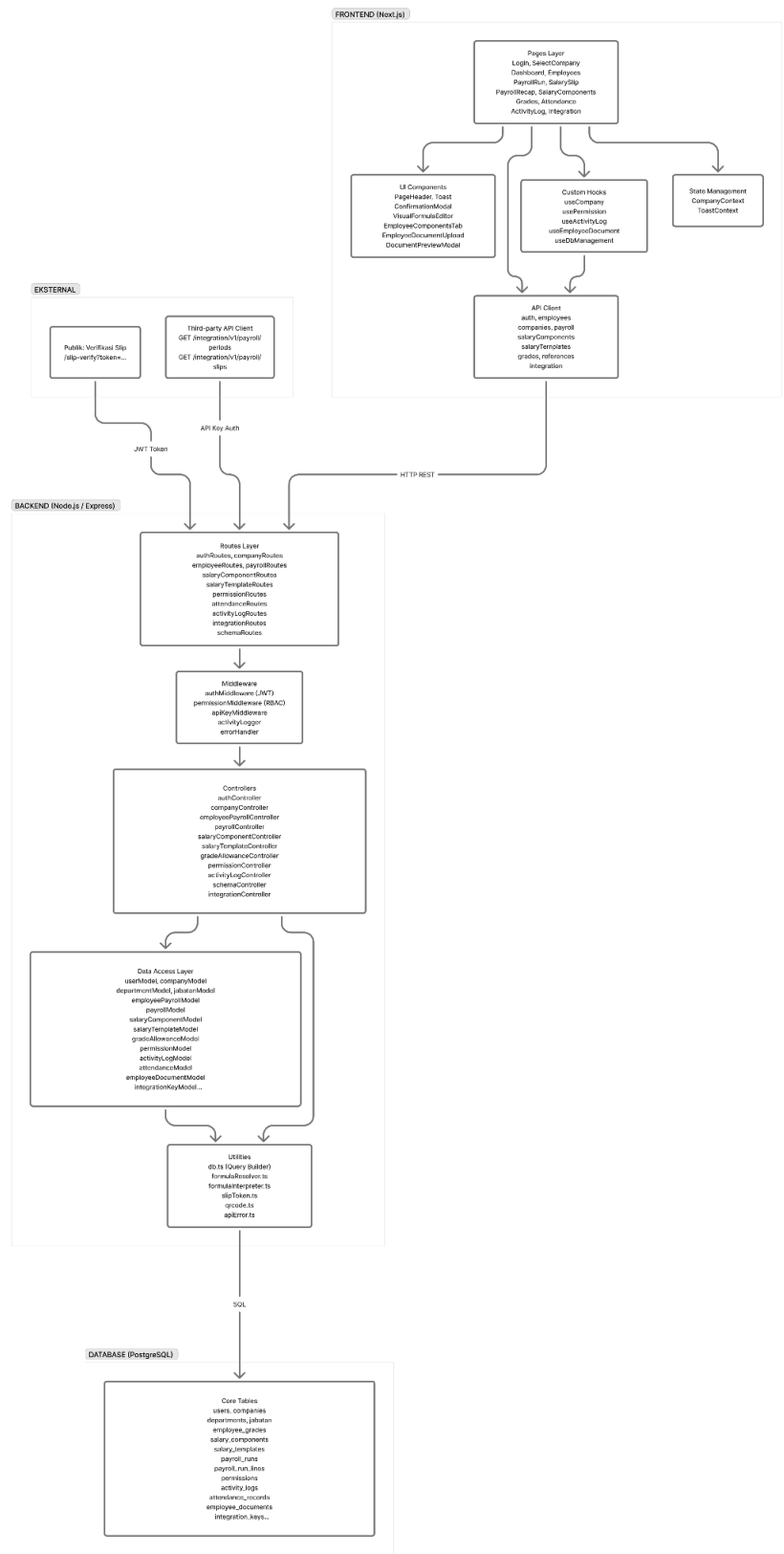
C-10	Payroll Module (payrollController, payrollModel)	Mengorkestrasi eksekusi penggajian bulanan, menyimpan rekap payroll run dan detail per karyawan, serta menyediakan endpoint rekap multi-dimensi dan analisis variansi.	Coordinator
C-11	Payroll Validation Module (payrollValidationModel)	Memvalidasi integritas rekap penggajian dengan menjalankan serangkaian pemeriksaan konsistensi pada payroll run sebelum finalisasi.	Service Provider
C-12	Salary Slip Module (slipToken.ts, qrcode.ts)	Menerbitkan slip gaji digital dengan tanda tangan kriptografis (HMAC), menghasilkan QR code verifikasi, dan memvalidasi keaslian slip terhadap data basis data.	Service Provider
C-13	Attendance Module (attendanceController, attendanceModel)	Mengelola pencatatan dan impor data kehadiran harian karyawan yang digunakan sebagai variabel dalam kalkulasi formula gaji.	Information Holder
C-14	Reference Data Module (referenceController, referenceModel)	Mengelola tabel referensi pendukung formula (ref_fungsional, ref_struktural, ref_proyek) yang digunakan sebagai sumber nilai lookup dalam konteks kalkulasi.	Information Holder
C-15	Activity Log Module (activityLogController, activityLogger.ts, activityLogModel)	Mencatat seluruh aktivitas pengguna secara otomatis dan immutable ke dalam activity log, mencakup operasi CRUD, eksekusi payroll, dan ekspor data.	Service Provider
C-16	DB Management Module (dbManagementController, schemaController)	Menyediakan antarmuka untuk eksekusi DDL (CREATE/ALTER/DROP TABLE) yang diaudit melalui db_audit, khusus untuk keperluan kustomisasi skema oleh administrator teknis.	Controller
C-17	Document Storage Module (employeeDocumentControll er, documentStorage.ts)	Mengelola unggahan, penyimpanan, dan pengambilan dokumen kepegawaian digital (KTP, KK, NPWP, CV) per karyawan.	Service Provider
C-18	Integration Module (integrationController, integrationKeyModel, apiKeyMiddleware.ts)	Mengelola siklus hidup Integration API Key dan menyediakan middleware validasi untuk akses machine-to-machine dari sistem HRD eksternal.	Interfacer
C-19	Next.js Route Guard (frontend/middleware.ts)	Melindungi seluruh rute di bawah /dashboard dengan memeriksa keberadaan token autentikasi pada cookie sebelum permintaan diteruskan ke halaman tujuan.	Controller
C-20	API Client Layer (frontend/lib/ frontend/utls/api.ts)	Mengabstraksi seluruh komunikasi HTTP dari komponen frontend ke Backend API Server, termasuk pengelolaan token dan penanganan respons error secara terpusat.	Interfacer

4.3 Architectural View

4.3.1 Structural View

Structural View menggambarkan struktur statis perangkat lunak dalam dua bentuk diagram, yaitu diagram komponen dan diagram kelas. Diagram komponen ditampilkan pada Gambar 4.2 dan diagram kelas pada Gambar 4.3.

4.3.1.1 Diagram Komponen



Gambar 4.2 Diagram Komponen Aplikasi Penggajian Terintegrasi

Gambar 4.2 menampilkan komponen utama perangkat lunak beserta relasinya. Frontend Application (SS-01) berkomunikasi dengan Backend API Server (SS-02) melalui API Client Layer (C-20) dan dilindungi Next.js

Route Guard (C-19). Backend API Server mendistribusikan permintaan ke modul fungsional C-01 sampai C-18, sedangkan seluruh modul mengakses Database (SS-03) melalui lapisan akses data. Formula Engine (C-09) dipanggil oleh Payroll Module (C-10) saat kalkulasi, dan Integration Module (C-18) menyediakan jalur akses machine-to-machine.

Diagram komponen menggambarkan dekomposisi sistem Aplikasi Penggajian ke dalam subsistem dan komponen utama berikut ketergantungan antar-komponen. Sistem dibagi menjadi empat subsistem: Frontend (Next.js), Backend (Node.js/Express), Database (PostgreSQL), dan Eksternal.

Subsistem Frontend (Next.js)

- Pages Layer : 13 halaman termasuk login, dashboard, manajemen karyawan, payroll, laporan, dan log aktivitas.
- UI Components : Komponen reusable: Toast, Modal, VisualFormulaEditor, EmployeeDocumentUpload, LogTable.
- State Management : CompanyContext dan ToastContext untuk state lintas halaman.
- Custom Hooks : useCompany, usePermission, useActivityLog, useEmployeeDocument, useDbManagement.
- API Client (lib/) : 11 modul yang mengenkapsulasi seluruh panggilan HTTP ke backend.

Subsistem Backend (Node.js/Express)

- Routes Layer : 21 modul rute mendefinisikan 60+ endpoint API REST.
- Middleware : Autentikasi JWT, otorisasi RBAC, API key guard, activity logger, dan global error handler.
- Controllers : 20 kontroler mengelola logika bisnis dan orkestrasi antar layanan.
- Data Access Layer (Models) : 23 model mengenkapsulasi query PostgreSQL per entitas domain.
- Utilities : db.ts (query builder), formulaResolver, formulaInterpreter, slipToken, qrcode, apiError.

Subsistem Database (PostgreSQL)

- 14 tabel inti dengan isolasi data per perusahaan (companyId) untuk mendukung arsitektur multi-tenant.

Subsistem Eksternal

- Third-party API Client : Akses data payroll via API key tanpa login penuh.
- Publik: Verifikasi Slip : Endpoint publik verifikasi slip gaji berbasis JWT token dari QR code.

Kelas-kelas Utama

- | | | |
|--------------------------------------|------------------------|-----------------------------|
| Program Studi Teknik Informatika ITB | IF-ITB-IF3250-PPL-2024 | Halaman 27/ dari 88 halaman |
|--------------------------------------|------------------------|-----------------------------|
- Seluruh isi dokumen ini dan informasi yang dimilikinya adalah milik Program Studi Teknik Informatika-ITB, serta bersifat rahasia.
 Dilarang me-reproduksi dokumen ini tanpa diketahui oleh Program Studi Teknik Informatika ITB.

4.3.1.3 Traceability Table

Tabel berikut memetakan setiap kelas domain ke komponen arsitektur yang mengimplementasikannya, mencakup tabel database, model (Data Access Layer), kontroler, halaman frontend, serta komponen/hook frontend yang terkait. Kolom terakhir menghubungkan setiap kelas ke item Product Backlog yang relevan.

Keterangan singkatan komponen:

- BE_Models = Data Access Layer (backend/src/models/)
- BE_Controllers = Controllers (backend/src/controllers/)
- BE_Middleware = Middleware (backend/src/middleware/)
- BE_Utils = Utilities (backend/src/lib/ & utils/)
- BE_Routes = Routes (backend/src/routes/)
- FE_Pages = Pages Layer (frontend/app/)
- FE_Components = UI Components (frontend/components/)
- FE_Contexts = State Management (frontend/context/)
- FE_Hooks = Custom Hooks (frontend/hooks/)
- FE_Lib = API Client (frontend/lib/)

Tabel 4.2 Traceability Kelas terhadap Komponen Perancangan

Kelas	Tabel Database	Model (DAL)	Controller	Halaman Frontend	Komponen / Hook	Related PB
User	users user_companies	userModel employeePayrollModel	authController userController employeePayrollController	Login Employee s Dashboard	CompanyContext useCompany	PB-02,03,05 PB-07,16,18
Company	companies user_companies	companyModel	companyController authController	Companies SelectCompany	CompanyContext	PB-04,05,06
Department	departments	departmentModel	departmentController	JabatanDepartment	—	PB-17,18
Jabatan	jabatan	jabatanModel	jabatanController	JabatanDepartment	—	PB-17,18
Employee Grade	employee_grades salary_grades	gradeAllowanceModel	gradeAllowanceController	Grades	—	PB-19
SalaryComponent	salary_components component_table_rows	salaryComponentModel	salaryComponentController	SalaryComponents	VisualFormulaEditor EmployeeComponentsTab	PB-19,20
SalaryTemplate	salary_templates	salaryTemplateModel	salaryTemplateController	SalaryComponents	EmployeeComponentsTab	PB-19
PayrollRun	payroll_runs payroll_settings	payrollModel	payrollController	PayrollRun PayrollRecap	—	PB-20,21,22
PayrollRun Line	payroll_run_lines	payrollModel salaryPreviewModel	payrollController	PayrollRun SalarySlip	—	PB-20,21,22,23

				PayrollRecap		
Permission	permissions	permissionModel	permissionController	Users	usePermission	PB-02,08,09
ActivityLog	activity_logs	activityLogModel	activityLogController	ActivityLog	LogTable activityLogger (MW)	PB-10,11
Attendance Record	attendance_records	attendanceModel	attendanceController	Attendance	—	PB-20
Employee Document	employee_documents	employeeDocumentModel	employeeDocumentController	Employees	EmployeeDocumentUpload DocumentPreviewModal	PB-16
Integration Key	integration_keys	integrationKeyModel	integrationController	Integration	apiKeyMiddleware (MW)	PB-02

4.3.2 Behavioural View

Untuk merepresentasikan perilaku dinamis perangkat lunak, bagian *Behavioural View* ini menjabarkan setiap *use case* ke dalam bentuk Skenario Use Case dan Sequence Diagram pada bagian-bagian selanjutnya. Interaksi antar-objek pada pemodelan ini merupakan representasi instansiasi kelas dari *Class Diagram* (Gambar 4.3). Seluruh pemodelan dirancang dengan memisahkan skenario normal dan alternatif secara spesifik, baik melalui baris pemisah pada tabel maupun penggunaan fragmen *alt* pada diagram.

4.3.2.1 Skenario Use Case 01

Nama Use Case: Login

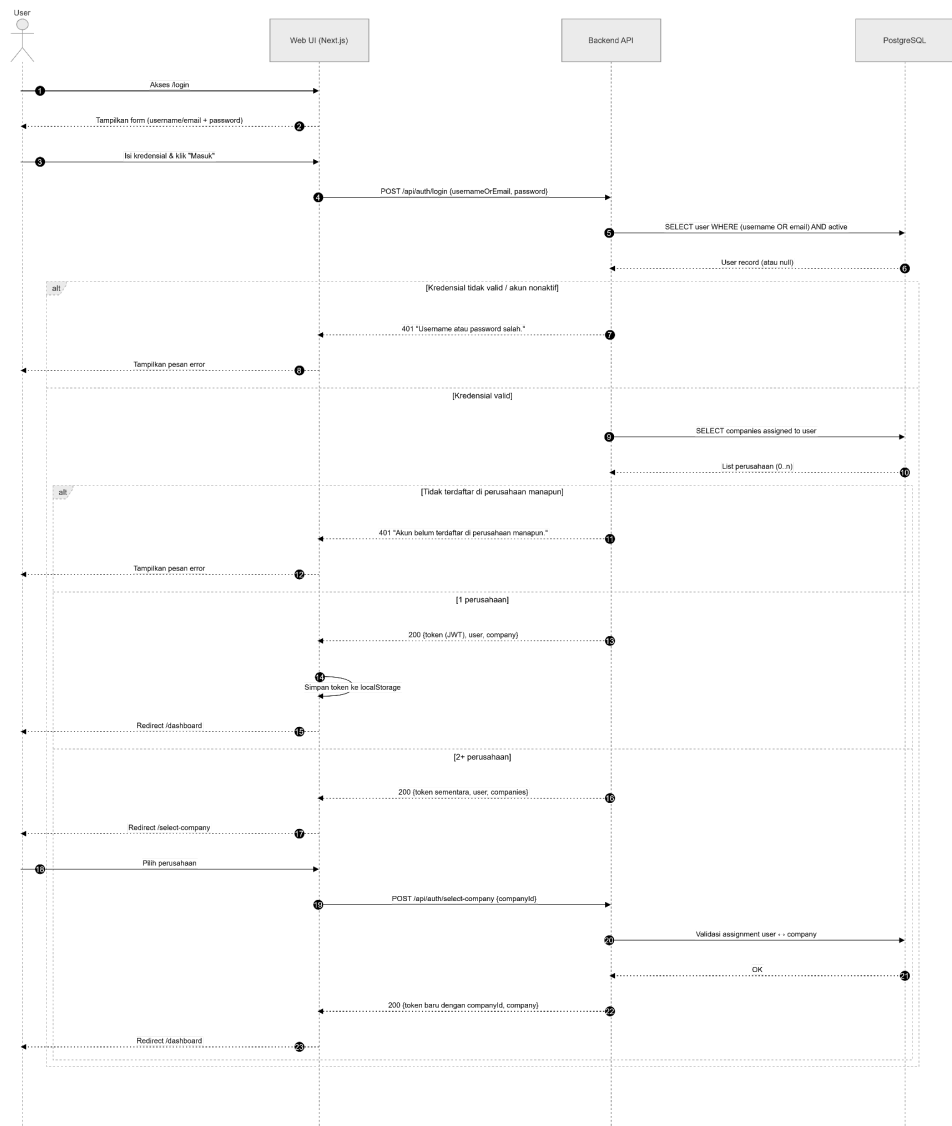
Skenario:

Tabel 4.3.2.1 Tabel Skenario Use Case 01

Aksi Actor	Reaksi Sistem
<i>Skenario Normal (Multi Perusahaan)</i>	
1. Mengakses halaman login dan memasukkan username/email serta password, lalu menekan tombol masuk	
	2. Memvalidasi kredensial dan status akun dengan database
	3. Sistem mendeteksi user memiliki akses ke lebih dari 1 perusahaan
	4. Mengarahkan user ke halaman "Pilih Perusahaan"

5. Memilih perusahaan yang ingin diakses	
	6. Membuat sesi aktif user dengan konteks perusahaan terpilih dan mengarahkan ke halaman <i>Dashboard</i> utama
<i>Skenario Alternatif 1 (Hanya 1 Perusahaan)</i>	
1. Mengakses halaman login dan memasukkan username/email serta password, lalu menekan tombol masuk	
	2. Memvalidasi kredensial dengan database
	3. Sistem mendeteksi user hanya memiliki akses ke 1 perusahaan
	4. Membuat sesi aktif user secara otomatis dengan perusahaan tersebut dan mengarahkan ke halaman <i>Dashboard</i> utama
<i>Skenario Alternatif (Kredensial Salah)</i>	
1. Memasukkan username/email atau password yang tidak sesuai	
	2. Menolak akses dan menampilkan pesan error "Username atau password salah"
<i>Skenario Alternatif (Akun Belum Terdaftar di Perusahaan Manapun)</i>	
1. Mengakses halaman login dan memasukkan username/email serta password, lalu menekan tombol masuk	
	2. Memvalidasi kredensial dengan database
	3. Sistem mendeteksi user belum memiliki akses ke perusahaan manapun
	3. Menolak akses dan menampilkan pesan error "Akun belum terdaftar di perusahaan manapun."
<i>Skenario Alternatif (Akun Nonaktif)</i>	
1. Mengakses halaman login menggunakan akun yang berstatus nonaktif	

	2. Memvalidasi kredensial dan memeriksa status akun
	3. Menolak akses dan menampilkan pesan "Akun Anda telah dinonaktifkan atau dihapus"



Gambar 4.4 Sequence Diagram UC-01 Login

[Gambar 4.4 Sequence Diagram UC-01 Login](#)

Gambar 4.4 menampilkan interaksi antar objek pada skenario UC-01. Objek yang berinteraksi merupakan instansiasi kelas pada diagram kelas (Gambar 4.3). Fragmen alt digunakan untuk memisahkan skenario alternatif dari skenario normal.

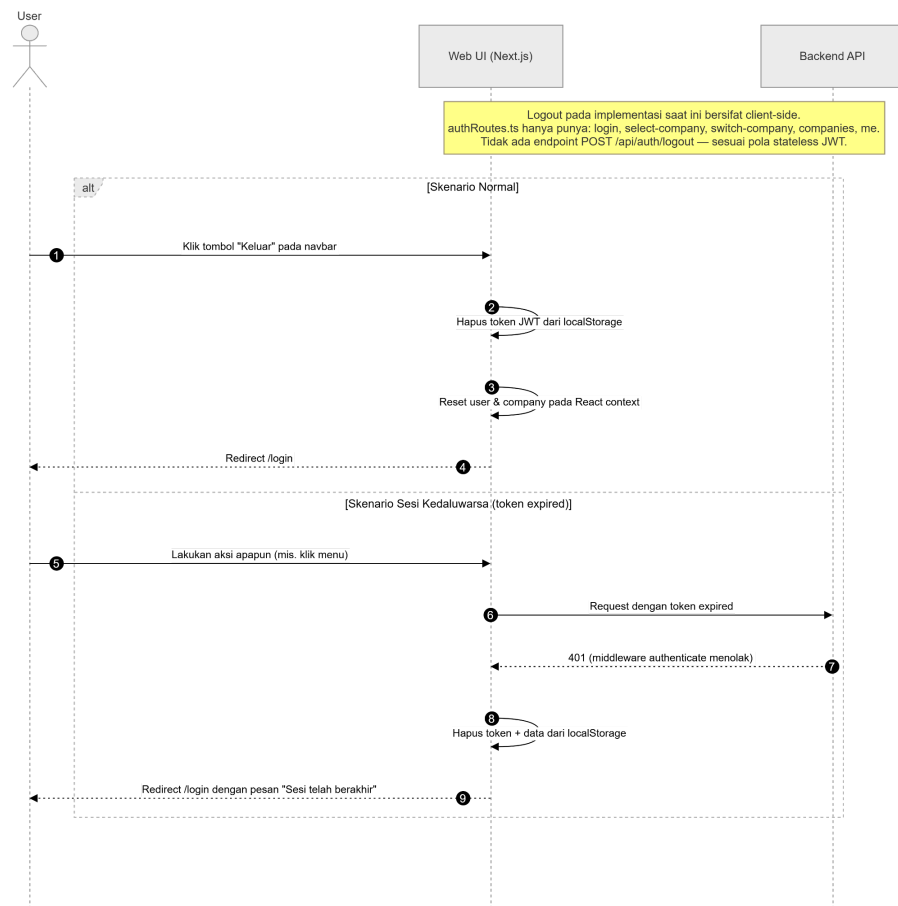
4.3.2.2 Skenario Use Case 02

Nama Use Case: Logout

Skenario:

Tabel 4.3.2.2 Tabel Skenario Use Case 02

<i>Aksi Actor</i>	<i>Reaksi Sistem</i>
<i>Skenario Normal</i>	
1. Menekan tombol "Logout" atau "Keluar" pada menu navigasi	
	2. Menghapus sesi aktif user di sistem
	3. Mengarahkan user kembali ke halaman Login
<i>Skenario Alternatif (Sesi sudah kedaluarsa)</i>	
1. Menekan tombol "Logout" saat sesi idle terlalu lama	
	2. Sistem mendeteksi sesi sudah habis dan langsung mengarahkan ke halaman Login



Gambar 4.5 Sequence Diagram UC-02 Logout

[Gambar 4.5 Sequence Diagram UC-02 Logout](#)

Gambar 4.5 menampilkan interaksi antar objek pada skenario UC-02. Objek yang berinteraksi merupakan instansiasi kelas pada diagram kelas (Gambar 4.3). Fragmen alt digunakan untuk memisahkan skenario alternatif dari skenario normal.

4.3.2.3 Skenario Use Case 03

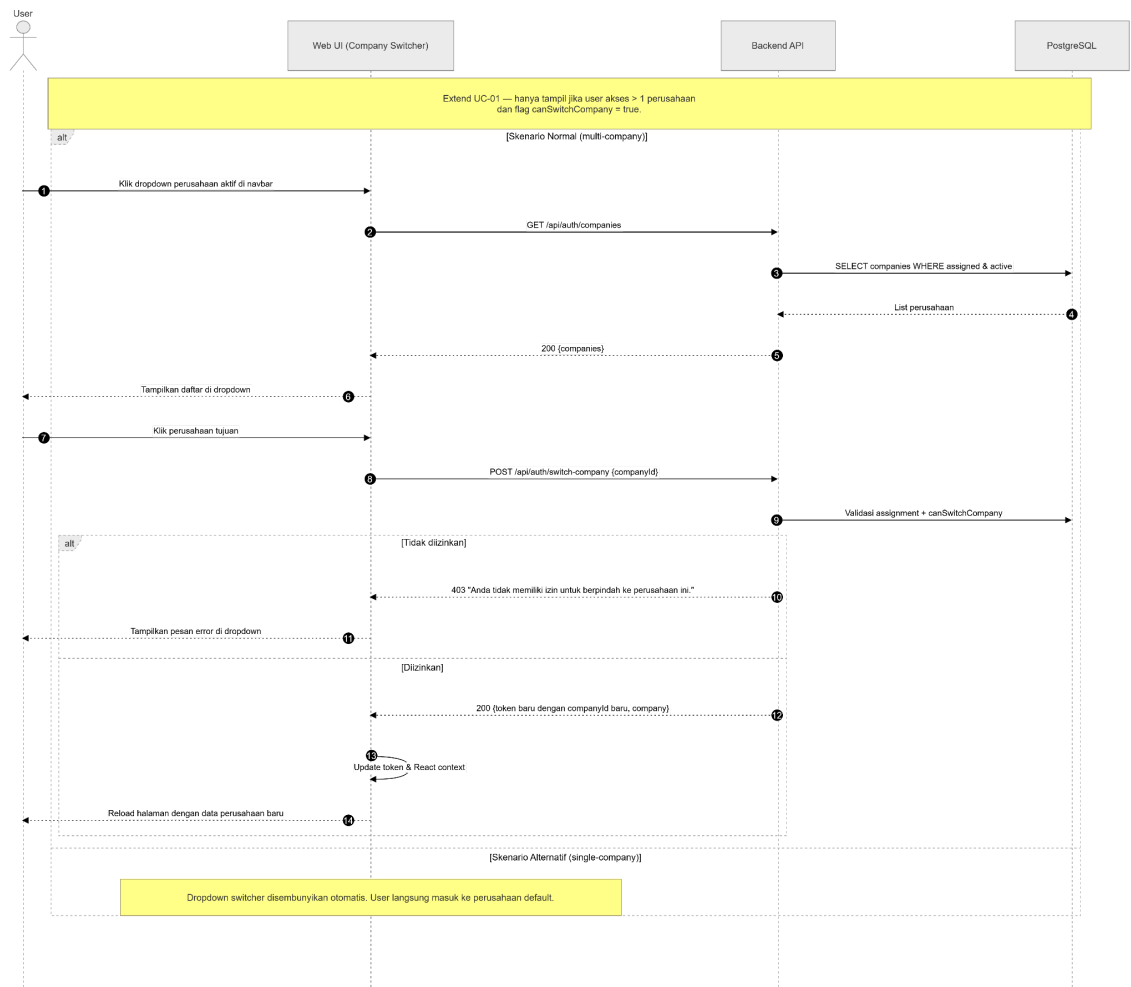
Nama Use Case: Pindah Perusahaan Aktif (Extend UC-01: Hanya muncul jika user memiliki akses > 1 perusahaan)

Skenario:

Tabel 4.3.2.3 Tabel Skenario Use Case 03

<i>Aksi Actor</i>	<i>Reaksi Sistem</i>
<i>Skenario Normal</i>	
1. Menekan menu dropdown perusahaan aktif di header aplikasi	
	2. Menampilkan daftar perusahaan yang diizinkan (di-assign) untuk user tersebut

3. Memilih salah satu perusahaan dari daftar	
	4. Memperbarui konteks sesi (Session Context) perusahaan aktif tanpa melakukan logout
	5. Me-reload halaman dan menampilkan data sesuai perusahaan yang baru dipilih
<i>Skenario Alternatif (Hanya 1 Perusahaan / Tidak ada akses extend)</i>	
1. User dengan akses hanya ke 1 perusahaan login	
	2. Sistem menyembunyikan menu dropdown pindah perusahaan (otomatis masuk ke perusahaan default)



Gambar 4.6 Sequence Diagram UC-03 Pindah Perusahaan Aktif

[Gambar 4.6 Sequence Diagram UC-03 Pindah Perusahaan Aktif](#)

Gambar 4.6 menampilkan interaksi antar objek pada skenario UC-03. Objek yang berinteraksi merupakan instansiasi kelas pada diagram kelas (Gambar 4.3). Fragmen alt digunakan untuk memisahkan skenario alternatif dari skenario normal.

4.3.2.4 Skenario Use Case 04

Nama Use Case: *Kelola User*

Skenario:

Tabel 4.3.2.4 Tabel Skenario Use Case 04

Aksi Aktor	Reaksi Sistem
<i>Skenario Normal Tambah / Edit User</i>	
1. Mengakses menu "Master Pegawai" dan menekan tombol "Tambah Pengguna" atau ikon edit pada baris user tertentu.	
	2. Menampilkan form data user (nama, email, NIK, role, departemen, jabatan, grade, flag superadmin, pilihan assignment perusahaan).
3. Mengisi data akun, memilih role (user / admin / superadmin), mengatur assignment perusahaan, lalu menekan "Simpan".	
	4. employeePayrollController memvalidasi data, menyimpan ke tabel users dan user_companies, lalu menampilkan Toast sukses.
<i>Skenario Alternatif Email / Username Sudah Digunakan</i>	
1. Memasukkan email/username yang sudah terdaftar di sistem lalu menekan Simpan.	
	2. Validasi di userModel mendeteksi duplikasi; controller mengembalikan HTTP 409. Frontend menampilkan pesan error "Gagal Membuat Pengguna: email/username sudah digunakan".

Gambar 4.7 Sequence Diagram UC-04 Kelola User

Gambar 4.7 menampilkan interaksi antar objek pada skenario UC-04. Objek yang berinteraksi merupakan instansiasi kelas pada diagram kelas (Gambar 4.3). Fragmen alt digunakan untuk memisahkan skenario alternatif dari skenario normal.

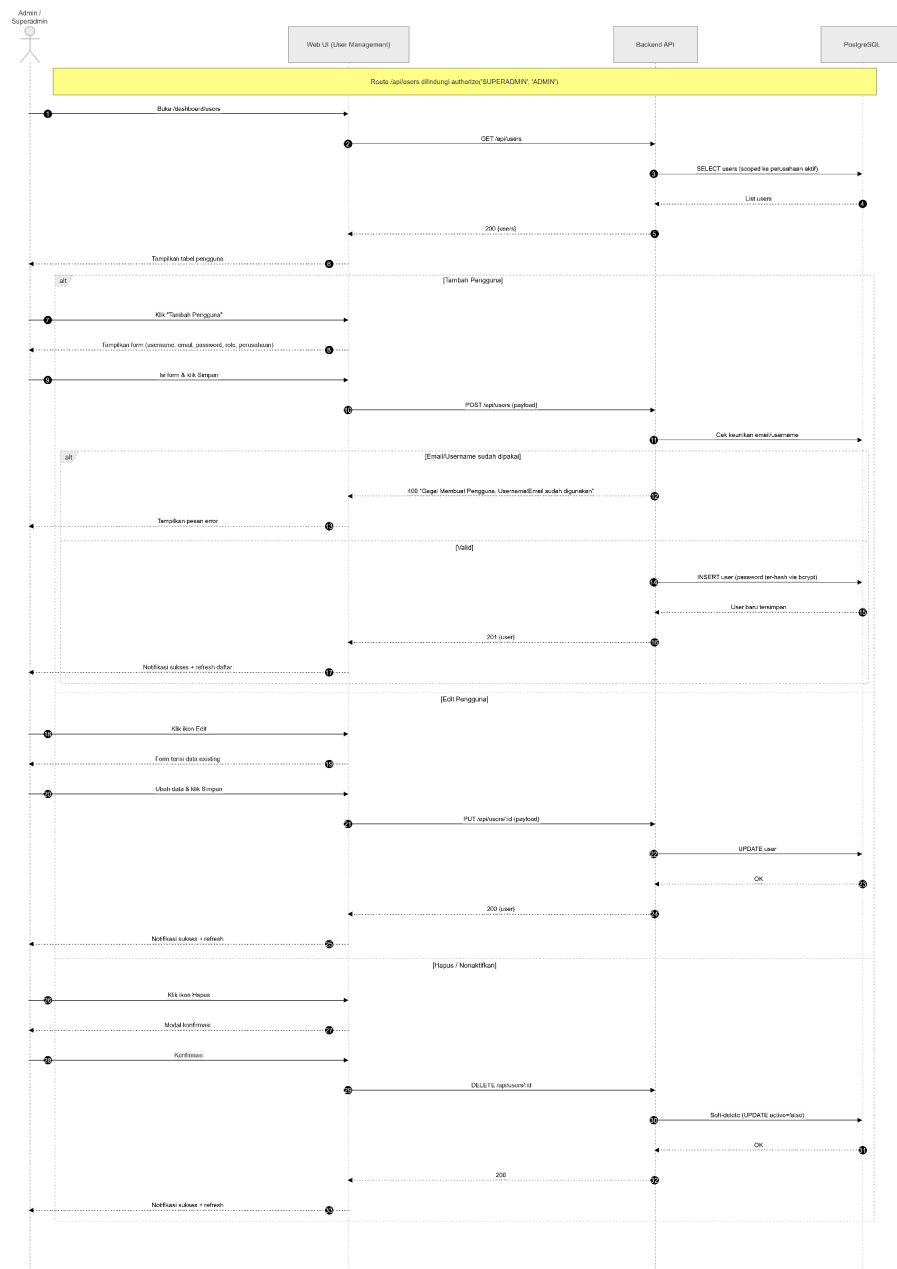
4.3.2.5 Skenario Use Case 05

Nama Use Case: *Kelola Hak Akses User (Include UC-06: Selalu berhubungan dengan hak akses terhadap konteks/perusahaan tertentu)*

Skenario:

Tabel 4.3.2.5 Tabel Skenario Use Case 05

<i>Aksi Actor</i>	<i>Reaksi Sistem</i>
<i>Skenario Normal</i>	
1. Memilih seorang user di menu Hak Akses	
	2. Menampilkan matriks menu dan jenis aksi (Create, Read, Update, Delete) yang tersedia
3. Mencentang/menghapus centang pada menu dan aksi yang diizinkan, lalu menyimpan	
	4. Memperbarui tabel roles/permissions di database dan menampilkan pesan sukses
<i>Skenario Alternatif (Menghapus akses diri sendiri)</i>	
1. Admin secara tidak sengaja mencoba menghapus hak akses "Kelola Hak Akses" pada akunnya sendiri	
	2. Memblokir aksi tersebut dan menampilkan peringatan "Anda tidak dapat menghapus hak akses ini pada akun Anda sendiri"



Gambar 4.8 Sequence Diagram UC-05 Kelola Hak Akses User

[Gambar 4.8 Sequence Diagram UC-05 Kelola Hak Akses User](#)

Gambar 4.8 menampilkan interaksi antar objek pada skenario UC-05. Objek yang berinteraksi merupakan instansiasi kelas pada diagram kelas (Gambar 4.3). Fragmen alt digunakan untuk memisahkan skenario alternatif dari skenario normal.

4.3.2.6 Skenario Use Case 06

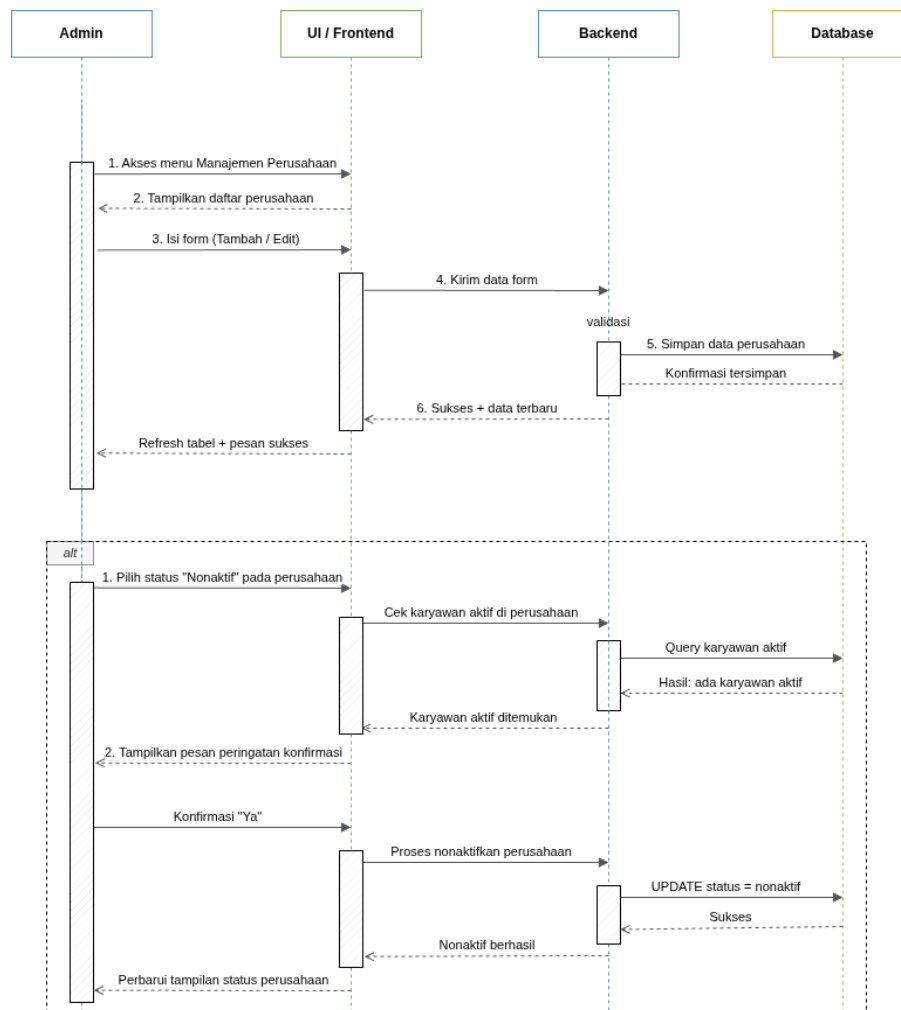
Nama Use Case: Kelola Perusahaan

Skenario:

Tabel 4.3.2.6 Tabel Skenario Use Case 06

<i>Aksi Actor</i>	<i>Reaksi Sistem</i>
-------------------	----------------------

<i>Skenario Normal</i>	
1. Mengakses menu "Manajemen Perusahaan"	
	2. Menampilkan daftar perusahaan
3. Mengisi form (Tambah/Edit)	
	4. Memvalidasi input form
	5. Menyimpan perubahan data perusahaan ke dalam database
	6. Me-refresh tabel daftar perusahaan dan menampilkan pesan sukses
<i>Skenario Alternatif (Menonaktifkan perusahaan yang sedang aktif digunakan user lain)</i>	
1. Memilih status "Nonaktif" untuk perusahaan yang masih memiliki karyawan aktif	
	2. Menampilkan pesan peringatan konfirmasi "Perusahaan ini memiliki karyawan aktif, yakin ingin menonaktifkan?" sebelum memproses



Gambar 4.9 Sequence Diagram UC-06 Kelola Perusahaan

Gambar 4.9 menampilkan interaksi antar objek pada skenario UC-06. Objek yang berinteraksi merupakan instansiasi kelas pada diagram kelas (Gambar 4.3). Fragmen alt digunakan untuk memisahkan skenario alternatif dari skenario normal.

4.3.2.7 Skenario Use Case 07

Nama Use Case: Kelola Departemen dan Jabatan

Tabel 4.3.2.7 Tabel Skenario Use Case 07

Aksi Actor	Reaksi Sistem
<i>Skenario Normal</i>	
1. Mengakses menu "Jabatan & Departemen" dan menambahkan/mengubah departemen atau jabatan	
	2. Menampilkan struktur organisasi yang tersedia

3. Menambah atau mengubah data departemen maupun jabatan	
	2. Menyimpan perubahan hierarki/data departemen dan jabatan ke database
	3. Memperbarui tampilan tree atau tabel struktur organisasi
Skenario Alternatif (Menghapus departemen yang berisi karyawan)	
1. Menekan tombol "Hapus" pada departemen/jabatan yang masih memiliki relasi dengan karyawan	
	2. Membatalkan penghapusan dan menampilkan pesan error "Departemen tidak dapat dihapus karena masih digunakan oleh data karyawan"



Gambar 4.10 Sequence Diagram UC-07 Kelola Struktur Organisasi

Gambar 4.10 menampilkan interaksi antar objek pada skenario UC-07. Objek yang berinteraksi merupakan instansiasi kelas pada diagram kelas (Gambar 4.3). Fragmen alt digunakan untuk memisahkan skenario alternatif dari skenario normal.

4.3.2.8 Skenario Use Case 08

Nama Use Case: *Kelola Data Karyawan*

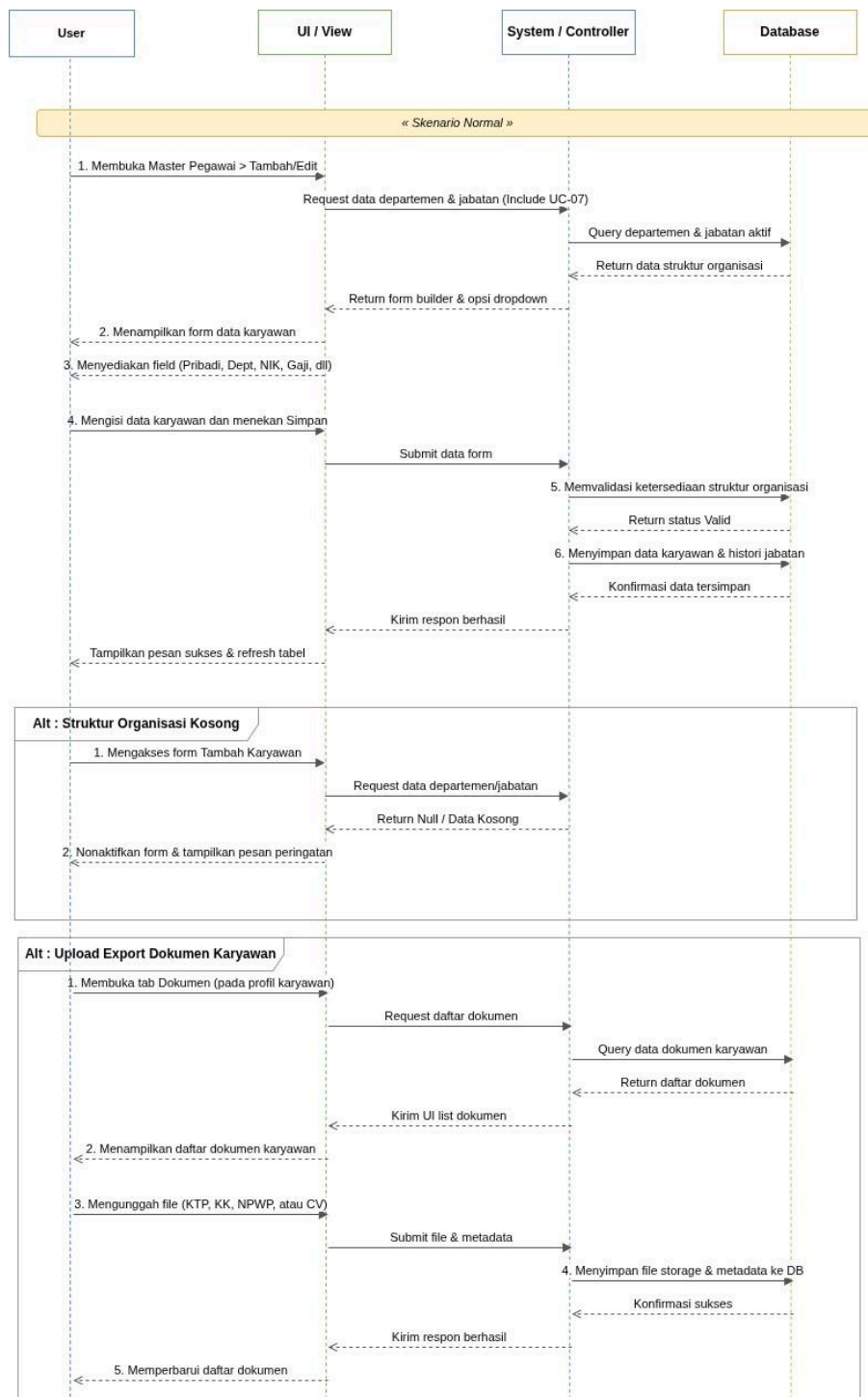
(Include UC-07: *Membutuhkan data departemen & jabatan yang sudah ada*)

Skenario:

Tabel 4.3.2.8 Tabel Skenario Use Case 08

<i>Aksi Actor</i>	<i>Reaksi Sistem</i>
<i>Skenario Normal</i>	
1. Membuka menu Master Pegawai dan memilih Tambah atau Edit Karyawan	
	2. Menampilkan form data karyawan
	3. Menyediakan field data pribadi, departemen, jabatan, NIK, grade, gaji manual, offset masa kerja, dan tunjangan keluarga (%)

4. Mengisi data karyawan dan menekan Simpan	
	5. Memvalidasi ketersediaan struktur organisasi yang dipilih
	6. Menyimpan data karyawan beserta histori (jika ada perubahan jabatan) ke database
<i>Skenario Alternatif (Struktur organisasi kosong)</i>	
1. Mengakses form "Tambah Karyawan" ketika data departemen/jabatan belum pernah dibuat	
	2. Sistem menonaktifkan form dan menampilkan pesan "Harap buat Departemen dan Jabatan (Struktur Organisasi) terlebih dahulu"
<i>Skenario Alternatif (Export dokumen)</i>	
1. Admin/Superadmin mengakses menu "Master Pegawai" dan membuka tab dokumen	
	2. Menampilkan daftar dokumen karyawan
3. Admin mengunggah dokumen KTP, KK, NPWP, atau CV	
	4. Menyimpan file dan metadata dokumen
	5. Memperbarui daftar dokumen karyawan



Gambar 4.11 Sequence Diagram UC-08 Kelola Data Karyawan

Gambar 4.11 menampilkan interaksi antar objek pada skenario UC-08. Objek yang berinteraksi merupakan instansiasi kelas pada diagram kelas (Gambar 4.3). Fragmen alt digunakan untuk memisahkan skenario alternatif dari skenario normal.

4.3.2.9 Skenario Use Case 09

*Nama Use Case: Kelola Komponen Gaji
(Include UC-08: Membutuhkan data karyawan yang valid)*

Skenario:

Tabel 4.3.2.09 Tabel Skenario Use Case 09

<i>Aksi Actor</i>	<i>Reaksi Sistem</i>
<i>Skenario Normal</i>	
1. Membuka menu “Master Komponen Gaji”	
	2. Menampilkan daftar komponen gaji dan form komponen gaji
3. Membuat atau mengubah komponen gaji dengan melengkapi form dan formula untuk komponen tersebut	
	4. Melakukan validasi data dan formula komponen
	5. Menyimpan komponen gaji ke database
	6. Memperbarui daftar komponen gaji
<i>Skenario Alternatif (Formula tidak valid)</i>	
1. Membuat komponen dengan formula yang tidak valid	
	2. Sistem menolak untuk menyimpan perubahan dan menampilkan pesan “Kode Tidak Valid”



Gambar 4.12 Sequence Diagram UC-09 Kelola Komponen Gaji

Gambar 4.12 menampilkan interaksi antar objek pada skenario UC-09. Objek yang berinteraksi merupakan instansiasi kelas pada diagram kelas (Gambar 4.3). Fragmen alt digunakan untuk memisahkan skenario alternatif dari skenario normal.

4.3.2.10 Skenario Use Case 10

Nama Use Case: *Proses Penggajian*

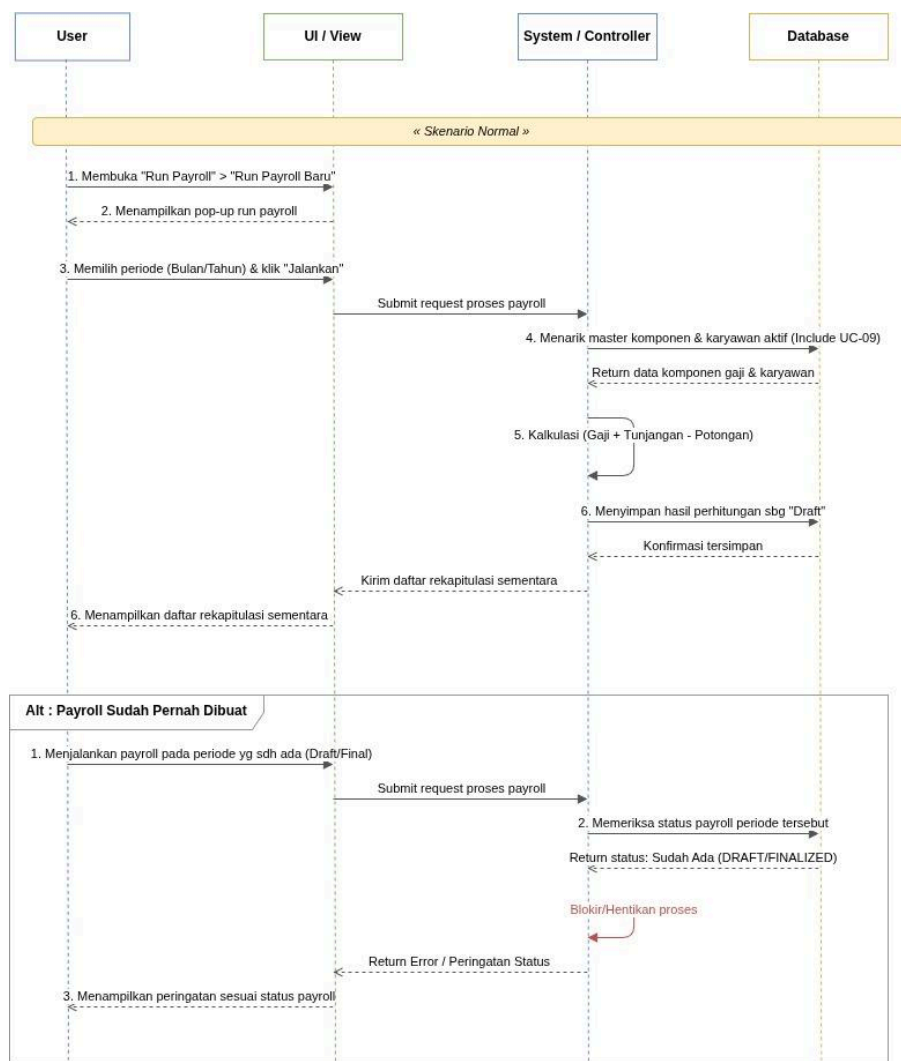
(Include UC-09: *Komponen gaji harus sudah ada*)

Skenario:

Tabel 4.3.2.10 Tabel Skenario Use Case 10

<i>Aksi Actor</i>	<i>Reaksi Sistem</i>
<i>Skenario Normal</i>	
1. Membuka menu “Run Payroll” dan mengakses “Run Payroll Baru”	
	2. Menampilkan pop-up run payroll
3. Memilih periode penggajian (Bulan/Tahun) dan menekan tombol "Jalankan"	

	4. Menarik data master komponen gaji seluruh karyawan aktif
	5. Melakukan kalkulasi (Gaji Pokok + Tunjangan - Potongan)
	6. Menyimpan hasil perhitungan sebagai status Draft dan menampilkan daftar rekapitulasi sementara
<i>Skenario Alternatif (Payroll sudah pernah dibuat)</i>	
1. Menjalankan payroll pada periode yang sudah memiliki payroll DRAFT atau FINALIZED	
	2. Sistem memeriksa status payroll periode tersebut
	3. Menampilkan peringatan sesuai status payroll yang ditemukan



Gambar 4.13 Sequence Diagram UC-10 Proses Penggajian

Gambar 4.13 menampilkan interaksi antar objek pada skenario UC-10. Objek yang berinteraksi merupakan instansiasi kelas pada diagram kelas (Gambar 4.3). Fragmen alt digunakan untuk memisahkan skenario alternatif dari skenario normal.

4.3.2.11 Skenario Use Case 11

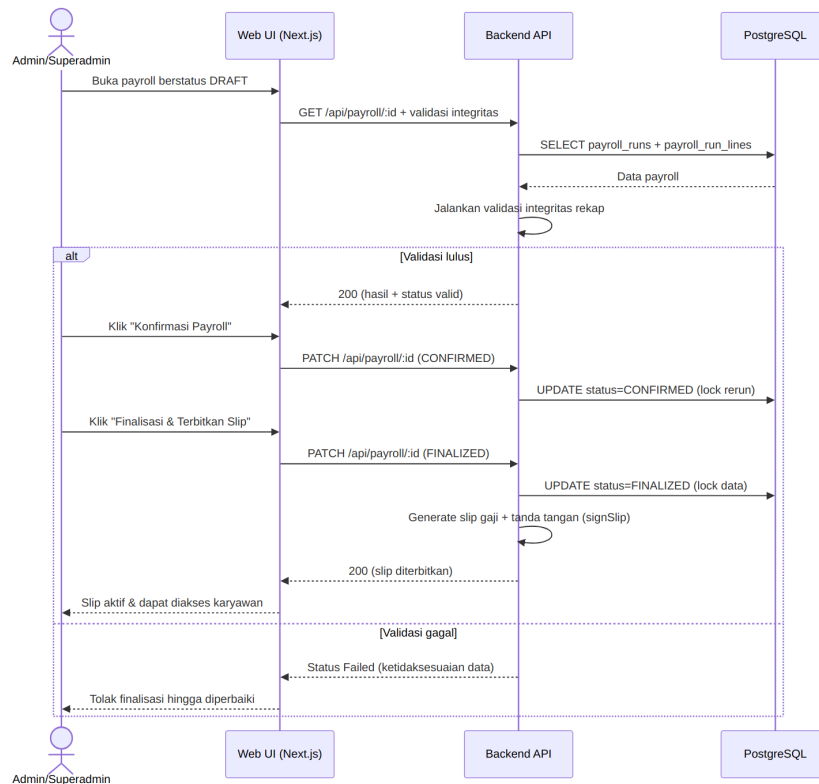
*Nama Use Case: Konfirmasi / Finalisasi Penggajian
(Include UC-10: Hanya bisa jika draft sudah terbentuk)*

Skenario:

Tabel 4.3.2.11 Tabel Skenario Use Case 11

<i>Aksi Actor</i>	<i>Reaksi Sistem</i>
<i>Skenario Normal</i>	
1. Admin/Superadmin membuka payroll berstatus DRAFT	
	2. Menampilkan hasil payroll dan validasi integritas

3. Admin/Superadmin menekan tombol Konfirmasi Payroll	
	4. Mengubah status menjadi CONFIRMED dan mengunci (locking) payroll dari proses rerun biasa
5. Superadmin membuka payroll berstatus CONFIRMED dan menekan tombol “Finalisasi & Terbitkan Slip”	
	6. Mengubah status data payroll periode tersebut dari CONFIRMED menjadi FINALIZED
	7. Mengunci (Lock) data agar komponen gajinya tidak bisa diubah lagi untuk periode ini
	8. Menerbitkan slip gaji digital dan mengaktifkan akses slip gaji bagi karyawan
<i>Skenario Alternatif (Validasi gagal)</i>	
1. Menjalankan Validasi Integritas	
	2. Sistem menemukan ketidaksesuaian data
	3. Menampilkan status Failed
	4. Menolak proses finalisasi hingga masalah diperbaiki
<i>Skenario Alternatif (Menjalankan ulang payroll)</i>	
1. Admin/Superadmin menemukan kesalahan pada hasil payroll berstatus DRAFT	
	2. Admin melakukan perbaikan data pada Master Pegawai, Komponen Gaji, Kehadiran, atau data terkait lainnya
3. Admin kembali ke menu Payroll Run dan memilih periode yang sama	
	4. Sistem mendeteksi bahwa payroll untuk periode tersebut sudah tersedia dan masih berstatus DRAFT
	5. Menampilkan peringatan bahwa data payroll sebelumnya akan ditimpa
6. Admin mengonfirmasi proses jalankan ulang payroll	
	7. Sistem memperbarui data payroll dan menampilkan hasil payroll yang sudah diperbarui



Gambar 4.14 Sequence Diagram UC-11 Konfirmasi / Finalisasi Penggajian

Gambar 4.14 menampilkan interaksi antar objek pada skenario UC-11. Objek yang berinteraksi merupakan instansiasi kelas pada diagram kelas (Gambar 4.3). Fragmen alt digunakan untuk memisahkan skenario alternatif dari skenario normal.

4.3.2.12 Skenario Use Case 12

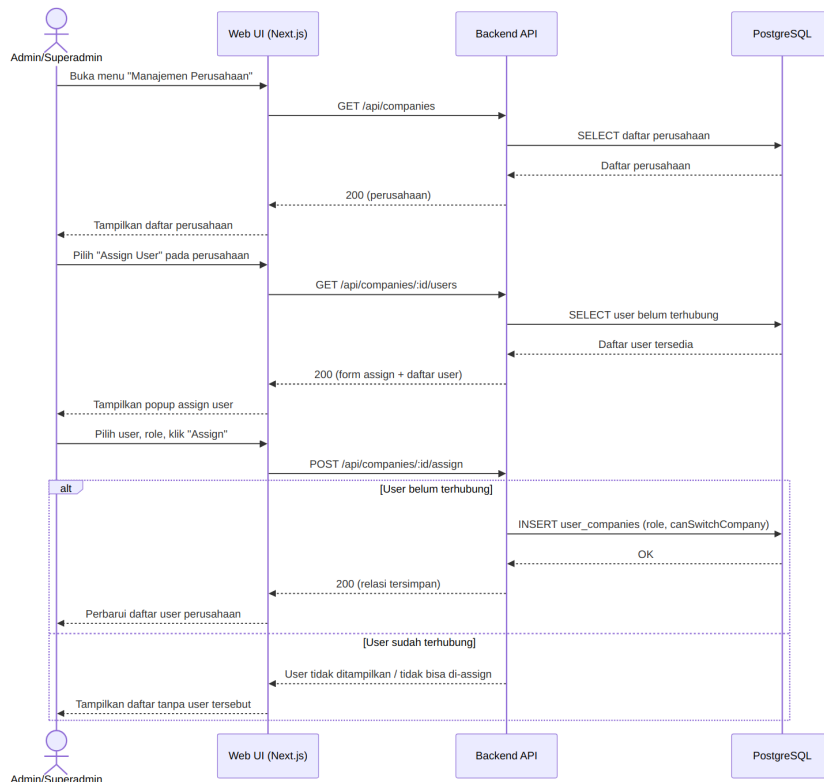
Nama Use Case: *Lihat Rekap & Export Penggajian*
(Extend UC-11: *Dilakukan setelah finalisasi untuk pelaporan*)

Skenario:

Tabel 4.3.2.12 Tabel Skenario Use Case 12

Aksi Actor	Reaksi Sistem
<i>Skenario Normal</i>	
1. Mengakses menu "Rekap Penggajian"	
	2. Menampilkan daftar periode payroll
3. Memilih periode payroll	
	4. Menampilkan ringkasan payroll
5. Menekan tombol Export Excel/CSV	
	6. Mengompilasi data payroll berdasarkan periode dan mengelompokkannya per departemen

	7. Membangkitkan file berformat Excel/CSV dan memberikannya (download) kepada user
	8. Sistem menampilkan status tiket
<i>Skenario Alternatif (Filter dan Pencarian)</i>	
1. Mengisi nama pegawai atau departemen pada kolom pencarian	
	2. Memfilter data, memperbarui isi tabel dan nilai "Total Terfilter" secara real-time



Gambar 4.15 Sequence Diagram UC-12 Assign Pegawai ke Perusahaan

Gambar 4.15 menampilkan interaksi antar objek pada skenario UC-12. Objek yang berinteraksi merupakan instansiasi kelas pada diagram kelas (Gambar 4.3). Fragmen alt digunakan untuk memisahkan skenario alternatif dari skenario normal.

4.3.2.13 Skenario Use Case 13

Nama Use Case: Lihat Slip Gaji

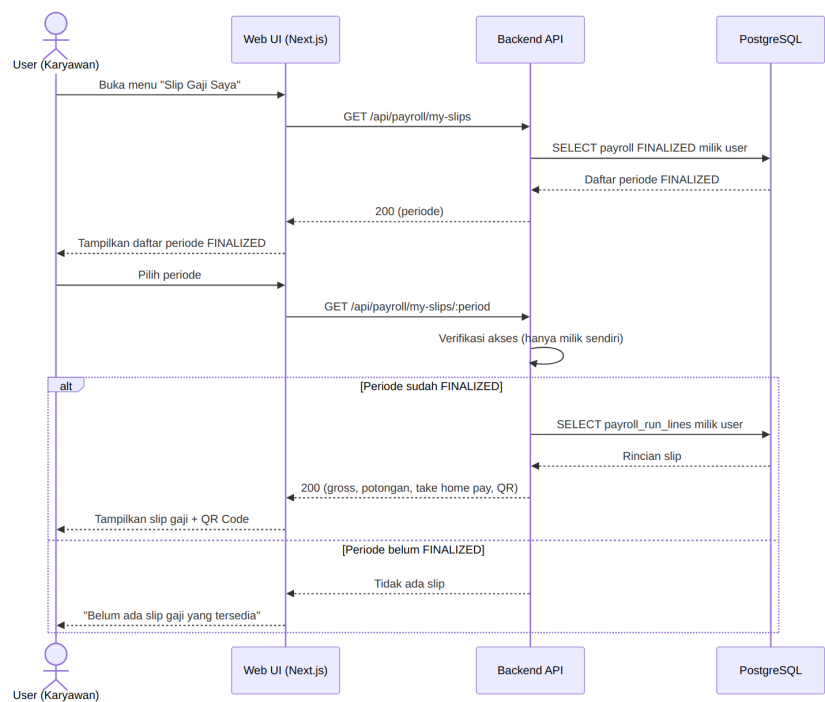
(Include UC-11: Hanya bisa jika data telah difinalisasi)

Skenario:

Tabel 4.3.2.13 Tabel Skenario Use Case 13

<i>Aksi Actor</i>	<i>Reaksi Sistem</i>
<i>Skenario Normal</i>	
1. Mengakses menu "Slip Gaji"	

	2. Menampilkan daftar periode payroll yang sudah berstatus FINALIZED
3. Memilih periode payroll	
	4. Melakukan verifikasi akses
	5. Menampilkan slip gaji berisi Gaji Pokok, Tunjangan, Potongan, dan Take Home Pay milik karyawan tersebut, serta QR Code verifikasi digital
<i>Skenario Alternatif (Melihat periode yang belum finalisasi)</i>	
1. Mengakses menu "Slip Gaji" dan memilih bulan berjalan (penggajian belum selesai diproses/final)	
	2. Menampilkan layar kosong dengan pesan "Belum ada slip gaji yang tersedia"



Gambar 4.15 Sequence Diagram UC-13 Lihat Slip Gaji

Gambar 4.15 menampilkan interaksi antar objek pada skenario UC-13. Objek yang berinteraksi merupakan instansiasi kelas pada diagram kelas (Gambar 4.3). Fragmen alt digunakan untuk memisahkan skenario alternatif dari skenario normal.

4.3.2.14 Skenario Use Case 14

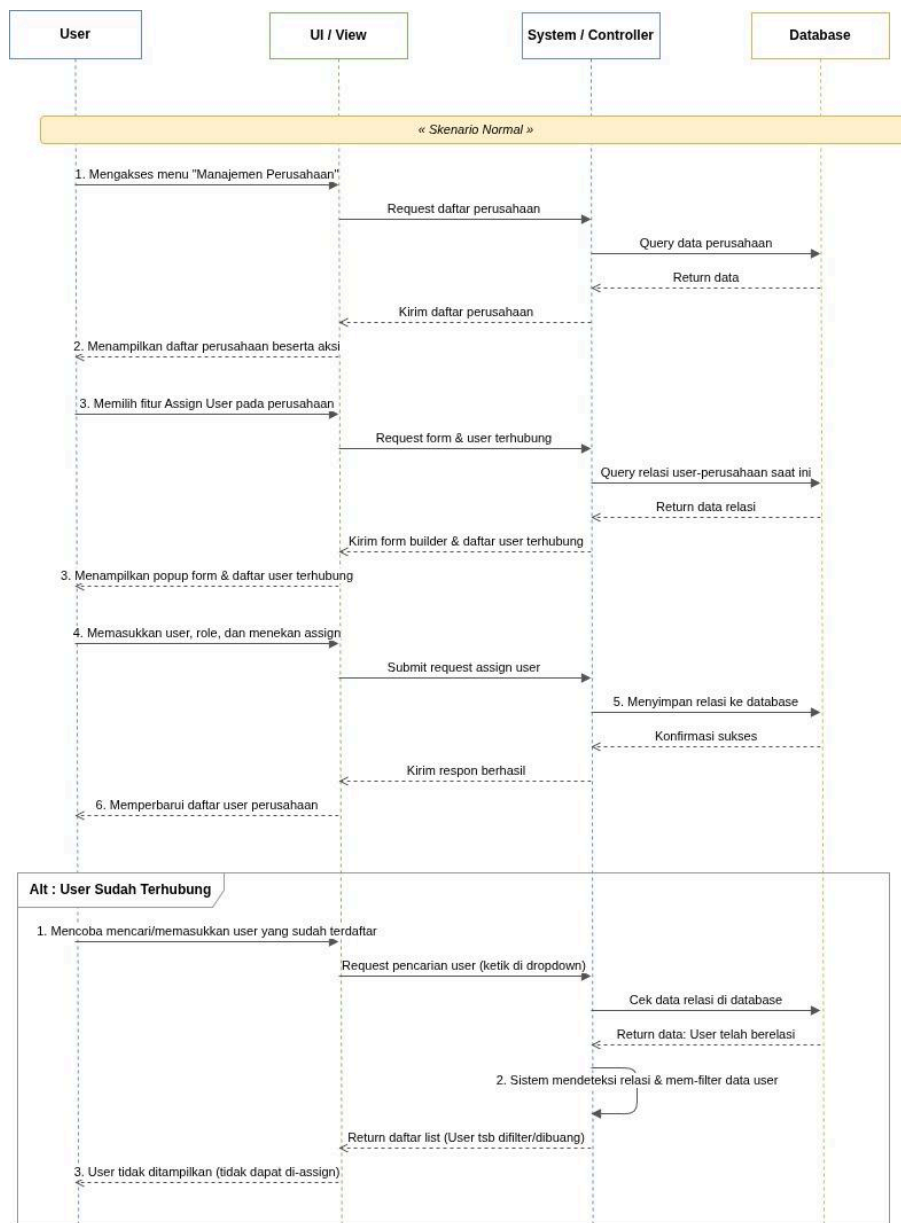
Nama Use Case: Assign Pegawai ke Perusahaan

Skenario:

Tabel 4.3.2.14 Tabel Skenario Use Case 14

<i>Aksi Actor</i>	<i>Reaksi Sistem</i>
-------------------	----------------------

<i>Skenario Normal</i>	
1. Mengakses menu "Manajemen Perusahaan"	
	2. Menampilkan daftar perusahaan beserta aksi yang dapat dilakukan
3. Memilih fitur Assign User pada perusahaan tertentu	
	3. Menampilkan popup berisi form assign pengguna dan daftar user yang sudah terhubung dengan perusahaan tersebut
4. Memasukkan nama/username pengguna, role, dan menekan assign	
	5. Menyimpan relasi user-perusahaan ke database
	6. Memperbarui daftar user perusahaan
<i>Skenario Alternatif (User Sudah Terhubung)</i>	
1. Mencoba menghubungkan user yang sudah terdaftar pada perusahaan tersebut	
	2. Sistem mendeteksi pengguna yang telah memiliki relasi dengan perusahaan tersebut
	3. User tidak akan ditampilkan sehingga tidak bisa di assign ke perusahaan tersebut



Gambar 4.16 Sequence Diagram UC-14 Lihat Slip Gaji

Gambar 4.16 menampilkan interaksi antar objek pada skenario UC-14. Objek yang berinteraksi merupakan instansiasi kelas pada diagram kelas (Gambar 4.3). Fragmen alt digunakan untuk memisahkan skenario alternatif dari skenario normal.

4.3.2.15 Skenario Use Case 15

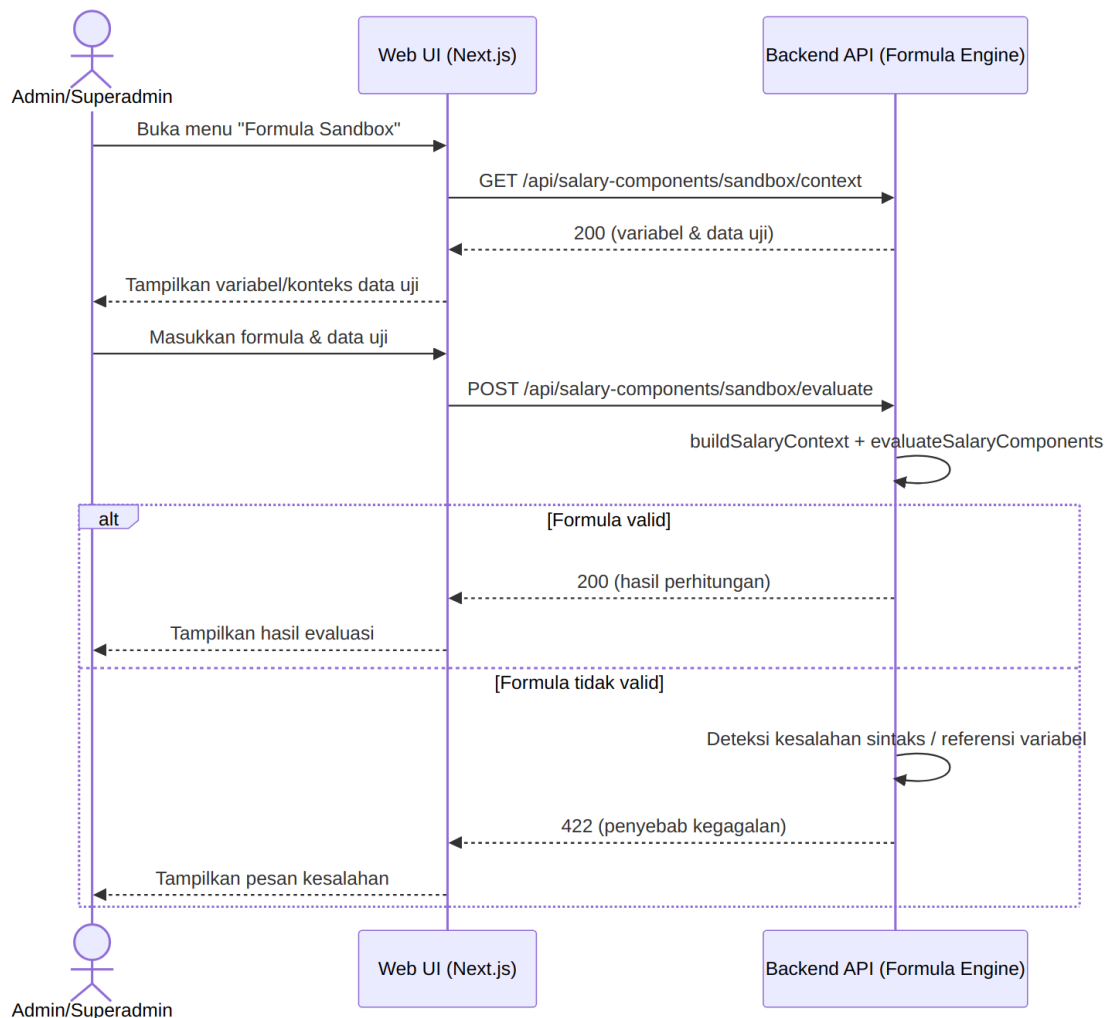
Nama Use Case: *Evaluasi Formula dengan Sandbox*

Skenario:

Tabel 4.3.2.15 Tabel Skenario Use Case 15

Aksi Actor	Reaksi Sistem
<i>Skenario Normal</i>	
1. Membuka menu "Formula Sandbox"	

	2. Menampilkan variabel dan konteks atau data uji yang dapat digunakan
3. Memasukkan formula dan data uji	
	4. Mengevaluasi formula
	5. Menampilkan hasil perhitungan
<i>Skenario Alternatif (Formula Tidak Valid)</i>	
1. Memasukkan formula yang tidak valid	
	2. Sistem mendeteksi kesalahan sintaks atau referensi variabel
	3. Menghentikan proses evaluasi
	4. Menampilkan pesan kesalahan beserta informasi penyebab kegagalan



Gambar 4.17 Sequence Diagram UC-15 Evaluasi Formula dengan Sandbox

Gambar 4.17 menampilkan interaksi antar objek pada skenario UC-15. Objek yang berinteraksi merupakan instansiasi kelas pada diagram kelas (Gambar 4.3). Fragmen alt digunakan untuk memisahkan skenario alternatif dari skenario normal.

4.3.2.16 Skenario Use Case 16

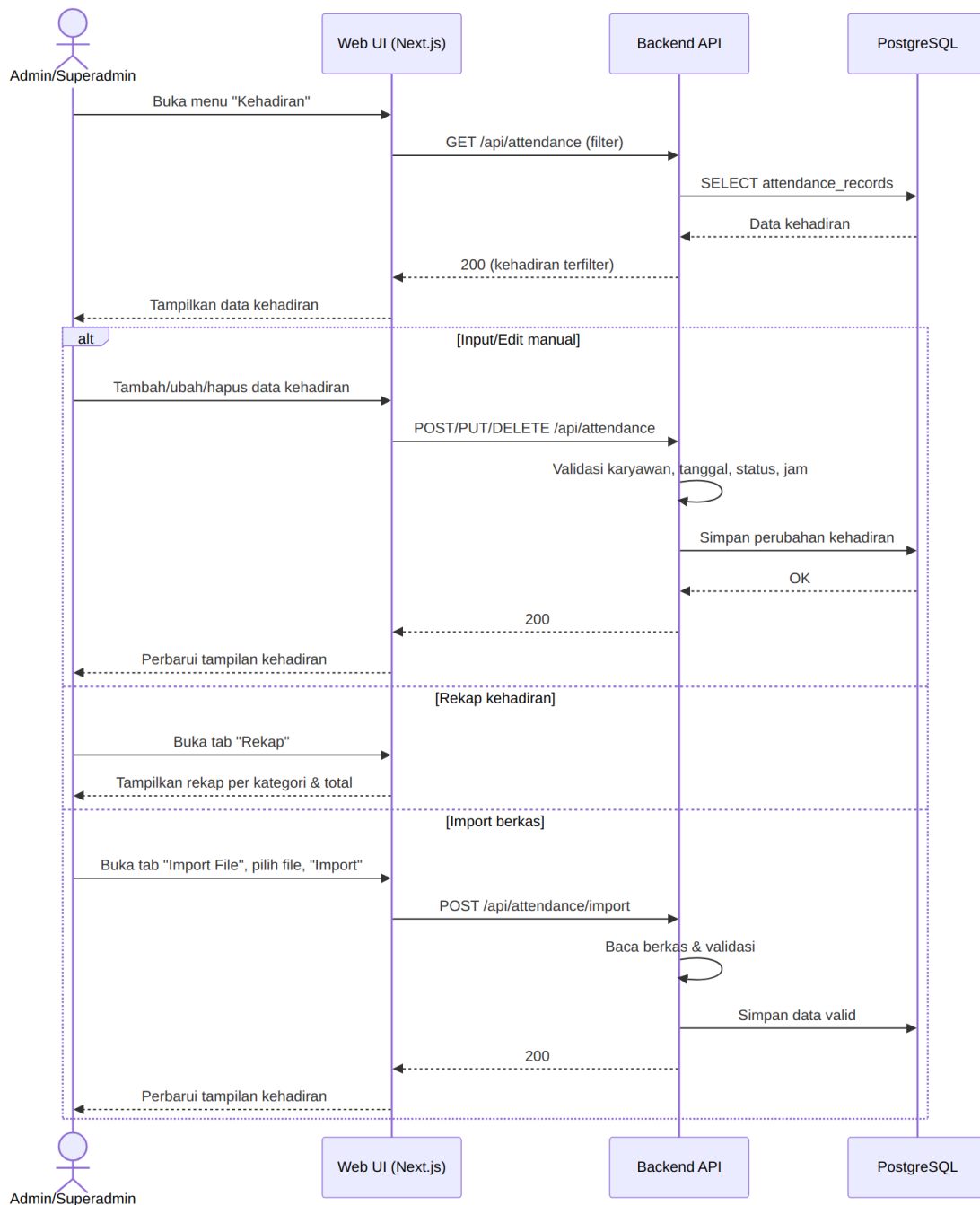
Nama Use Case: *Kelola Kehadiran*

Skenario:

Tabel 4.3.2.16 Tabel Skenario Use Case 16

<i>Aksi Actor</i>	<i>Reaksi Sistem</i>
<i>Skenario Normal</i>	
1. Membuka menu “Kehadiran”	
	2. Menampilkan kehadiran sesuai filter
3. Menambah, mengubah, atau menghapus data kehadiran	

	4. Memvalidasi karyawan, tanggal, status, jam masuk, dan jam keluar
	5. Menyimpan perubahan kehadiran ke database
	6. Memperbarui tampilan data kehadiran
<i>Skenario Alternatif (Rekap kehadiran)</i>	
1. Membuka tab “Rekap”	
	2. Menampilkan rekap kehadiran berupa jumlah kehadiran atau ketidakhadiran berdasarkan kategori dan total kehadiran sesuai filter
<i>Skenario Alternatif (Import kehadiran)</i>	
1. Membuka tab “Import File”, memilih file, dan menekan “Import”	
	2. Membaca file, melakukan validasi, dan menyimpan data valid
	3. Sistem memperbarui tampilan kehadiran



Gambar 4.18 Sequence Diagram UC-16 Kelola Kehadiran

Gambar 4.18 menampilkan interaksi antar objek pada skenario UC-16. Objek yang berinteraksi merupakan instansiasi kelas pada diagram kelas (Gambar 4.3). Fragmen alt digunakan untuk memisahkan skenario alternatif dari skenario normal.

4.3.2.17 Skenario Use Case 17

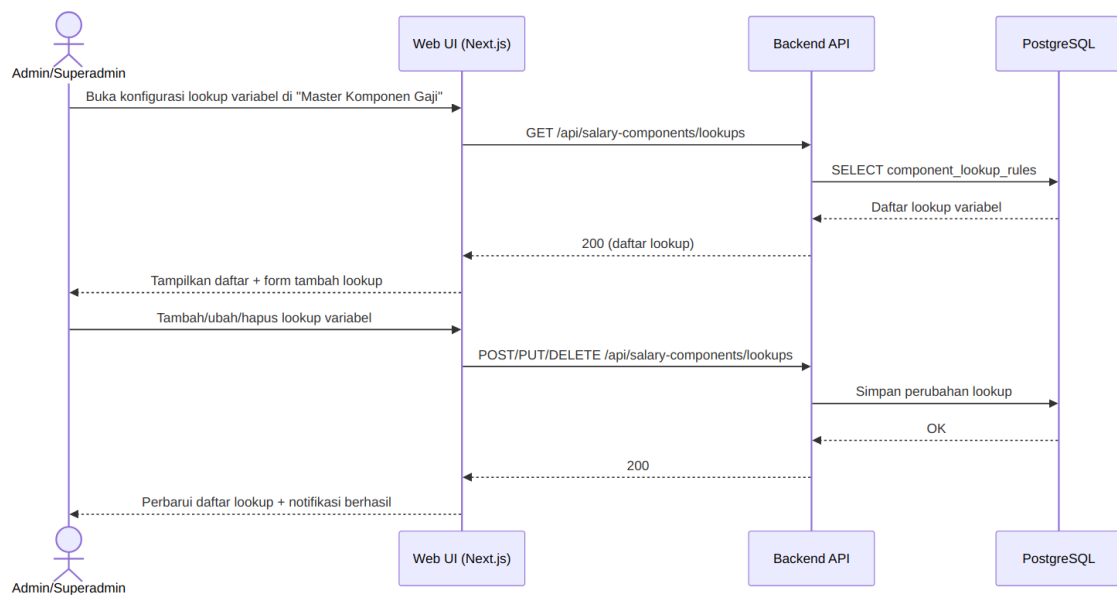
Nama Use Case: *Kelola Lookup Variabel Komponen Gaji*

Skenario:

Tabel 4.3.2.17 Tabel Skenario Use Case 17

Aksi Actor	Reaksi Sistem
------------	---------------

<i>Skenario Normal</i>	
1. Admin/Superadmin membuka konfigurasi lookup variabel komponen gaji di “Master Komponen Gaji”	
	2. Menampilkan daftar lookup variabel dan form tambah lookup variabel
3. Menambah, mengubah, atau menghapus lookup variabel	
	4. Menyimpan perubahan lookup ke database
	5. Memperbarui daftar lookup variable
	6. Menampilkan notifikasi berhasil



Gambar 4.19 Sequence Diagram UC-17 Kelola Lookup Variabel Komponen Gaji

Gambar 4.19 menampilkan interaksi antar objek pada skenario UC-17. Objek yang berinteraksi merupakan instansiasi kelas pada diagram kelas (Gambar 4.3). Fragmen alt digunakan untuk memisahkan skenario alternatif dari skenario normal.

4.3.2.18 Skenario Use Case 18

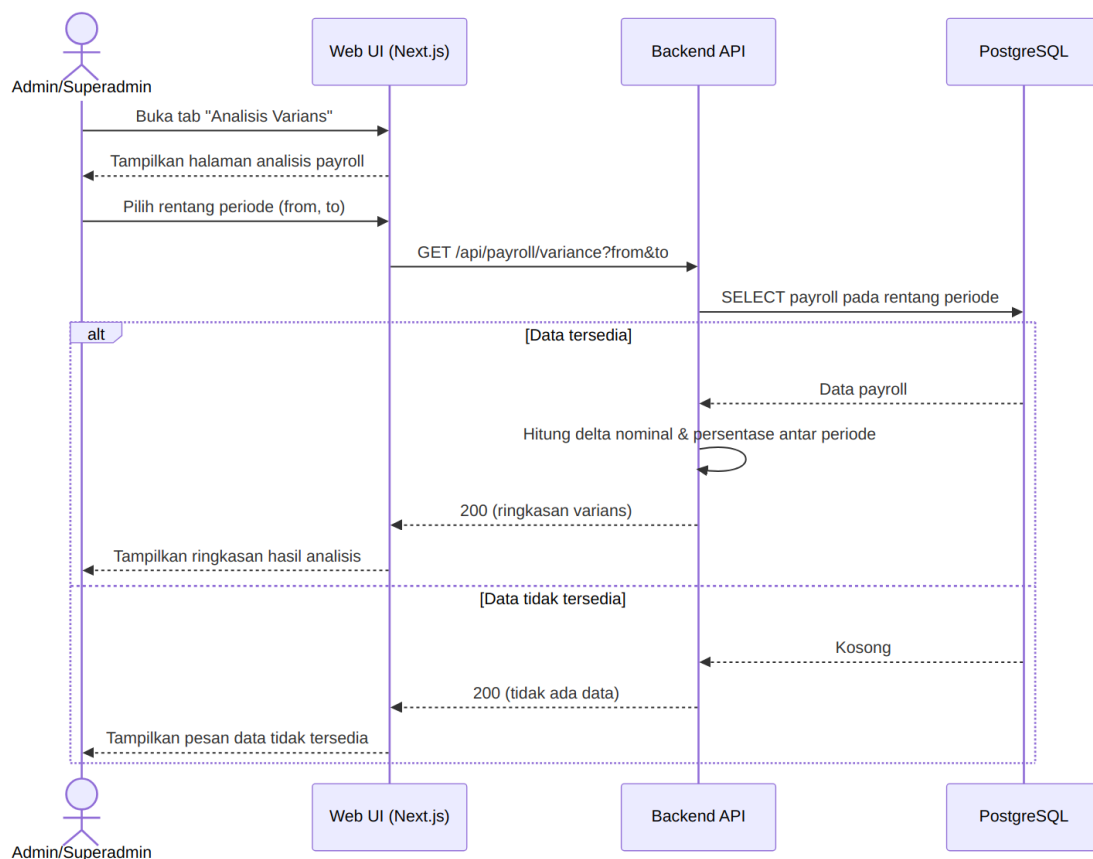
Nama Use Case: Analisis Varians Penggajian

Skenario:

Tabel 4.3.2.18 Tabel Skenario Use Case 18

<i>Aksi Actor</i>	<i>Reaksi Sistem</i>
<i>Skenario Normal</i>	
1. Membuka tab Analisis Varians	

	2. Menampilkan halaman analisis payroll
3. Memilih rentang periode	
	4. Mengambil data payroll pada rentang periode tersebut
	9. Menampilkan ringkasan hasil analisis
<i>Skenario Alternatif (Data tidak tersedia)</i>	
1. Memilih rentang periode yang tidak memiliki data payroll	
	2. Menampilkan pesan bahwa data tidak tersedia



Gambar 4.20 Sequence Diagram UC-18 Analisis Varians Penggajian

Gambar 4.20 menampilkan interaksi antar objek pada skenario UC-18. Objek yang berinteraksi merupakan instansiasi kelas pada diagram kelas (Gambar 4.3). Fragmen alt digunakan untuk memisahkan skenario alternatif dari skenario normal.

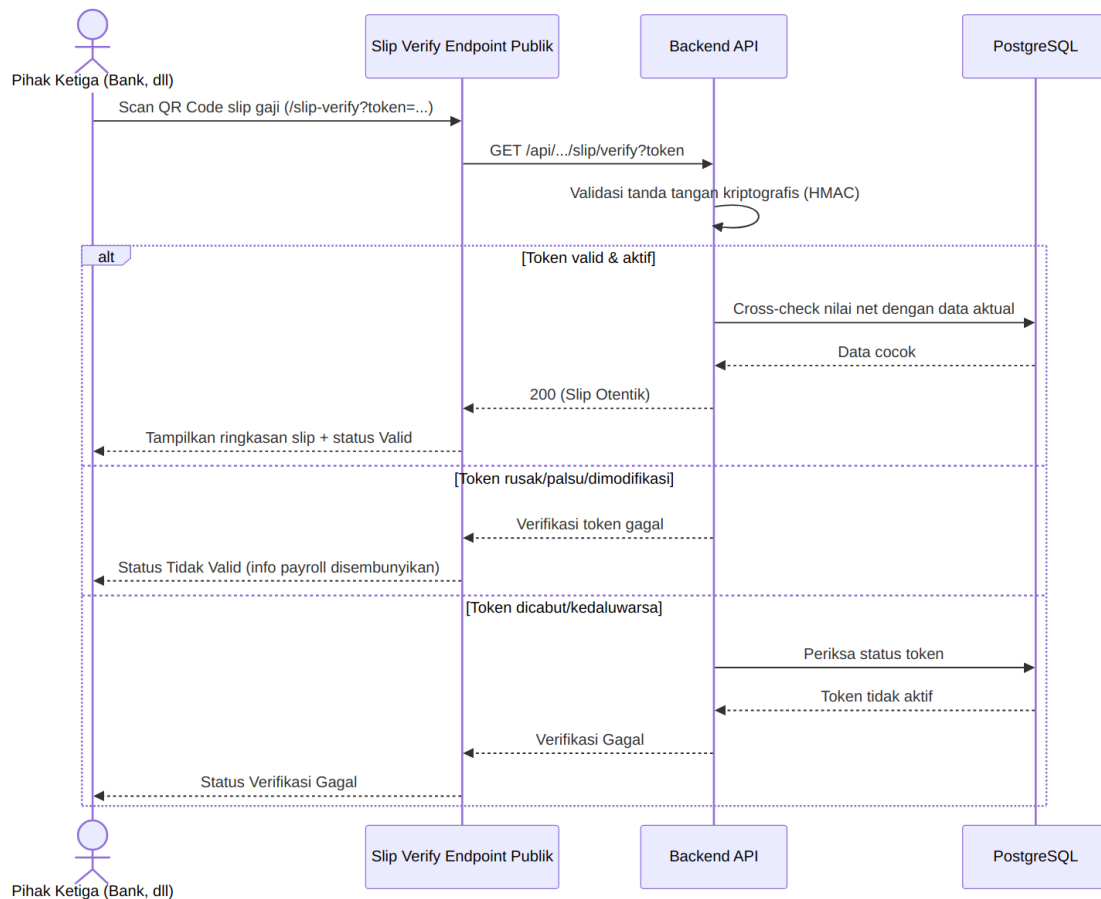
4.3.2.19 Skenario Use Case 19

Nama Use Case: Verifikasi Slip Gaji melalui QR Code

Skenario:

Tabel 4.3.2.16 Tabel Skenario Use Case 19

<i>Aksi Actor</i>	<i>Reaksi Sistem</i>
<i>Skenario Normal</i>	
1. Pihak ketiga (Bank, lembaga pembiayaan, atau instansi lain) melakukan scan QR Code pada slip gaji	
	2. Mengakses endpoint verifikasi publik dan melakukan validasi token
	3. Menampilkan ringkasan slip gaji dan status valid
<i>Skenario Alternatif (QR tidak valid)</i>	
1. Melakukan scan QR Code yang rusak, palsu, atau telah dimodifikasi	
	2. Sistem gagal melakukan verifikasi token
	3. Menampilkan status Tidak Valid dan informasi payroll tidak ditampilkan
<i>Skenario Alternatif (Slip sudah tidak berlaku)</i>	
1. Melakukan scan QR pada slip yang token verifikasinya telah dicabut atau kedaluwarsa	
	2. Sistem mendeteksi token tidak lagi aktif
	3. Menampilkan status Verifikasi Gagal



Gambar 4.21 Sequence Diagram UC-18 Verifikasi Slip Gaji melalui QR Code

Gambar 4.21 menampilkan interaksi antar objek pada skenario UC-18. Objek yang berinteraksi merupakan instansiasi kelas pada diagram kelas (Gambar 4.3). Fragmen alt digunakan untuk memisahkan skenario alternatif dari skenario normal.

4.3.3 Deployment View

4.3.3.1 Ringkasan

Aplikasi Penggajian ini dideploy menggunakan pendekatan kontainerisasi dengan memanfaatkan Docker. Seluruh komponen utama aplikasi dibagi menjadi dua kontainer terpisah, yaitu kontainer frontend (berbasis Next.js) dan kontainer backend (berbasis Express.js). Kedua kontainer tersebut berjalan secara efisien di dalam satu mesin server virtual yang sama (Virtual Private Server / VPS).

Untuk manajemen web server publik dan keamanan, Apache (bawaan dari panel pengelolaan cPanel) dikonfigurasi sebagai reverse proxy sekaligus sebagai terminator TLS/SSL. Dengan konfigurasi ini, seluruh permintaan (request) publik yang menuju ke path spesifik /pay2 akan dienkripsi menggunakan SSL pada port 443, untuk kemudian diteruskan oleh Apache ke kontainer internal yang berjalan di lingkungan localhost.

Di sisi penyimpanan data, infrastruktur basis data tidak di-host secara mandiri (self-hosted) di dalam VPS guna menjaga performa dan keandalan. Sebagai gantinya, aplikasi menggunakan layanan basis data cloud terkelola dari Supabase (PostgreSQL terkelola). Melalui arsitektur ini, secara fisik sistem hanya mengandalkan 1 mesin aplikasi utama (VPS) dan 1 layanan basis data berbasis cloud (Supabase) yang diakses langsung oleh perangkat klien melalui web browser.

4.3.3.2 Kebutuhan Perangkat Keras dan Perangkat Lunak

Berikut adalah rincian kebutuhan spesifikasi teknis minimum yang harus dipenuhi pada setiap komponen lingkungan deployment:

A. Mesin Server Host (VPS) 1 Unit

Tabel 4.3.3.2.1 Kebutuhan HW/SW

Kategori	Kebutuhan & Spesifikasi Teknis
Hardware Minimum	±2 vCPU, 2–4 GB RAM, ~20 GB Disk Space. *Kapasitas RAM yang cukup sangat krusial saat melakukan proses penyusunan (build) image kontainer.
Sistem Operasi (OS)	Linux distribusi Debian atau Ubuntu yang telah dilengkapi dengan panel manajemen cPanel/WHM.
Web Server Host	Apache dengan modul aktif: mod_proxy, mod_proxy_http, mod_rewrite, dan mod_headers.
Container Runtime	Docker Engine beserta dengan Docker Compose v2.
Keamanan / TLS	Sertifikat SSL valid untuk domain bukujurnal.com (dikelola melalui AutoSSL atau Let's Encrypt via cPanel).
Utilitas Lain-lain	Git (digunakan untuk aktivitas deployment dan penarikan/pull kode sumber dari repositori).

B. Layanan Cloud Pihak Ketiga (Third-Party Cloud Services) 1 Unit

Tabel 4.3.3.2.2 Layanan Cloud Pihak ketiga

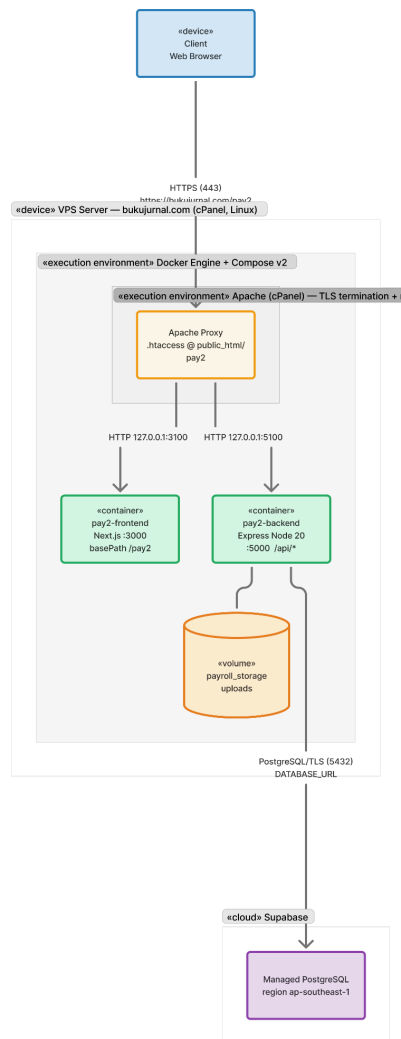
Nama Layanan	Fungsi & Peran Infrastruktur
Supabase (Managed PostgreSQL)	Bertindak sebagai basis data utama aplikasi dengan penempatan wilayah (region) di ap-southeast-1 (Singapura). Layanan ini menyediakan fitur pencadangan otomatis (backup) serta jaminan ketersediaan tinggi (high availability) secara terkelola.

C. Lingkungan Sisi Klien (Client-Side)

Tabel 4.3.3.2.3 Lingkungan Client-Side

Kebutuhan Komponen	Keterangan Penggunaan
Web Browser Modern	Pengguna akhir hanya memerlukan peramban web modern (seperti Google Chrome, Mozilla Firefox, Edge, atau Safari) untuk mengakses alamat publik https://bukujurnal.com/pay2 tanpa perlu melakukan instalasi aplikasi tambahan apa pun.

4.3.3.3 Diagram Deployment (Arsitektur Fisik)



Gambar 4.3.3.3. Deployment Diagram

4.3.3.4 Alur Kerja Operasional dan Komunikasi Sistem

1. Inisiasi Permintaan Klien: Pengguna mengakses alamat URL publik aplikasi di <https://bukujurnal.com/pay2> melalui web browser. Permintaan ini pertama kali diterima oleh web server Apache di VPS host pada port standar HTTPS 443 dengan jaminan enkripsi aman (SSL/TLS).
2. Mekanisme Reverse Proxy: Berdasarkan konfigurasi berkas penataan `.htaccess` yang terletak di direktori `public_html/pay2`, Apache bertindak sebagai jembatan pembagi lalu lintas:
 - Setiap permintaan yang mengarah ke path API atau endpoint backend (`/pay2/api/*`) akan diteruskan ke kontainer backend (port internal `:5100`).
 - Seluruh permintaan aset statis, routing halaman, dan antarmuka lainnya akan dialihkan menuju kontainer frontend (port internal `:3100`).
 - Permintaan akses langsung ke root path `/pay2` secara otomatis dialihkan (redirect) menuju halaman utama sistem di `/pay2/dashboard`. Apabila status otentikasi sesi login pengguna ditemukan belum valid, maka sistem secara otomatis mengarahkan rute menuju halaman login di `/pay2/login`.
3. Komunikasi Frontend ke API: Komponen frontend berinteraksi secara dinamis dengan memanggil URL API publik (<https://bukujurnal.com/pay2/api>), yang proses peruteannya kembali melewati lapisan keamanan Apache sebelum mencapai mesin backend utama.

4. Akses Basis Data & Penyimpanan Berkas: Komponen backend menangani seluruh logika bisnis aplikasi. Untuk operasi manipulasi data (CRUD), backend berkomunikasi dengan layanan cloud Supabase melalui koneksi aman terenkripsi (port 5432) menggunakan parameter DATABASE_URL. Sementara itu, untuk seluruh berkas fisik/dokumen digital terkait kepegawaian yang diunggah oleh pengguna akan disimpan secara aman di dalam media penyimpanan lokal persisten berupa Docker volume bernama payroll_storage.

5 Implementasi Perangkat Lunak

Sistem penggajian diimplementasikan dalam dua aplikasi terpisah yang berada dalam satu repositori: aplikasi backend pada direktori backend/ dan aplikasi frontend pada direktori frontend/. Backend dibangun menggunakan Node.js v20 + Express.js + TypeScript dan dikompilasi ke direktori dist/ untuk keperluan produksi. Frontend dibangun menggunakan Next.js v16 + React v19 + TypeScript dan dikompilasi ke direktori .next/. Pengelolaan skema basis data dilakukan melalui skrip migrasi SQL pada backend/src/config/ yang dijalankan menggunakan perintah npm run db:migrate.

5.1 Struktur Direktori Implementasi

```
aplikasi-penggajian
├── backend/
│   └── src/
│       ├── test_/
│       │   ├── integration/
│       │   │   ├── activityLog.integration.test.ts
│       │   │   ├── auth.integration.test.ts
│       │   │   ├── ddl.integration.test.ts
│       │   │   ├── employee.integration.test.ts
│       │   │   ├── health.integration.test.ts
│       │   │   ├── payroll.integration.test.ts
│       │   │   ├── payrollRecap.integration.test.ts
│       │   │   ├── schema.integration.test.ts
│       │   │   └── schema.integration.test.ts
│       │   ├── activityLog.test.ts
│       │   ├── auth.test.ts
│       │   ├── company.test.ts
│       │   ├── ddl.test.ts
│       │   ├── dml.test.ts
│       │   ├── employee.test.ts
│       │   ├── employeePayroll.test.ts
│       │   ├── formulaResolver.test.ts
│       │   ├── health.test.ts
│       │   ├── integrationApi.test.ts
│       │   ├── middleware.test.ts
│       │   ├── payrollDashboardRecap.test.ts
│       │   ├── payrollRecapValidation.test.ts
│       │   ├── permission.test.ts
│       │   ├── salaryComponent.test.ts
│       │   ├── schema.test.ts
│       │   ├── sprint4.test.ts
│       │   ├── sprint4_complete.test.ts
│       │   └── user.test.ts
│       └── config/
│           ├── scripts/
│           │   ├── migrate.ts
│           │   └── rollback.ts
│           ├── 001_init.sql
│           ├── 002_add_can_switch_company.sql
│           └── 003_activity_audit.sql
```


- 004_activity_log_refactor.sql
- 005_add_grade_and_offset.sql
- 005_salary_components.sql
- 006_employee_documents.sql
- 006_employee_grade_multitenancy.sql
- 007_salary_components_and_payroll.sql
- 008_payroll_gap_fixes.sql
- 009_attendance.sql
- 009_integration_api_keys.sql
- 010_show_on_payslip.sql
- 011_component_lookup_rules.sql
- 012_salary_templates.sql
- db.ts
- seed.ts
- swagger.ts
- controllers/
 - activityController.ts
 - activityLogController.ts
 - attendanceController.ts
 - authController.ts
 - companyController.ts
 - dbManagementController.ts
 - departmentController.ts
 - employeeController.ts
 - employeeDocumentController.ts
 - employeePayrollController.ts
 - gradeAllowanceController.ts
 - integrationController.ts
 - jabatanController.ts
 - payrollController.ts
 - permissionController.ts
 - referenceController.ts
 - salaryComponentController.ts
 - salaryTemplateController.ts
 - schemaController.ts
 - userController.ts
- lib/
 - apiError.ts
 - context.ts
 - db.ts
 - formulaResolver.ts
- middleware/
 - activityLogger.ts
 - apiKeyMiddleware.ts
 - authMiddleware.ts
 - permissionMiddleware.ts
- middlewares/
 - errorHandler.ts
- models/
 - activityLogModel.ts
 - activityModel.ts
 - attendanceModel.ts
 - companyModel.ts
 - componentTableRowModel.ts
 - dbManagementModel.ts
 - departmentModel.ts
 - employeeDocumentModel.ts
 - employeeModel.ts
 - employeeOverrideModel.ts



		└─ page.tsx
		└─ companies
		└─ AssignUserModal.tsx
		└─ CompanyFormModal.tsx
		└─ page.tsx
		└─ employees
		└─ page.tsx
		└─ grades
		└─ page.tsx
		└─ integration
		└─ page.tsx
		└─ jabatan-department
		└─ page.tsx
		└─ payroll-recap
		└─ page.tsx
		└─ _VarianceTab.tsx
		└─ payroll-run
		└─ page.tsx
		└─ salary-components
		└─ page.tsx
		└─ _ReferenceTab.tsx
		└─ _SandboxTab.tsx
		└─ salary-slip
		└─ page.tsx
		└─ users
		└─ page.tsx
		└─ login/
		└─ page.tsx
		└─ select-company/
		└─ page.tsx
		└─ slip-verify/
		└─ page.tsx
		└─ layout.tsx
		└─ page.tsx
	components/	└─ ActionBadge.tsx
		└─ CommaSeparatedField.tsx
		└─ ConfirmationModal.tsx
		└─ DDLFormInput.tsx
		└─ DocumentPreviewModal.tsx
		└─ EmployeeComponentsTab.tsx
		└─ EmployeeDocumentUpload.tsx
		└─ LoadingSpinner.tsx
		└─ LogTable.tsx
		└─ PageHeader.tsx
		└─ Toast.tsx
		└─ VisualFormulaEditor.tsx
	contexts/	└─ CompanyContext.tsx
		└─ ToastContext.tsx
	hooks/	└─ useActivityLog.ts
		└─ useCompany.ts
		└─ useDbManagement.ts
		└─ useEmployeeDocument.ts
		└─ usePermission.ts
	lib/	└─ auth.ts
		└─ employees.ts

	- grades.ts - integration.ts - jabatanDepartment.ts - payroll.ts - payrollRecap.ts - references.ts - salaryComponents.ts - salaryTemplates.ts - schema.ts
	- types/ - index.ts
	- utils/ - api.ts - format.ts
	middleware.ts

5.2 Traceability Table

Keterhubungan setiap berkas kode program dengan subsistem atau komponen perancangan ditampilkan pada Tabel 5.1. Tabel ini menelusuri elemen model perancangan (Tabel 4.1) terhadap berkas hasil implementasi. Judul kolom diulang pada setiap halaman ketika tabel berpindah halaman.

Tabel 5.1 Traceability Table Berkas Implementasi terhadap Komponen Perancangan

No	File Implementasi	Komponen (ID)
1	backend/src/index.ts	SS-02
2	backend/src/app.ts	SS-02
3	backend/src/config/db.ts	SS-03
4	backend/src/config/seed.ts	SS-03
5	backend/src/config/swagger.ts	SS-02
6	backend/src/config/scripts/migrate.ts	SS-03
7	backend/src/config/scripts/rollback.ts	SS-03
8	backend/src/config/001_init.sql	SS-03
9	backend/src/config/002_add_can_switch_company.sql	SS-03
10	backend/src/config/003_activity_audit.sql	SS-03, C-15
11	backend/src/config/004_activity_log_refactor.sql	SS-03, C-15
12	backend/src/config/005_add_grade_and_offset.sql	SS-03, C-07
13	backend/src/config/005_salary_components.sql	SS-03, C-08
14	backend/src/config/006_employee_documents.sql	SS-03, C-17
15	backend/src/config/006_employee_grade_multitenancy.sql	SS-03, C-07
16	backend/src/config/007_salary_components_and_payroll.sql	SS-03, C-08, C-10
17	backend/src/config/008_payroll_gap_fixes.sql	SS-03, C-05, C-10

18	backend/src/config/009_attendance.sql	SS-03, C-13
19	backend/src/config/009_integration_api_keys.sql	SS-03, C-18
20	backend/src/config/009_reference_tables.sql	SS-03, C-14
21	backend/src/lib/db.ts	SS-03
22	backend/src/lib/context.ts	SS-02
23	backend/src/lib/apiError.ts	SS-02
24	backend/src/lib/formulaResolver.ts	C-09
25	backend/src/middleware/authMiddleware.ts	C-01
26	backend/src/middleware/permissionMiddleware.ts	C-02
27	backend/src/middleware/apiKeyMiddleware.ts	C-18
28	backend/src/middleware/activityLogger.ts	C-15
29	backend/src/middlewares/errorHandler.ts	SS-02
30	backend/src/controllers/authController.ts	C-01
31	backend/src/controllers/permissionController.ts	C-02
32	backend/src/controllers/companyController.ts	C-03
33	backend/src/controllers/userController.ts	C-04
34	backend/src/controllers/employeeController.ts	C-05
35	backend/src/controllers/employeePayrollController.ts	C-05
36	backend/src/controllers/departmentController.ts	C-06
37	backend/src/controllers/jabatanController.ts	C-06
38	backend/src/controllers/gradeAllowanceController.ts	C-07
39	backend/src/controllers/salaryComponentController.ts	C-08
40	backend/src/controllers/payrollController.ts	C-10, C-11, C-12
41	backend/src/controllers/attendanceController.ts	C-13
42	backend/src/controllers/referenceController.ts	C-14
43	backend/src/controllers/activityController.ts	C-15
44	backend/src/controllers/activityLogController.ts	C-15
45	backend/src/controllers/dbManagementController.ts	C-16
46	backend/src/controllers/schemaController.ts	C-16
47	backend/src/controllers/employeeDocumentController.ts	C-17
48	backend/src/controllers/integrationController.ts	C-18

49	backend/src/models/companyModel.ts	C-03
50	backend/src/models/userModel.ts	C-04
51	backend/src/models/employeeModel.ts	C-05
52	backend/src/models/employeePayrollModel.ts	C-05
53	backend/src/models/employeeOverrideModel.ts	C-05, C-08
54	backend/src/models/departmentModel.ts	C-06
55	backend/src/models/jabatanModel.ts	C-06
56	backend/src/models/gradeAllowanceModel.ts	C-07
57	backend/src/models/salaryComponentModel.ts	C-08
58	backend/src/models/componentTableRowModel.ts	C-08
59	backend/src/models/salaryPreviewModel.ts	C-08, C-09
60	backend/src/models/payrollModel.ts	C-10
61	backend/src/models/employeePayrollModel.ts	C-10
62	backend/src/models/payrollValidationModel.ts	C-11
63	backend/src/models/attendanceModel.ts	C-13
64	backend/src/models/referenceModel.ts	C-14
65	backend/src/models/activityModel.ts	C-15
66	backend/src/models/activityLogModel.ts	C-15
67	backend/src/models/dbManagementModel.ts	C-16
68	backend/src/models/schemaModel.ts	C-16
69	backend/src/models/employeeDocumentModel.ts	C-17
70	backend/src/models/integrationKeyModel.ts	C-18
71	backend/src/models/permissionModel.ts	C-02
72	backend/src/routes/authRoutes.ts	C-01
73	backend/src/routes/permmisionRoutes.ts	C-02
74	backend/src/routes/companyRoutes.ts	C-03
75	backend/src/routes/userRoutes.ts	C-04
76	backend/src/routes/employeeRoutes.ts	C-05
77	backend/src/routes/departmentRoutes.ts	C-06
78	backend/src/routes/jabatanRoutes.ts	C-06
79	backend/src/routes/employeeGradeRoutes.ts	C-07

80	backend/src/routes/salaryComponentRoutes.ts	C-08
81	backend/src/routes/payrollRoutes.ts	C-10, C-11, C-12
82	backend/src/routes/attendanceRoutes.ts	C-13
83	backend/src/routes/referenceRoutes.ts	C-14
84	backend/src/routes/activityRoutes.ts	C-15
85	backend/src/routes/activityLogRoutes.ts	C-15
86	backend/src/routes/dbManagementRoutes.ts	C-16
87	backend/src/routes/schemaRoutes.ts	C-16
88	backend/src/routes/employeeDocumentRoutes.ts	C-17
89	backend/src/routes/integrationRoutes.ts	C-18
90	backend/src/utills/formulaInterpreter.ts	C-09
91	backend/src/utills/slipToken.ts	C-12
92	backend/src/utills/qrcode.ts	C-12
93	backend/src/utills/documentStorage.ts	C-17
94	backend/src/types/index.ts	SS-02
95	backend/src/types/admin.ts	SS-02
96	backend/src/___test___/auth.test.ts	C-01
97	backend/src/___test___/middleware.test.ts	C-01, C-02
98	backend/src/___test___/permission.test.ts	C-02
99	backend/src/___test___/company.test.ts	C-03
100	backend/src/___test___/employee.test.ts	C-05
101	backend/src/___test___/employeePayroll.test.ts	C-05, C-10
102	backend/src/___test___/salaryComponent.test.ts	C-08
103	backend/src/___test___/formulaResolver.test.ts	C-09
104	backend/src/___test___/offsetSalaryPreview.test.ts	C-09, C-10
105	backend/src/___test___/offsetSalaryBatch.test.ts	C-09, C-10
106	backend/src/___test___/payrollDashboardRecap.test.ts	C-10
107	backend/src/___test___/payrollRecapValidation.test.ts	C-11
108	backend/src/___test___/tenureCalculation.test.ts	C-07, C-09
109	backend/src/___test___/activityLog.test.ts	C-15
110	backend/src/___test___/ddl.test.ts	C-16

111	backend/src/ __test__ /dml.test.ts	C-16
112	backend/src/ __test__ /schema.test.ts	C-16
113	backend/src/ __test__ /integrationApi.test.ts	C-18
114	backend/src/ __test__ /user.test.ts	C-04
115	backend/src/ __test__ /health.test.ts	SS-02
116	backend/src/ __test__ /integration/auth.integration.test.ts	C-01
117	backend/src/ __test__ /integration/employee.integration.test.ts	C-05
118	backend/src/ __test__ /integration/payroll.integration.test.ts	C-10, C-11
119	backend/src/ __test__ /integration/payrollRecap.integration.test.ts	C-10
120	backend/src/ __test__ /integration/offsetSalary.integration.test.ts	C-09, C-10
121	backend/src/ __test__ /integration/activityLog.integration.test.ts	C-15
122	backend/src/ __test__ /integration/ddl.integration.test.ts	C-16
123	backend/src/ __test__ /integration/schema.integration.test.ts	C-16
124	backend/src/ __test__ /integration/switchCompany.integration.test.ts	C-01, C-03
125	backend/src/ __test__ /integration/health.integration.test.ts	SS-02
126	frontend/middleware.ts	C-19
127	frontend/app/layout.tsx	SS-01
128	frontend/app/page.tsx	SS-01
129	frontend/app/login/page.tsx	C-01
130	frontend/app/select-company/page.tsx	C-01, C-03
131	frontend/app/slip-verify/page.tsx	C-12
132	frontend/app/dashboard/layout.tsx	SS-01, C-19
133	frontend/app/dashboard/page.tsx	C-10
134	frontend/app/dashboard/employees/page.tsx	C-05
135	frontend/app/dashboard/companies/page.tsx	C-03
136	frontend/app/dashboard/companies/CompanyFormModal.tsx	C-03
137	frontend/app/dashboard/companies/AssignUserModal.tsx	C-03, C-04
138	frontend/app/dashboard/users/page.tsx	C-04
139	frontend/app/dashboard/jabatan-department/page.tsx	C-06
140	frontend/app/dashboard/salary-components/page.tsx	C-08
141	frontend/app/dashboard/formula-editor/page.tsx	C-09

142	frontend/app/dashboard/payroll-run/page.tsx	C-10
143	frontend/app/dashboard/payroll-recap/page.tsx	C-10, C-11
144	frontend/app/dashboard/payroll-variance/page.tsx	C-10
145	frontend/app/dashboard/salary-slip/page.tsx	C-12
146	frontend/app/dashboard/my-salary-slip/page.tsx	C-12
147	frontend/app/dashboard/attendance/page.tsx	C-13
148	frontend/app/dashboard/reference-data/page.tsx	C-14
149	frontend/app/dashboard/activity-log/page.tsx	C-15
150	frontend/app/dashboard/activity-log/components/ActivityTable.tsx	C-15
151	frontend/app/dashboard/activity-log/components/ExportButton.tsx	C-15
152	frontend/app/dashboard/activity-log/components/FilterPanel.tsx	C-15
153	frontend/app/dashboard/activity-log/components/Pagination.tsx	SS-01
154	frontend/app/dashboard/integration/page.tsx	C-18
155	frontend/components/VisualFormulaEditor.tsx	C-09
156	frontend/components/EmployeeDocumentUpload.tsx	C-17
157	frontend/components/DocumentPreviewModal.tsx	C-17
158	frontend/components/ConfirmationModal.tsx	SS-01
159	frontend/components/DDLFormInput.tsx	C-16
160	frontend/components/LogTable.tsx	C-15
161	frontend/components/ActionBadge.tsx	SS-01
162	frontend/components/CommaSeparatedField.tsx	SS-01
163	frontend/components/LoadingSpinner.tsx	SS-01
164	frontend/components/PageHeader.tsx	SS-01
165	frontend/components/Toast.tsx	SS-01
166	frontend/contexts/CompanyContext.tsx	C-03
167	frontend/contexts/ToastContext.tsx	SS-01
168	frontend/hooks/useActivityLog.ts	C-15
169	frontend/hooks/useCompany.ts	C-03
170	frontend/hooks/useDbManagement.ts	C-16
171	frontend/hooks/useEmployeeDocument.ts	C-17
172	frontend/hooks/usePermission.ts	C-02

173	frontend/lib/auth.ts	C-01, C-20
174	frontend/lib/employees.ts	C-05, C-20
175	frontend/lib/jabatanDepartment.ts	C-06, C-20
176	frontend/lib/salaryComponents.ts	C-08, C-20
177	frontend/lib/payroll.ts	C-10, C-20
178	frontend/lib/payrollRecap.ts	C-10, C-20
179	frontend/lib/references.ts	C-14, C-20
180	frontend/lib/schema.ts	C-16, C-20
181	frontend/lib/integration.ts	C-18, C-20
182	frontend/utils/api.ts	C-20
183	frontend/utils/format.ts	SS-01
184	frontend/types/index.ts	SS-01

6 Pengujian Perangkat Lunak

Bagian ini menjelaskan tujuan, rencana, kasus uji, pengujian per sprint, dan hasil pengujian Aplikasi Penggajian Terintegrasi. Seluruh pengujian yang dilaporkan bersifat otomatis (*automated testing*) menggunakan *test runner* Jest dan dapat dijalankan ulang melalui perintah `npm run test`, `npm run test:integration`, dan `npm run test:coverage`. Bagian ini bersifat *living document* yang dilengkapi secara bertahap pada setiap sprint.

6.1 Tujuan Pengujian

Pengujian dilaksanakan untuk mengevaluasi apakah aplikasi telah berfungsi sesuai kebutuhan, aman digunakan, dan cukup andal sebelum setiap rilis sprint. Secara spesifik, tujuan pengujian adalah sebagai berikut:

- **Verifikasi fungsionalitas**, yaitu memastikan setiap *endpoint* API dan logika bisnis, seperti autentikasi, manajemen perusahaan, manajemen karyawan, perhitungan gaji, rekap penggajian, dan *activity log*, berperilaku sesuai spesifikasi Dokumen Scrum dan Catatan Penggajian.
- **Menjamin keamanan akses**, yaitu memvalidasi autentikasi berbasis JWT dan otorisasi berbasis peran (RBAC: SUPERADMIN, ADMIN, USER), termasuk isolasi data antarperusahaan pada sistem yang bersifat *multi-tenant*.
- **Menjaga integritas data penggajian**, yaitu memastikan perhitungan komponen gaji dinamis, validasi nilai *gross*, *deduction*, dan *net*, serta hasil rekap penggajian tetap konsisten dan tahan terhadap manipulasi data.
- **Mencegah regresi**, yaitu menyediakan jaring pengaman otomatis agar perubahan pada sprint berikutnya tidak merusak fungsi yang sudah berjalan.
- **Memvalidasi kontrak API dan operasi basis data dinamis**, yaitu memastikan *status code*, *header*, format respons, format *error*, serta operasi DDL/DML dinamis berjalan sesuai aturan dan aman terhadap input yang tidak valid.

6.2 Rencana Pengujian

6.2.1 Jenis Pengujian

Aplikasi diuji dengan pendekatan piramida pengujian yang terdiri atas dua lapis utama. Seluruh pengujian dilakukan secara otomatis.

Tabel 6.1 Jenis Pengujian

Jenis	Cakupan	Basis Data	Berkas	Kasus Uji
Unit & API Test	Menguji <i>controller</i> , <i>middleware</i> , dan fungsi murni secara terisolasi. Dependensi seperti basis data dan JWT di- <i>mock</i> . Pengujian level HTTP dilakukan menggunakan Supertest tanpa menyalakan server nyata.	Di- <i>mock</i>	19	232
Integration Test	Menguji alur <i>end-to-end</i> dari HTTP request, <i>route</i> , <i>controller</i> , <i>model</i> , hingga basis data PostgreSQL nyata. Pengujian ini memvalidasi <i>query</i> , <i>constraint</i> , dan isolasi data <i>multi-tenant</i> .	PostgreSQL nyata	9	119
			Total	351

Selain dua jenis pengujian utama tersebut, terdapat beberapa kategori pengujian khusus yang tercakup di dalamnya, yaitu pengujian keamanan, pengujian kontrak API, dan pengujian logika bisnis. Pengujian keamanan berfokus pada validasi *middleware*, autentikasi JWT, otorisasi peran, dan hak akses pengguna. Pengujian kontrak API memverifikasi *status code*, *header*, dan bentuk respons. Pengujian logika bisnis memvalidasi *formula resolver*, rekap penggajian, perhitungan varians, token slip, serta QR Code.

6.2.2 Perangkat dan Lingkungan Pengujian

Perangkat dan lingkungan pengujian ditampilkan pada Tabel 6.2.

Tabel 6.2 Perangkat dan Lingkungan Pengujian

Komponen	Teknologi
<i>Test Runner</i> / kompilasi	Jest v30 + ts-jest v29
Pengujian HTTP API	Supertest v7
Bahasa / Runtime	TypeScript v5.6 / Node.js v20
Basis data integrasi	PostgreSQL

Konfigurasi pengujian dipisahkan menjadi dua. Pengujian unit menggunakan `jest.config.js` dan `jest.setup.js` untuk menyetel variabel lingkungan tiruan sehingga dapat berjalan tanpa basis data nyata. Pengujian integrasi menggunakan `jest.integration.config.js` dan `jest.integration.setup.js` untuk memuat konfigurasi dari `.env`, serta dijalankan secara serial karena berbagi *state* basis data.

6.2.3 Perintah Pengujian

Perintah pengujian ditampilkan pada Tabel 6.3.

Tabel 6.3 Perintah Pengujian

Perintah	Fungsi
<code>npm run test</code>	Menjalankan 232 kasus uji unit tanpa basis data nyata.
<code>npm run test:integration</code>	Menjalankan 119 kasus uji integrasi terhadap basis data PostgreSQL yang dikonfigurasi pada <code>.env</code> .
<code>npm run test:coverage</code>	Menjalankan pengujian unit dan menghasilkan laporan cakupan kode.

6.2.4 Strategi Pengujian

Pengujian unit dilakukan dengan mengisolasi dependensi menggunakan `jest.mock(...)`. Setiap kasus uji dibersihkan menggunakan `jest.clearAllMocks()` agar hasil pengujian tidak saling memengaruhi. Pengujian integrasi dilakukan dengan menyiapkan data pada basis data nyata, menjalankan alur sistem secara lengkap, lalu membersihkan data setelah pengujian selesai. Strategi ini memungkinkan tim memvalidasi logika aplikasi sekaligus memastikan bahwa *query*, *constraint*, dan relasi data berjalan sesuai harapan.

Penamaan kasus uji dibuat deskriptif menggunakan struktur `describe(...)` dan `it(...)` atau `test(...)`. Sebagian kasus uji juga dipetakan ke ID tugas *backlog* untuk menjaga keterlacakan antara kebutuhan, implementasi, dan pengujian.

6.2.5 Integrasi Berkelanjutan (CI/CD)

Pengujian otomatis dirancang agar dapat diintegrasikan ke *pipeline* GitLab CI. *Pipeline* saat ini mencakup tahap instalasi dependensi, pengecekan kode, dan proses *build*. Unit test direkomendasikan untuk dijalankan sebagai gerbang mutu sebelum *build*, sedangkan integration test dapat dijalankan sebagai *job* terpisah dengan konfigurasi basis data PostgreSQL. Dengan demikian, pengujian otomatis dapat membantu mendeteksi kesalahan lebih awal sebelum kode digabungkan ke cabang utama.

6.3 Kasus Uji

Kasus uji disusun dengan format yang mencakup identitas (ID), kondisi awal (prasyarat), langkah pengujian beserta masukan konkret, keluaran yang diharapkan, dan hasil aktual (Sesuai atau Tidak Sesuai). Setiap kasus uji dapat dieksekusi ulang dengan hasil yang sama. Kasus uji fungsional ditampilkan pada Tabel 6.4 dan kasus uji nonfungsional pada Tabel 6.5.

Tabel 6.4 Kasus Uji Fungsional

Modul	Berkas Uji	Unit	Integrasi	Fokus Skenario Uji
Autentikasi & Sesi	<code>auth</code> , <code>auth.integration</code> , <code>switchCompany.integration</code>	17	22	Login, validasi kredensial, token JWT, pemilihan perusahaan, pergantian perusahaan, GET <code>/me</code> , dan alur <i>end-to-end</i> .

Otorisasi (RBAC) & Middleware	middleware, permission	12	-	Validasi autentikasi, otorisasi, <i>bypass</i> SUPERADMIN, penolakan pengguna nonaktif, akses perusahaan dicabut, dan pengelolaan izin.
Manajemen Perusahaan	company	26	-	CRUD perusahaan, <i>soft delete</i> , penetapan pengguna ke perusahaan, batasan peran, dan isolasi <i>multi-tenant</i> .
Manajemen Pengguna	user	6	-	CRUD pengguna, proteksi eskalasi peran ke SUPERADMIN, dan penanganan data tidak ditemukan.
Skema Dinamis (DDL/DML)	ddl, dml, schema, ddl.integration, schema.integration	58	38	CREATE/ALTER/DROP TABLE, INSERT/UPDATE/DELETE baris, konfirmasi ganda, validasi nama dan tipe kolom, <i>keyword</i> terlarang, serta audit data lama dan baru.
Activity Log	activityLog, activityLog.integration	8	8	Pencatatan jejak aktivitas, paginasi, filter, pembatasan akses berdasarkan peran, dan ekspor data.
Mesin Formula Gaji	formulaResolver, sprint4	20	-	Validasi karakter rumus, ekstraksi referensi, deteksi dependensi melingkar, <i>topological sort</i> , dan evaluasi rumus berantai.
Karyawan & Komponen Gaji	employee, employeePayroll, salaryComponent, employee.integration, payroll.integration	21	26	CRUD karyawan, <i>soft delete</i> , <i>preview</i> gaji, komponen FIXED/TABLE, rumus melingkar, dan konflik email.
Rekap Penggajian	payrollDashboardRecap, payrollRecapValidation, payrollRecap.integration	42	24	Statistik dashboard, rekap summary/departemen/karyawan, ekspor CSV, dan validasi integritas perhitungan penggajian.

Integrasi HRD & Slip	integrationApi, sprint4_complete	21	-	API key, token slip, deteksi pemalsuan slip, QR Code, validasi UTF-8, dan perhitungan varians antarperiode.
Health Check	health, health.integration	1	1	Ketersediaan layanan dan validasi format <i>timestamp</i> .
Total		232	119	351 kasus uji

Contoh kasus uji negatif yang representatif antara lain login tanpa *password* menghasilkan respons **400**, akses oleh peran USER ke rekap penggajian menghasilkan respons **403**, INSERT dengan tipe nilai yang salah menghasilkan respons **400**, DROP TABLE tanpa konfirmasi ganda menghasilkan respons **400**, komponen gaji dengan rumus melingkar menghasilkan respons **422**, serta slip gaji dengan *net amount* yang dipalsukan terdeteksi tidak valid.

6.4 Pengujian Setiap Sprint

Pengujian otomatis ditambahkan secara inkremental mengikuti perkembangan fitur. Setiap sprint menambah kasus uji baru tanpa menghapus pengujian sprint sebelumnya, sehingga kumpulan pengujian juga berfungsi sebagai pengujian regresi. Ringkasan ditampilkan pada Tabel 6.6.

Tabel 6.6 Pengujian Setiap Sprint

Sprint	Periode	Modul yang Diuji	Berkas Uji Ditambahkan
Sprint 1	Maret 2026	Fondasi sistem, meliputi autentikasi, login, JWT, manajemen karyawan dasar, <i>health check</i> , dan eksekusi skema SQL.	auth, employee, health, schema, serta pengujian integrasi terkait.
Sprint 2	April 2026 awal	Fitur <i>multi-tenant</i> dan manajemen akses pengguna, meliputi manajemen perusahaan, manajemen pengguna, hak akses, <i>middleware</i> keamanan, dan <i>switch company</i> .	company, user, permission, middleware, dan switchCompany.integration.
Sprint 3	April 2026 akhir	Audit dan skema dinamis, meliputi <i>activity log</i> , DDL dinamis, dan DML dinamis.	activityLog, ddl, dml, serta pengujian integrasi terkait.

Sprint 4	Mei 2026	Fitur penggajian utama, meliputi mesin formula gaji, komponen gaji, <i>preview</i> gaji karyawan, slip gaji dengan QR Code dan token autentik, rekap penggajian, integrasi HRD, serta validasi rekap.	<code>formulaResolver</code> , <code>sprint4</code> , <code>salaryComponent</code> , <code>employeePayroll</code> , <code>sprint4_complete</code> , <code>payrollDashboardRecap</code> , <code>payrollRecapValidation</code> , <code>integrationApi</code> , <code>payroll.integration</code> , dan <code>payrollRecap.integration</code> .
-----------------	----------	---	---

Hingga Sprint 4, total kasus uji otomatis yang tersedia berjumlah 351 kasus. Jumlah tersebut terdiri atas 232 kasus uji unit dan 119 kasus uji integrasi.

6.5 Hasil Pengujian

6.5.1 Hasil Pengujian Unit

Eksekusi `npm run test` menjalankan seluruh 232 kasus uji unit tanpa memerlukan basis data nyata. Ringkasan hasil eksekusi terakhir ditunjukkan pada Gambar 6.1.

```
Test Suites: 19 passed, 19 total
Tests:      232 passed, 232 total
Snapshots:  0 total
Time:       25.965 s, estimated 33 s
Ran all test suites.
```

Gambar 6.1 Hasil Eksekusi Pengujian Unit Menggunakan Jest

Berdasarkan Gambar 6.1, sebanyak seluruh 232 dari 232 kasus uji unit berhasil lulus tingkat kelulusan 100% tidak terdapat kasus uji unit yang gagal terdapat pada skenario `PATCH /employees/:id/status`. Kegagalan ini dikategorikan sebagai temuan pada *test double*, bukan cacat utama pada produk, karena kegagalan terjadi akibat konfigurasi *mock* yang belum lengkap.

Selain itu, pada eksekusi penuh `npm run test`, terdapat potensi kegagalan tambahan yang bersifat tidak konsisten akibat *handle* basis data yang belum tertutup sempurna saat proses *teardown*. Beberapa *suite* yang terdampak tetap dapat lulus ketika dijalankan secara terpisah. Temuan ini dicatat sebagai masalah keandalan pengujian yang perlu ditindaklanjuti.

6.5.2 Hasil Pengujian Integrasi

Pengujian integrasi dijalankan menggunakan perintah `npm run test:integration`. Pengujian ini terdiri atas 119 kasus uji dalam 9 *suite* dan bertujuan memvalidasi alur *end-to-end* seperti autentikasi, manajemen karyawan, komponen gaji, rekap penggajian, *switch company*, serta operasi skema dinamis terhadap basis data PostgreSQL.

Hasil eksekusi terakhir pengujian integrasi ditunjukkan pada Gambar 6.2.

```
Test Suites: 9 passed, 9 total
Tests:      119 passed, 119 total
Snapshots:  0 total
Time:       29.647 s
```

Gambar 6.2 Hasil Eksekusi Pengujian Integrasi Menggunakan Jest

Berdasarkan Gambar 6.2, seluruh pengujian integrasi berhasil dijalankan. Sebanyak 9 dari 9 *test suite* berhasil lulus, dengan total 119 dari 119 kasus uji integrasi berhasil lulus. Hasil ini menunjukkan bahwa alur integrasi utama antar-komponen sistem, mulai dari permintaan HTTP, pemrosesan *route* dan *controller*, akses *model*, hingga interaksi dengan basis data PostgreSQL, telah berjalan sesuai skenario pengujian yang ditentukan.

Dengan lulusnya seluruh pengujian integrasi, sistem telah tervalidasi tidak hanya pada tingkat unit, tetapi juga pada alur kerja yang melibatkan beberapa komponen secara bersamaan. Hal ini memperkuat keyakinan bahwa fitur-fitur utama seperti autentikasi, pengelolaan karyawan, proses penggajian, rekap payroll, pergantian perusahaan, dan operasi skema dinamis dapat berjalan secara konsisten pada lingkungan pengujian yang telah dikonfigurasi.

6.5.3 Cakupan Kode

Laporan cakupan kode dihasilkan melalui perintah `npm run test:coverage`. Hasil ringkasan metrik cakupan kode ditunjukkan pada Gambar 6.3.

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	37.28	26.6	27.27	38.59	

Gambar 6.3 Ringkasan Cakupan Kode Unit Test

Gambar 6.3 menampilkan ringkasan cakupan kode. Rinciannya ditampilkan pada Tabel 6.7.

Tabel 6.7 Ringkasan Cakupan Kode Unit Test

Metrik	Cakupan	Rasio
Statements	37,28%	1668 / 4097
Branches	26,6%	684 / 2315
Functions	27,27%	155 / 514
Lines	38,59%	1545 / 3686

Beberapa modul inti memiliki cakupan tinggi, antara lain `authMiddleware` sebesar 100%, `errorHandler` sebesar 100%, `payrollValidationModel` sebesar 100%, `formulaResolver` sebesar 97,9%, `activityLogController` sebesar 89,7%, `permissionMiddleware` sebesar 88,9%, `schemaController` sebesar 81,2%, dan `companyController` sebesar 80,4%.

Angka cakupan agregat belum sepenuhnya merepresentasikan keseluruhan pengujian sistem karena laporan coverage tersebut hanya dihitung dari pengujian unit. Beberapa *controller* dan *model* direncanakan untuk divalidasi lebih lanjut melalui pengujian integrasi terhadap basis data nyata, namun hasil integrasi saat ini masih menunjukkan banyak kegagalan sehingga perlu ditindaklanjuti sebelum dapat dijadikan bukti stabilitas sistem secara menyeluruh.

6.5.4 Ringkasan Hasil Pengujian

Ringkasan hasil pengujian ditampilkan pada Tabel 6.9.

Program Studi Teknik Informatika ITB	IF-ITB-IF3250-PPL-2024	Halaman 80/ dari 88 halaman
Template dokumen ini dan informasi yang dimilikinya adalah milik Program Studi Teknik Informatika-ITB, serta bersifat rahasia. Dilarang me-reproduksi dokumen ini tanpa diketahui oleh Program Studi Teknik Informatika ITB.		

Tabel 6.9 Ringkasan Hasil Pengujian

Aspek	Nilai
Total kasus uji otomatis	351 kasus uji
Pengujian unit	232 kasus uji
Hasil pengujian unit	232 lulus, 0 gagal
Persentase kelulusan unit	100%
Pengujian integrasi	119 kasus uji
Hasil pengujian integrasi	119 lulus, 0 gagal
Jumlah <i>suite</i> unit	19 suite
Jumlah <i>suite</i> integrasi	9 suite
Cakupan kode unit	40,71% <i>statements</i> / 41,91% <i>lines</i>
Perangkat pengujian	Jest, ts-jest, Supertest, PostgreSQL
Temuan terbuka	0 temuan

Secara keseluruhan, Aplikasi Penggajian Terintegrasi telah memiliki 351 kasus uji otomatis yang mencakup 232 kasus uji unit dan 119 kasus uji integrasi. Berdasarkan hasil eksekusi terakhir, seluruh kasus uji berhasil lulus, baik pada pengujian unit maupun pengujian integrasi. Dengan demikian, fitur-fitur utama sistem telah tervalidasi melalui pengujian otomatis dan dapat digunakan sebagai dasar untuk menjaga stabilitas aplikasi pada iterasi pengembangan berikutnya.

7 Petunjuk Instalasi Perangkat Lunak

Aplikasi Penggajian Terintegrasi terdiri atas tiga komponen utama, yaitu *backend* sebagai REST API berbasis Express dan TypeScript, *frontend* sebagai antarmuka pengguna berbasis Next.js dan React, serta basis data PostgreSQL sebagai penyimpanan data utama. Bagian ini memandu administrator atau evaluator dalam melakukan instalasi dan menjalankan sistem, baik pada lingkungan lokal maupun server. Seluruh perintah dijalankan dari direktori komponen yang bersangkutan, yaitu *backend/* atau *frontend/*.

7.1 Kebutuhan Lingkungan

Sebelum instalasi, pastikan perangkat memenuhi kebutuhan berikut.

- Node.js versi 20 atau lebih baru beserta npm.
- PostgreSQL versi 14 atau lebih baru yang sudah berjalan.
- Git untuk mengunduh kode sumber.
- Peramban modern (Chrome, Edge, atau Firefox).

7.2 Persiapan Kode Sumber

Unduh kode sumber dari repositori menggunakan Git, lalu masuk ke direktori proyek.

```
git clone
https://gitlab.divusi.co.id/gabungan/aplikasi-penggajian.git
cd aplikasi-penggajian
```

7.3 Konfigurasi Environment

Buat berkas `.env` pada direktori `backend/` dengan templat berikut, lalu sesuaikan nilainya dengan konfigurasi basis data dan kunci rahasia yang digunakan.

```
# backend/.env
DATABASE_URL=postgresql://user:password@localhost:5432/payroll
JWT_SECRET=ganti_dengan_kunci_rahasia_yang_kuat
PAYROLL_SLIP_SECRET=ganti_dengan_kunci_tanda_tangan_slip
PORT=4000
```

Buat pula berkas `.env.local` pada direktori `frontend/` yang mengarahkan aplikasi *frontend* ke alamat *Backend API*.

```
# frontend/.env.local
NEXT_PUBLIC_API_URL=http://localhost:4000/api
```

7.4 Instalasi Backend

Masuk ke direktori *backend*, pasang dependensi, lalu kompilasi kode TypeScript.

```
cd backend
npm install
npm run build
```

7.5 Instalasi Frontend

Pada terminal terpisah, masuk ke direktori *frontend* dan pasang dependensi.

```
cd frontend
npm install
```

7.6 Konfigurasi Basis Data

Jalankan migrasi untuk membentuk skema basis data, kemudian jalankan seed untuk membuat akun superadmin awal. Migrasi dapat dibatalkan dengan perintah `rollback` bila diperlukan.

```
cd backend
npm run db:migrate
npm run db:seed
# rollback bila perlu:
npm run db:rollback
```

7.7 Menjalankan Aplikasi

Jalankan *backend* dan *frontend* pada terminal terpisah.

```
# Terminal 1 (backend)
cd backend
npm run dev
# Terminal 2 (frontend)
cd frontend
npm run dev
```

Setelah keduanya berjalan, akses aplikasi melalui peramban pada alamat <http://localhost:3000>. Dokumentasi *API* tersedia pada <http://localhost:4000/api/docs>.

7.8 Menjalankan Pengujian

Pengujian dapat dijalankan untuk memverifikasi instalasi.

```
cd backend
npm run test
npm run test:integration
npm run test:coverage
```

7.9 Penanganan Masalah Umum

Masalah umum saat instalasi beserta solusinya ditampilkan pada Tabel 7.1.

Tabel 7.1 Penanganan Masalah Umum Instalasi

Gejala	Penyebab	Solusi
Backend gagal terhubung ke basis data.	DATABASE_URL salah atau PostgreSQL belum berjalan.	Periksa kredensial pada .env dan pastikan layanan PostgreSQL aktif pada porta 5432.
Login selalu gagal meski kredensial benar.	JWT_SECRET berbeda atau seed belum dijalankan.	Jalankan npm run db:seed dan pastikan JWT_SECRET konsisten.
Frontend tidak dapat memanggil API.	NEXT_PUBLIC_API_URL salah.	Sesuaikan NEXT_PUBLIC_API_URL pada .env.local dan jalankan ulang npm run dev.
Verifikasi slip selalu tidak valid.	PAYROLL_SLIP_SECRET berubah setelah slip diterbitkan.	Gunakan PAYROLL_SLIP_SECRET yang sama dengan saat penerbitan slip.

8 Panduan Penggunaan Perangkat Lunak

Panduan ini menjelaskan cara penggunaan sistem penggajian bagi seluruh kategori pengguna. Sistem dapat diakses melalui peramban web dengan membuka alamat yang telah dikonfigurasi oleh administrator.

8.1 Login dan Pemilihan Perusahaan

Berlaku untuk Superadmin, Admin, dan User

Buka halaman login sistem. Masukkan username atau email beserta password pada kolom yang tersedia, kemudian klik tombol Masuk. Apabila kredensial tidak valid, sistem menampilkan pesan kesalahan pada bagian atas formulir.

Setelah login berhasil, sistem memeriksa jumlah perusahaan yang terdaftar pada akun pengguna. Apabila akun hanya terdaftar di satu perusahaan, sistem langsung mengarahkan pengguna ke halaman dashboard. Apabila akun terdaftar di lebih dari satu perusahaan, sistem menampilkan halaman Pilih Perusahaan yang memuat daftar

perusahaan beserta peran pengguna di masing-masing perusahaan. Klik kartu perusahaan yang dituju untuk masuk. Sistem akan menerbitkan JWT token baru yang mencakup konteks perusahaan yang dipilih dan mengarahkan pengguna ke dashboard.

Untuk keluar dari sistem, klik tombol Keluar yang tersedia di halaman pemilihan perusahaan atau melalui menu navigasi pada dashboard.

8.2 *Pengelolaan Perusahaan dan Pengguna*

Berlaku untuk Superadmin

8.2.1 *Mendaftarkan Perusahaan Baru*

Buka menu Companies pada dashboard. Klik tombol Tambah Perusahaan untuk membuka formulir pendaftaran. Isi nama perusahaan, kode unik, alamat, nomor telepon, email, dan logo perusahaan. Klik Simpan untuk menyimpan data. Perusahaan yang berhasil didaftarkan akan muncul dalam daftar dan siap digunakan.

8.2.2 *Membuat Akun Pengguna*

Buka menu Users pada dashboard. Klik Tambah User dan isi formulir yang mencakup username, email, password, nama lengkap, dan peran (SUPERADMIN, ADMIN, atau USER). Klik Simpan untuk membuat akun.

8.2.3 *Menugaskan pengguna ke perusahaan*

Pada halaman Companies, pilih perusahaan yang dituju, kemudian klik Assign User. Pilih pengguna dari daftar, tentukan peran dalam perusahaan tersebut, dan atur opsi canSwitchCompany sesuai kebutuhan. Klik Simpan untuk menyelesaikan penugasan.

8.2.4 *Mengelola Integration API Key*

Buka menu Integration pada dashboard. Klik Tambah API Key, isi nama kunci dan tentukan cakupan akses (scopes), misalnya payroll:read. Sistem akan menampilkan kunci API yang dihasilkan sekali saja. Salin dan simpan kunci tersebut di tempat yang aman karena tidak akan ditampilkan kembali. Untuk mencabut akses, klik Revoke pada baris kunci yang bersangkutan.

8.3 *Pengelolaan Data Master*

Berlaku untuk Superadmin dan Admin

8.3.1 *Mengelola Departemen dan Jabatan*

Buka menu Jabatan & Departemen pada dashboard. Untuk menambah departemen, klik Tambah Departemen, isi nama departemen, lalu klik Simpan. Untuk menambah jabatan, klik Tambah Jabatan, isi nama jabatan, pilih departemen yang sesuai, tentukan level hierarki, lalu klik Simpan. Data yang telah disimpan dapat diedit atau dinonaktifkan melalui tombol aksi pada setiap baris.

8.3.2 *Mengelola Employee Grade dan Tunjangan Masa Kerja*

Buka menu Employee Grades pada dashboard. Klik Tambah Grade dan isi kode grade, nama, level hierarki, serta nilai gaji pokok acuan (baseSalary) apabila diperlukan. Setelah grade tersimpan, tambahkan tier tunjangan masa kerja dengan mengklik Tambah Tunjangan Masa Kerja pada baris grade yang bersangkutan. Isi ambang batas masa kerja minimum dalam bulan (minTenureMonths), nilai tunjangan atau formula, serta tanggal berlaku. Sistem akan memilih tier yang sesuai secara otomatis saat kalkulasi gaji berdasarkan masa kerja efektif karyawan.

8.3.3 *Mengelola Master Komponen Gaji*

Buka menu Salary Components pada dashboard. Halaman ini menampilkan daftar komponen gaji di panel kiri dan editor formula di panel kanan.

Untuk menambah komponen baru, klik + Tambah Komponen. Isi kode komponen (digunakan sebagai identifier dalam formula, misalnya GAJI_POKOK), nama tampilan, tipe komponen (Penghasilan atau Potongan), urutan evaluasi, dan centang Aktif apabila komponen harus ikut diperhitungkan dalam payroll. Pada bagian Formula, tulis ekspresi kalkulasi menggunakan variabel yang tersedia di panel Lookup Variabel (misalnya gaji_pokok * 0.1 untuk tunjangan 10% dari gaji pokok). Editor menampilkan pratinjau hasil kalkulasi secara langsung menggunakan data uji standar. Klik Simpan Baru untuk menyimpan komponen.

Untuk mengubah komponen yang sudah ada, klik komponen di panel kiri, lakukan perubahan yang diperlukan, lalu klik Simpan Perubahan. Komponen yang belum disimpan ditandai dengan indikator merah (●) di panel kiri.

Untuk menambah variabel lookup (nilai yang bergantung pada atribut karyawan), klik Lookup Variabel, isi nama variabel, label, field karyawan yang menjadi acuan (matchField), nilai yang dicocokkan (matchValue), dan nilai numerik yang akan digunakan dalam formula. Klik Tambah Lookup untuk menyimpan.

8.3.4 Mengelola Tabel Referensi

Buka menu Reference Data pada dashboard untuk mengelola data tunjangan fungsional (ref_fungsional), tunjangan struktural (ref_struktural), dan tunjangan proyek (ref_proyek). Tambah entri baru dengan mengisi kode, nama, dan nilai nominal, lalu klik Simpan.

8.3.5 Mengonfigurasi Cutoff Penggajian

Pada menu Payroll Run, klik Pengaturan Cut-off. Isi tanggal mulai cutoff (cutoffStartDay, rentang 1–28) dan tanggal selesai cutoff (cutoffEndDay, rentang 1–31), lalu klik Simpan Pengaturan. Konfigurasi ini menentukan rentang tanggal yang digunakan dalam setiap eksekusi penggajian.

8.4 Pengelolaan Data Karyawan

Berlaku untuk Superadmin dan Admin

Buka menu Employees (Master Pegawai) pada dashboard. Daftar karyawan ditampilkan di panel kiri.

8.4.1 Menambah karyawan baru

Klik + Tambah Pegawai. Formulir terbagi ke dalam beberapa bagian. Pada bagian Data Diri, isi username, password, nama lengkap, email, NIK, jabatan, departemen, grade, jenis kelamin, status pernikahan, jumlah anak, dan status karyawan (Kontrak, Tetap, atau Probation). Pada bagian Pengaturan Gaji & Masa Kerja, isi TMT Kerja, TMT Tetap, offset masa kerja (dalam tahun), offset tunjangan masa kerja, dan nilai offset gaji pokok. Centang Gaji Pokok Ditentukan Manual apabila nilai gaji pokok tidak mengacu pada grade, lalu isi nilai gaji pokoknya. Pada bagian Tunjangan Keluarga, isi persentase tunjangan pasangan dan tunjangan anak. Pada bagian Kode Referensi Tunjangan, pilih kode keahlian fungsional, kode posisi struktural, dan kode tunjangan proyek dari daftar yang tersedia. Klik Simpan Perubahan untuk menyimpan data karyawan baru.

8.4.2 Mengedit data karyawan

Pilih karyawan dari panel kiri, kemudian klik Edit pada area detail. Lakukan perubahan yang diperlukan pada bidang yang relevan, lalu klik Simpan Perubahan. Klik Batal untuk membatalkan perubahan.

8.4.3 Mengunggah dokumen karyawan

Pilih karyawan dari panel kiri, kemudian klik tab Dokumen. Pilih tipe dokumen (KTP, KK, NPWP, atau CV) dari dropdown, lalu pilih file yang akan diunggah. Setelah berhasil diunggah, dokumen akan muncul dalam daftar di bawahnya. Klik Preview untuk melihat isi dokumen.

8.4.4 Menyalin komponen gaji ke karyawan

Setelah Master Komponen Gaji perusahaan selesai didefinisikan, kembali ke menu Salary Components. Klik Salin ke Pegawai, pilih karyawan dari dropdown, tentukan apakah komponen dengan kode yang sama akan ditimpa (overwrite), lalu klik Salin Sekarang. Sistem menampilkan ringkasan jumlah komponen yang dibuat, diperbarui, dan dilewati. Langkah ini wajib dilakukan sebelum karyawan dapat diproses dalam payroll run.

8.4.5 Menghapus karyawan

Pilih karyawan dari panel kiri, klik Hapus, dan konfirmasi tindakan pada dialog yang muncul. Data karyawan akan dinonaktifkan dari sistem.

8.5 Pengelolaan Data Kehadiran

Berlaku untuk Superadmin dan Admin

Buka menu Attendance pada dashboard. Untuk menginput data kehadiran secara manual, pilih karyawan dan tanggal yang bersangkutan, kemudian pilih status kehadiran (HADIR, IZIN, SAKIT, ALPHA, atau CUTI) dan isi waktu masuk serta keluar apabila tersedia. Untuk mengimpor data dari mesin fingerprint, gunakan fitur impor massal yang tersedia pada halaman yang sama. Data kehadiran yang telah diinput akan digunakan secara otomatis oleh sistem saat menghitung nilai hari_masuk_kerja dalam formula penggajian periode yang bersangkutan.

8.6 Eksekusi Penggajian Bulanan

Berlaku untuk Superadmin dan Admin

Buka menu Payroll Run pada dashboard. Halaman ini menampilkan riwayat payroll di panel kiri dan rincian payroll yang dipilih di panel kanan.

8.6.1 Menjalankan payroll baru

Klik Run Payroll Baru. Sistem menampilkan prompt untuk mengisi periode dalam format YYYY-MM (misalnya 2025-01). Klik OK untuk memulai proses. Sistem akan melakukan kalkulasi secara otomatis untuk seluruh karyawan aktif. Apabila terdapat karyawan yang belum memiliki komponen gaji aktif, sistem menampilkan pesan kesalahan yang menyebutkan nama karyawan tersebut. Lakukan penyalinan komponen terlebih dahulu melalui menu Salary Components, kemudian jalankan ulang payroll.

Setelah proses selesai, rincian payroll ditampilkan di panel kanan, mencakup status, tanggal pemrosesan, total netto, serta tabel per karyawan yang memuat nama, departemen, total bruto, total potongan, dan take home pay. Payroll yang baru dieksekusi memiliki status DRAFT.

8.6.2 Melihat rincian payroll periode sebelumnya

Klik nama periode pada panel riwayat di sebelah kiri untuk memuat dan menampilkan rinciannya di panel kanan.

8.7 Rekap dan Validasi Penggajian

Berlaku untuk Superadmin dan Admin

Buka menu Payroll Recap pada dashboard. Pilih periode penggajian dari dropdown di bagian atas. Sistem menampilkan ringkasan dalam empat kartu: total penghasilan, total potongan, total take-home, dan rata-rata per karyawan.

Di bawah kartu ringkasan, terdapat dua panel: rekap per departemen (ditampilkan dengan proporsi bar) dan rekap per komponen (dipisahkan antara penghasilan dan potongan). Di bagian paling bawah, tabel rekap per karyawan menampilkan seluruh karyawan beserta nilai gross, potongan, dan net masing-masing. Tabel ini dapat difilter berdasarkan nama karyawan atau departemen, serta diurutkan berdasarkan kolom yang diinginkan dengan mengklik kepala kolom.

8.7.1 Memvalidasi integritas rekap

Klik tombol Validasi Integritas pada toolbar. Sistem menjalankan pemeriksaan dan menampilkan laporan di bagian atas halaman. Setiap pemeriksaan ditandai dengan centang (✓) apabila lulus atau silang (✗) apabila gagal, disertai keterangan detail dan selisih nilai apabila ada. Lakukan koreksi data dan jalankan ulang payroll apabila terdapat pemeriksaan yang gagal sebelum melanjutkan ke finalisasi.

8.7.2 Mengekspor rekap

Klik Export CSV untuk mengunduh data rekap dalam format CSV. Klik Export Excel untuk mengunduh dalam format XLS yang dapat dibuka di Microsoft Excel. Klik Cetak/PDF untuk membuka pratinjau cetak di jendela baru yang kemudian dapat disimpan sebagai PDF melalui dialog cetak peramban.

8.8 Analisis Variansi Penggajian

Berlaku untuk Admin dan Superadmin

Buka menu Payroll Variance pada dashboard. Tentukan rentang periode analisis dengan mengisi periode awal (from) dan periode akhir (to) dalam format YYYY-MM. Sistem menampilkan perbandingan nilai gross, potongan, dan net antar periode secara berurutan, termasuk delta nominal dan persentase perubahan dibandingkan periode sebelumnya. Data variansi per komponen gaji juga tersedia untuk mengidentifikasi komponen mana yang mengalami perubahan signifikan.

8.9 Slip Gaji Karyawan (Self-Service)

Berlaku untuk User

Setelah login dan memilih perusahaan aktif, buka menu Slip Gaji Saya pada dashboard. Sistem menampilkan tabel riwayat slip gaji yang mencantumkan periode, status payroll, total penghasilan, total potongan, dan take home pay untuk setiap periode yang tersedia. Klik tombol Lihat pada baris periode yang ingin dilihat untuk memuat rincian slip. Rincian slip menampilkan informasi berikut.

Bagian kiri memuat identitas karyawan (nama, jabatan, departemen), tabel penghasilan per komponen beserta subtotal, tabel potongan per komponen beserta subtotal, dan nilai bersih yang diterima (take home pay) yang ditampilkan secara menonjol di bagian bawah. Bagian kanan memuat kode QR verifikasi yang dapat dipindai oleh pihak lain untuk memverifikasi keaslian slip, beserta tautan URL verifikasi yang dapat dibagikan.

Untuk mengunduh slip, klik Export PDF untuk membuka pratinjau cetak yang dapat disimpan sebagai PDF, atau klik Export Excel untuk mengunduh data slip dalam format XLS.

8.10 Verifikasi Keaslian Slip Gaji

Berlaku untuk siapa saja (termasuk pihak ketiga tanpa akun)

Halaman verifikasi slip dapat diakses tanpa login melalui alamat /slip-verify?token=... yang termuat dalam tautan pada QR code slip gaji. Pindai QR code pada slip gaji menggunakan kamera perangkat untuk membuka tautan verifikasi secara otomatis, atau salin tautan URL verifikasi dari slip dan buka di peramban. Sistem akan memverifikasi tanda tangan kriptografis token dan mencocokkan nilainya dengan data aktual di basis data.

Apabila slip valid, halaman menampilkan tanda centang hijau (✓) dengan label Slip Otentik, disertai informasi nama perusahaan, nama karyawan, periode, nilai bersih yang diterima, status payroll, dan tanggal penerbitan. Apabila slip tidak valid atau terindikasi dipalsukan, halaman menampilkan tanda silang merah (✗) dengan label Slip Tidak Valid beserta keterangan penyebabnya.

8.11 Pengelolaan Log Aktivitas

Berlaku untuk Superadmin dan Admin

Buka menu Activity Log pada dashboard. Halaman ini menampilkan seluruh aktivitas pengguna dalam perusahaan aktif secara kronologis. Gunakan panel Filter untuk mempersempit tampilan berdasarkan rentang tanggal, jenis aksi (LOGIN, CREATE, UPDATE, DELETE, dan lain-lain), atau menu terkait. Klik tombol Export untuk mengunduh data log aktivitas dalam format yang tersedia.

Lampiran

Component diagram:

<https://www.figma.com/board/EILYNlIXvMDAB2xpfY2mxz/Payroll-System---Architecture---Class-Diagram?node-id=0-1&t=1EIO6UX2Q3qnnBbN-1>

Link Deployment:

<https://bukujurnal.com/pay2/>

Deployment Diagram:

<https://www.figma.com/board/aEU8hONBIqs6Fmju4d6U7Y/Deployment-Diagram-%E2%80%94bukujurnal.com-pay2?node-id=0-1&t=8S0rwGsBR8jlb8QJ-1>

Sequence Diagram:

<https://drive.google.com/file/d/19o1kXAM8h1O-14DS14I84f1qmIwLKZy0/view?usp=sharing>

<https://drive.google.com/file/d/1W8g-erzY23RaWabEhfDHhbSv6kzgMsDi/view?usp=sharing>

<https://drive.google.com/file/d/17KKGSzCpbSbW4seDXB3R2XXg5jOdO3fg/view?usp=sharing>

<https://drive.google.com/file/d/1DXvSHxW9fiZw1kVwB0JZYwuQs56QzhCS/view?usp=sharing>